

**CDAC - MUMBAI
ADS ASSIGNMENT
May 2021**

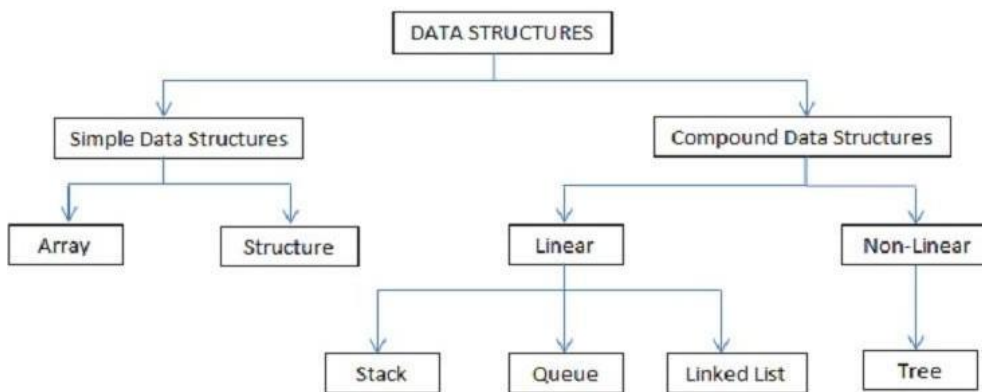
Q 1) What do you mean by Data Structures?

ANS :

- Definition

Data structure refers to methods of organizing units of data within larger data sets. Achieving and maintaining specific data structures help improve data access and value. Data structures also help programmers implement various programming tasks.

- Diagram



Functions of data structures?

Data structures handle four main functions for us:

- Inputting information
- Processing information
- Maintaining information
- Retrieving information

•Inputting is largely concerned with how the data is received. What kind of information can be included? Will the new data be added to the beginning, end, or somewhere in the middle of the existing data? Does an existing point of data need to be updated or destroyed?

•Processing gets at the way that data is manipulated in the data structure. This can occur concurrently or as a result of other processes that data structures handle. How does existing data that has been stored need to change to accommodate new, updated, or removed data?

•Maintaining is focused on how the data is organized within the structure. Which relationships need to be maintained between pieces of data? How much memory must the system reserve (allocate) to accommodate the data?

•Retrieving is devoted to finding and returning the data that is stored in the structure. How can we access that information again? What steps does the data structure need to take to get the information back to us?

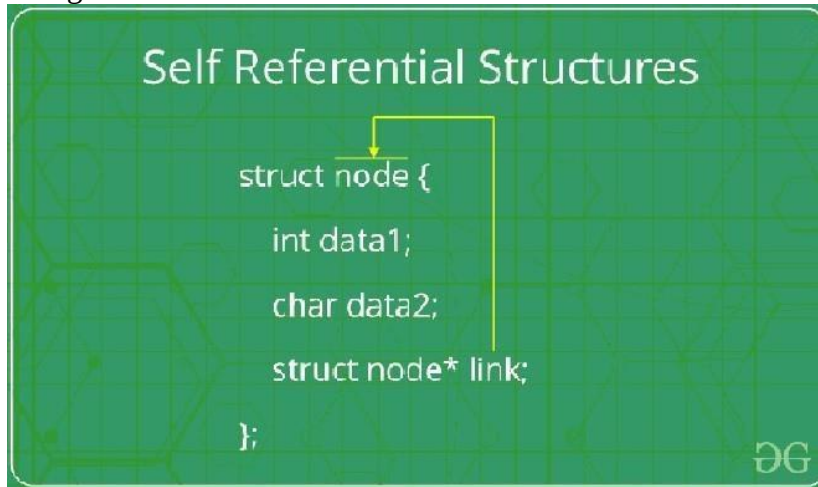
Q 2). Explain self-referential structure? Explain its need ?

ANS :-

- Definition

Self Referential structures are those structures that have one or more pointers which point to the same type of structure, as their member.

- Diagram



- Types of Self Referential Structures

- Self Referential Structure with Single Link

- Self Referential Structure with Multiple Links

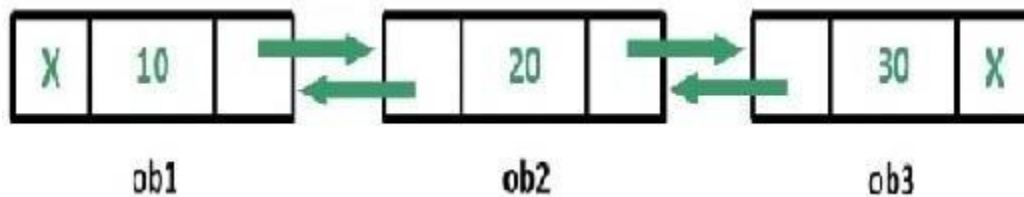
Self Referential Structure with Single Link:

•These structures can have only one self-pointer as their member. The following example will show us how to connect the objects of a self-referential structure with the single link and access the corresponding data members. The connection formed is shown in the following figure.



Self Referential Structure with Multiple Links:

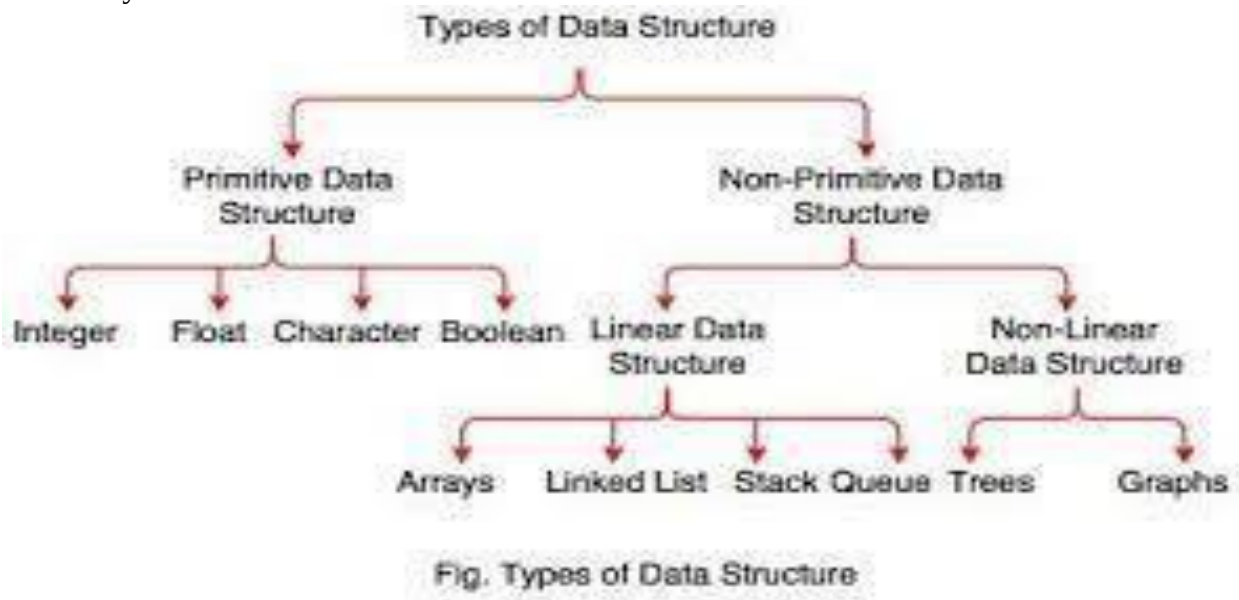
•Self referential structures with multiple links can have more than one self-pointers. Many complicated data structures can be easily constructed using these structures. Such structures can easily connect to more than one nodes at a time. The following example shows one such structure with more than one links. The connections made in the above example can be understood using the following figure.



Q 3). List out the areas in which data structures are applied extensively? Ans ->

Data Structure :

A data structure is a particular way of organizing data in a computer so that it can be used effectively.



There are mainly two types :

Linear Data structure: Here the Data elements are organised in a sequence of somemanner.

Non-linear Data structure: Here the data is ordered in any arbitrary order and not in a sequence

Some common data structures are :

Linked list

Arrays

Stacks

Queues

Binary trees

Hash tables

Areas of Application :

- Data structures are used in any program or software.
- They are used in the areas of
- Compiler Design
- Operating System
- DBMS
- Graphics
- Simulation
- Numerical Analysis
- Artificial Intelligence

Real Time Examples of Data structure :

1. You have to store social network “feeds”. You do not know the size, and things may need to be dynamically added.
2. You need to store undo/redo operations in a word processor.
3. You need to evaluate an expression (i.e., parse).
4. You need to store the friendship information on a social networking site. I.e., who is friends with who.
5. You need to store an image (1000 by 1000 pixels) as a bitmap.
6. To implement printer spooler so that jobs can be printed in the order of their arrival.
7. To implement back functionality in the internet browser.
8. To store the possible moves in a chess game.
9. To store a set of fixed key words which are referenced very frequently.
10. To store the customer order information in a drive-in burger place. (Customers keep on coming and they have to get their correct food at the payment/food collection window.)
11. To store the genealogy information of biological species

Q 4). Write a binary search algorithm with and without recursion? Ans.

1) Definition:

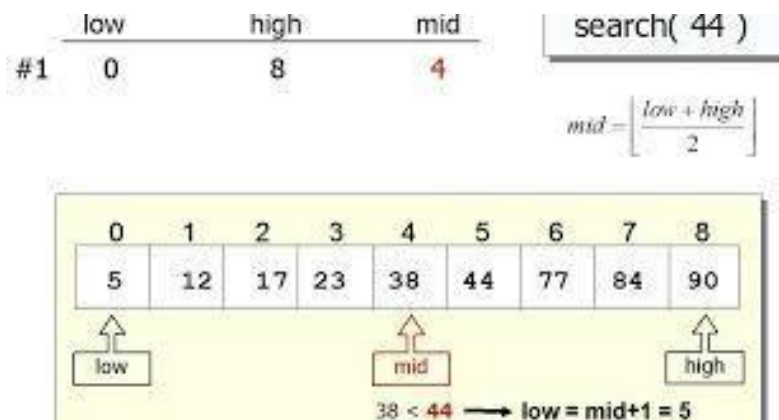
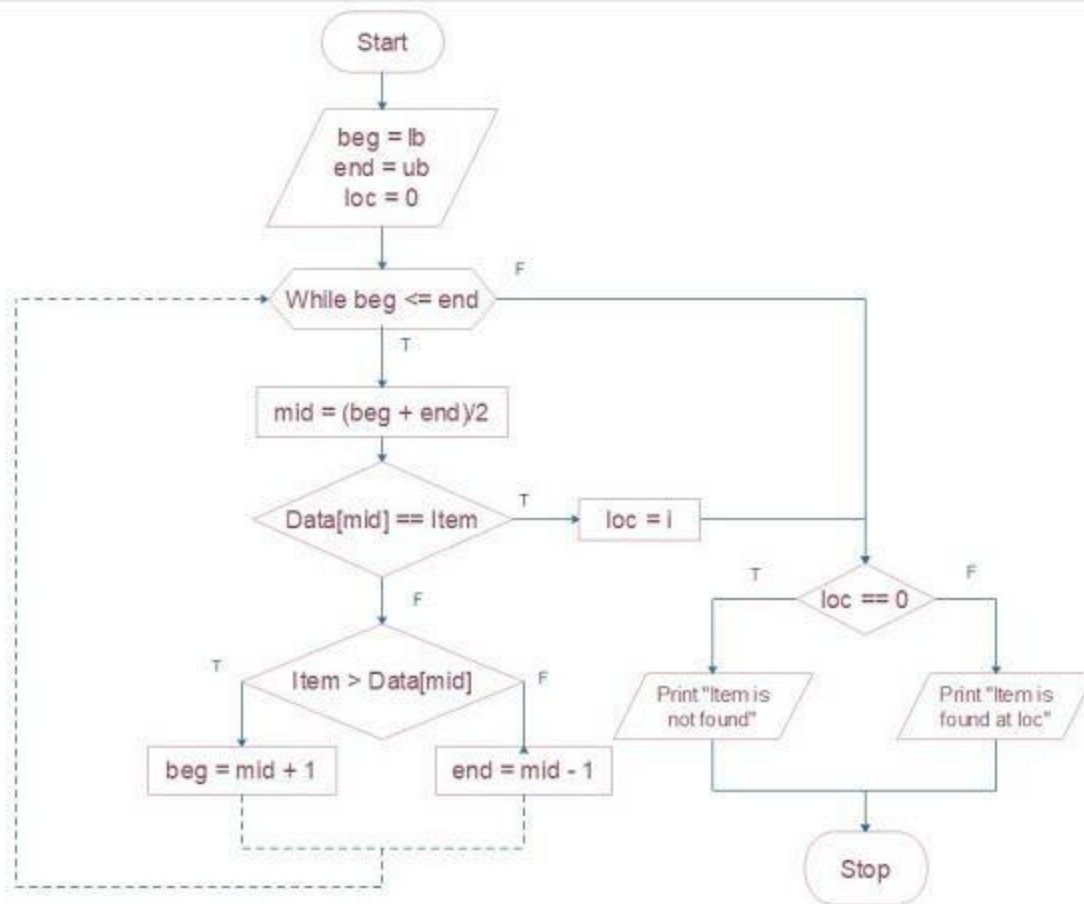
i) Binary search:

Search a sorted array by repeatedly dividing the search interval in half. Begin with an interval covering the whole array. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half. Otherwise narrow it to the upper half. Repeatedly check until the value is found or the interval is empty.

ii) Recursion:

When a function is called within the same function, it is known as recursion in C. The function which calls the same function is known as a recursive function. Recursion is defined as defining anything in terms of itself

2) Diagram:



3)Explanation :

A) Binary search with Recursion:

- 1 .Compare x with the middle element.
 - 2 .If x matches with the middle element, we return the mid index.
 3. Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element.
- So we recur for the right half. Else (x is smaller) recur for the left half.

a) Code Example:

```

class BinarySearch
{
int binarySearch(int arr[], int l, int r, int x)
{
if (r >= l)
{
int mid = l + (r - l) / 2;
if (arr[mid] == x)
return mid;
if (arr[mid] > x)
return binarySearch(arr, l, mid - 1, x);

return binarySearch(arr, mid + 1, r, x);
}
return -1;
}
// Driver method to test above public
static void main(String args[])
{
BinarySearch ob = new BinarySearch();int
arr[] = { 2, 3, 4, 10, 40 };
int n = arr.length;
int x = 10;
int result = ob.binarySearch(arr, 0, n - 1, x);if
(result == -1) System.out.println("Element not
present"); else
System.out.println("Element found at index " + result);
}
}

```

B) Binary Search without Recursion:

In a binary search algorithm, you first find the middle element of the array and compare that with the number you are searching. If it's equal then you return true or index of that number and your binary search is complete but if it doesn't match then you divide the array in two-part based upon whether the middle element is greater than or less than the key value, which you are searching.

If the key (target value) is greater than the middle element then search values in the second half of the array because the array is sorted in increasing order. Similarly, if the key is lower than the middle element it means it's in the first part of the array.

After each iteration, you rule out half of the array elements. This means if you have 100 elements in the array then you need $O(\log_2 100)$

b) Code Example: Binary Search without Recursion

```
import java.util.Scanner;
public class BinarySearch
{
    public static void main(String[] args)
    {
        Scanner commandReader = new Scanner(System.in);
        System.out.println("Welcome to Java Program to perform binary search on int array");
        System.out.println("Enter total number of elements");
        int length = commandReader.nextInt();
        int[] input = new int[length];
        System.out.printf("Enter %d integers %n", length);
        for (int i = 0; i < length; i++)
        {
            input[i] = commandReader.nextInt();
        }
        System.out.println("Please enter number to be searched in array (sorted order)");
        int key = commandReader.nextInt();
        int index = performBinarySearch(input, key);
        if (index == -1)
        {
            System.out.printf("Sorry, %d is not found in array %n", key);
        }
        else
        {
            System.out.printf("%d is found in array at index %d %n", key, index);
        }
        commandReader.close();
    }
    public static int performBinarySearch(int[] input, int number)
    {
        int low = 0; int high = input.length - 1;
        while (high >= low)
        {
            int middle = (low + high) / 2;
            if (input[middle] == number)
            {
                return middle;
            }
            else if (input[middle] < number)
            {
                low = middle + 1;
            }
        }
    }
}
```

```

else if (input[middle] > number)
{
    high = middle - 1;
}
}
return -1;
}
}

```

5) Real Time example:

i) Dictionary:

English contains thousands of words. We cannot possibly remember every single word. So, we use dictionaries to help us find the meaning of different words. A dictionary is basically a list of words of a language in alphabetical order.

ii) Height of Students:

Suppose you require some students for annual function, for some drama, or sports-related activity. The task demands someone who is taller than 5 inches. The problem is most students aren't sure about their height. You have a measuring instrument but measuring the height of each student is time-consuming. We can optimize the process using a binary search.

iii) Library:

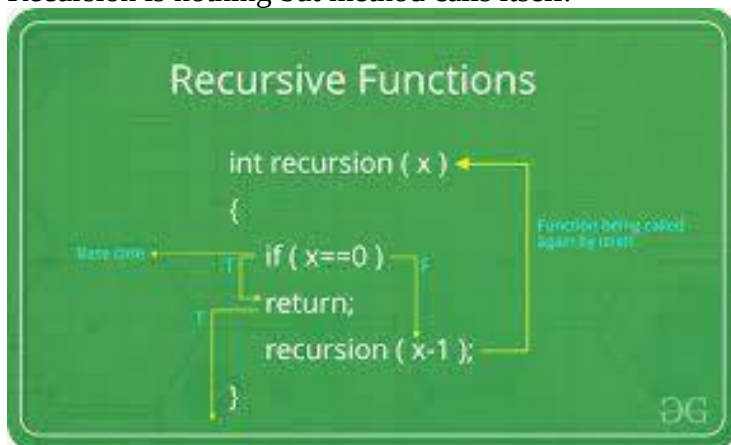
A library contains thousands of books. Searching for a book linearly is not feasible. Books in a library are generally arranged in alphabetical order or by some integer code. Therefore, instead of linearly checking each and every book in the library, we can simply use a binary search to find the book.

iv) University:

Many universities or colleges ask for certain documents during admission. The documents are generally stored in an ordered manner on the basis of student roll number. Using a linear search to find student documents can be cumbersome and inefficient. The university staff can simply use binary search and access the data.

Q 5). Explain the data structures used to perform recursion? Ans ->

Recursion is nothing but method calls itself.



Stack is used to perform the recursion. Because of its LIFO (Last In First Out) property it remembers its 'caller' so knows whom to return when the function has to return. Recursion makes use of system stack for storing the return addresses of the function calls.

Q 6). How to write a program to implement merge sort?ANS

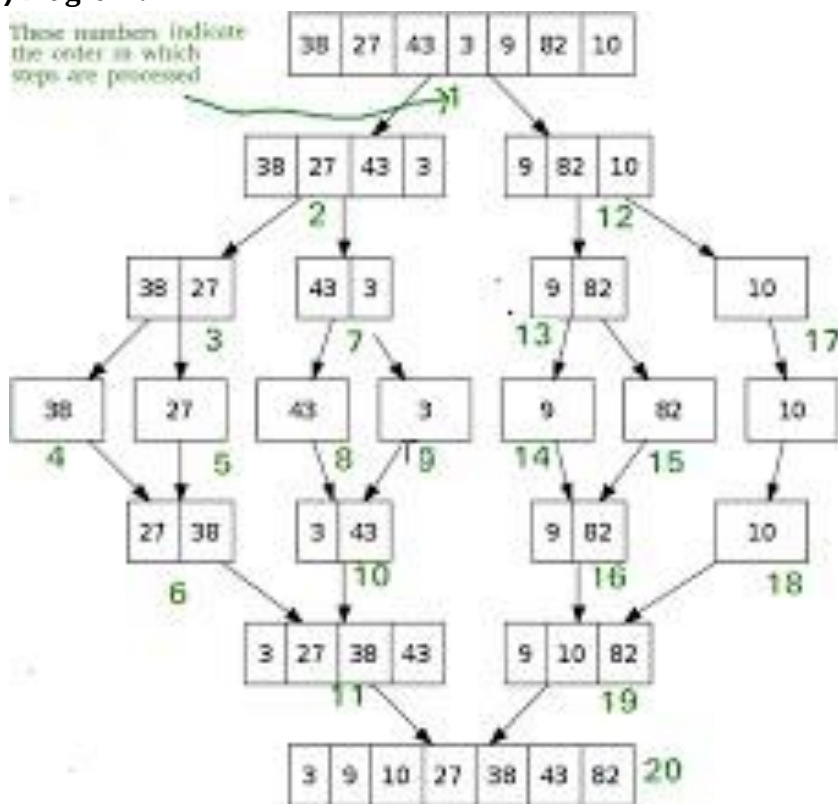
:

1)Definition:

Merge sort:

A sort algorithm that splits the items to be sorted into two groups, recursively sorts each group, and merges them into a final, sorted sequence. Run time is $\Theta(n \log n)$

2)Diagram:



2)Explanation:

Find the middle point to divide the array into two halves:middle

$$m = l + (r-l)/2$$

Call mergeSort for first half:

Call mergeSort(arr, l, m)

Call mergeSort for second half:

Call mergeSort(arr, m+1, r)

Merge the two halves sorted in step 2 and 3:

Call merge(arr, l, m, r)

In step1, we need to divide the array into two sub-arrays. Here the first sub-array will contain four elements (such as 38, 27, 43, and 3) and the other sub-array will contain 3 elements such as (9, 82, and 10).

The first subarray which contains 38, 27, 43, and 3 is further divided into two subarrays (38, 27) and (43, 3). The subarray which contains (38, 27) is further divided into (28) and (27). Similarly, the subarray which contains (43, 3) will be divided into subarrays (43) and (3). At this point, the division will stop as each sub-array contains a single element.

The second subarray which contains 9, 82, and 10 is further divided into two subarrays. One sub-array with two elements i.e. 9 and 82 and the other sub-array with a single element i.e. 10. Further, the subarray which contains (9, 82) is divided into (9) and (82). At this point as each sub-array contains a single element so the division will stop here.

At the end of the step1, we have seven subarrays and each one containing a single element in it. In step2, we need to merge the sub-arrays. The Subarrays (38) and (27) will merge together to form a sorted subarray i.e. (27, 38). Similarly, the other subarrays (43) and (3) will merge together to form a sorted subarray (3, 43) and the subarrays (9) and (82) will merge together to form a sorted subarray (9, 82). In this step, we will keep 10 as it is.

Now the sub-arrays (27, 38) and (43, 3) will merge together to form a sorted subarray i.e. (3, 27, 38, and 43). Similarly, the other two subarrays (9, 82) and 10 merge together to form a sorted subarray (i.e. 9, 10, and 82).

Now, we have two sub-arrays i.e. (3, 27, 38, and 43) and (9, 10, and 82) will merge together to finally produce the sorted array such as (3, 9, 10, 27, 38, 43, and 82).

3) Code Example:

```
class MergeSort
{
    void merge(int arr[], int l, int m, int r)
    {
        int n1 = m - l + 1;
        int n2 = r - m;
        int L[] = new int[n1];
        int R[] = new int[n2];
        for (int i = 0; i < n1; ++i)
            L[i] = arr[l + i];
        for (int j = 0; j < n2; ++j)
            R[j] = arr[m + 1 + j];
        int i = 0, j = 0;
        int k = l;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k] =
                    L[i]; i++;
            }
            else {
                arr[k] = R[j];
                j++;
            }
        }
    }
}
```

```

        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void sort(int arr[], int l, int r)
{
    if (l < r) {
        // Find the middle point
        int m = l + (r-l)/2;

        // Sort first and second halves
        sort(arr, l, m);
        sort(arr, m + 1, r);
        // Merge the sorted halves
        merge(arr, l, m, r);
    }
}

static void printArray(int arr[])
{
    int n = arr.length;
    for (int i = 0; i < n; ++i)
        System.out.print(arr[i] + " ");
    System.out.println();
}

// Driver code
public static void main(String args[])
{
    int arr[] = { 38, 27, 43, 3, 9, 82, 10 };

    System.out.println("Given Array");
    printArray(arr);

    MergeSort ob = new MergeSort();
    ob.sort(arr, 0, arr.length - 1);
    System.out.println("\nSorted array"); printArray(arr);
}
}

```

5)Real time Example:

- Merge Sort is useful for sorting linked lists in $O(n \log n)$ time.
- Merge sort can be implemented without extra space for linked lists.
- Merge sort is used for counting inversions in a list.
- Merge sort is used in external sorting.

Q 7). How to write a program to implement quick sort?

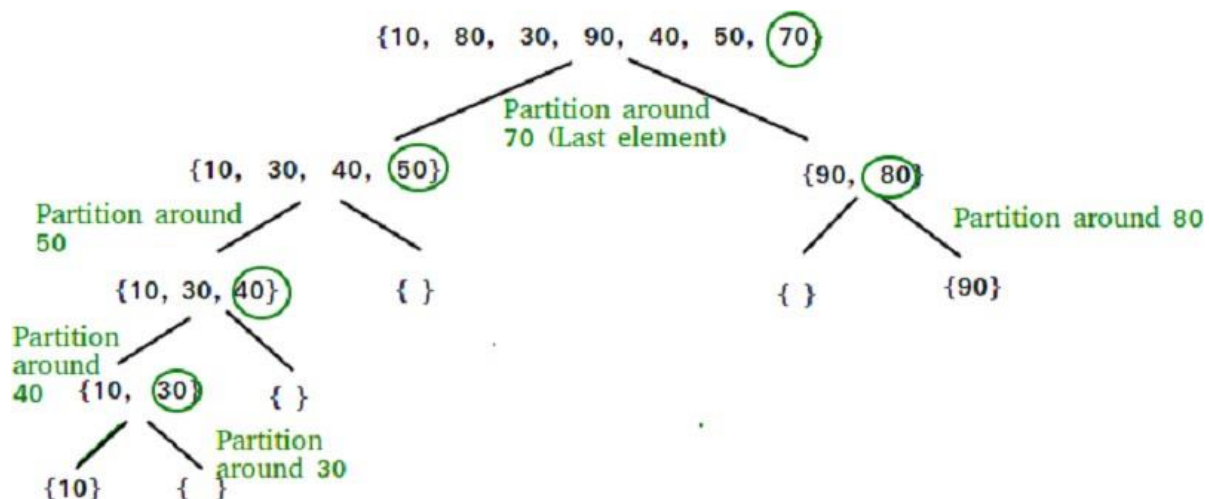
Ans:-

QuickSort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array around the picked pivot.

There are many different versions of quickSort that pick pivot in different ways.

- Always pick first element as pivot.
- Always pick last element as pivot (implemented below)
- Pick a random element as pivot.
- Pick median as pivot.

The key process in quickSort is partition(). Target of partitions is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x. All this should be done in linear time.



Logic of partition function: The logic is simple, we start from the leftmost element and keep track of index of smaller (or equal to) elements as i. While traversing, if we find a smaller element, we swap current element with arr[i]. Otherwise we ignore current element.

Illustration of partition() :

arr[] = {10, 80, 30, 90, 40, 50, 70}

Indexes: 0 1 2 3 4 5 6

low = 0, high = 6, pivot = arr[h] = 70

Initialize index of smaller element, i = -1

Traverse elements from j = low to high-1

j = 0 : Since arr[j] <= pivot, do i++ and swap(arr[i], arr[j]) i =

0

arr[] = {10, 80, 30, 90, 40, 50, 70} // No change as i and j
// are same

j = 1 : Since arr[j] > pivot, do nothing

// No change in i and arr[]

j = 2 : Since arr[j] <= pivot, do i++ and swap(arr[i], arr[j]) i =

1

arr[] = {10, 30, 80, 90, 40, 50, 70} // We swap 80 and 30

j = 3 : Since arr[j] > pivot, do nothing

// No change in i and arr[]

j = 4 : Since arr[j] <= pivot, do i++ and swap(arr[i], arr[j]) i =

2

arr[] = {10, 30, 40, 90, 80, 50, 70} // 80 and 40 Swapped

j = 5 : Since arr[j] <= pivot, do i++ and swap arr[i] with arr[j] i =

3

arr[] = {10, 30, 40, 50, 80, 90, 70} // 90 and 50 Swapped

We come out of loop because j is now equal to high-1.

Finally we place pivot at correct position by swapping

arr[i+1] and arr[high] (or pivot)

arr[] = {10, 30, 40, 50, 70, 90, 80} // 80 and 70 Swapped

Now 70 is at its correct place. All elements smaller than 70 are before it and all elements greater than 70 are after it.

Code:

// Java implementation of QuickSort

import java.io.*;

class GFG{

// A utility function to swap two elements

static void swap(int[] arr, int i, int j)

{

int temp = arr[i];

arr[i] = arr[j];

arr[j] = temp;

}

```

/* This function takes last element as pivot, place the
pivot element at its correct position in sorted array, and
places all smaller (smaller than pivot) to left of pivot
and all greater elements to right
of pivot */

```

```

static int partition(int[] arr, int low, int high)
{

```

```

    // pivot
    int pivot = arr[high];

```

```

    // Index of smaller element and
    // indicates the right position
    // of pivot found so far
    int i = (low - 1);

```

```

    for(int j = low; j <= high - 1; j++)
    {

```

```

        // If current element is smaller
        // than the pivot
        if (arr[j] < pivot)
        {

```

```

            // Increment index of
            // smaller element
            i++;
            swap(arr, i, j);

```

```

        }
    }
    swap(arr, i + 1, high);
    return (i + 1);
}

```

```

/* The main function that implements QuickSort
arr[] --> Array to be sorted,
low --> Starting index,
high --> Ending index
*/

```

```

static void quickSort(int[] arr, int low, int high)
{
    if (low < high)
    {

```

```

        // pi is partitioning index, arr[p]
        // is now at right place

```

```

        int pi = partition(arr, low, high);

        // Separately sort elements before
        // partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to print an array
static void printArray(int[] arr, int size)
{
    for(int i = 0; i < size; i++)
        System.out.print(arr[i] + " ");

    System.out.println();
}

// Driver Code
public static void main(String[] args)
{
    int[] arr = { 10, 7, 8, 9, 1, 5 };
    int n = arr.length;

    quickSort(arr, 0, n - 1);
    System.out.println("Sorted array: ");
    printArray(arr, n);
}
}

```

Time complexity:

Class	Sorting algorithm
Worst-case performance	$O(n^2)$
Best-case performance	$O(n \log n)$ (simple partition) or $O(n)$
Average performance	$O(n \log n)$

Real life examples:

Commercial Computing is used in various government and private organizations for the purpose of sorting various data like sorting files by name/date/price, sorting of students by their roll no., sorting of account profile by given id, etc.

- The sorting algorithm is used for information searching and as Quicksort is the fastest algorithm so it is widely used as a better way of searching.

- It is used everywhere where a stable sort is not needed.
- Quicksort is a cache-friendly algorithm as it has a good locality of reference when used for arrays.
- It is tail -recursive and hence all the call optimization can be done.
- It is an in-place sort that does not require any extra storage • memory.
- It is used in operational research and event-driven simulation.
- Numerical computations and in scientific research, for accuracy in calculations most of the efficiently developed algorithm uses priority queue and quick sort is used for sorting.
- Variants of Quicksort are used to separate the Kth smallest or largest elements.
- It is used to implement primitive type methods.
- If data is sorted then the search for information became easy and efficient.

Q 8) Sorting is not possible by using which of the following methods? (Insertion, Selection, Exchange, Deletion)

ANS :-

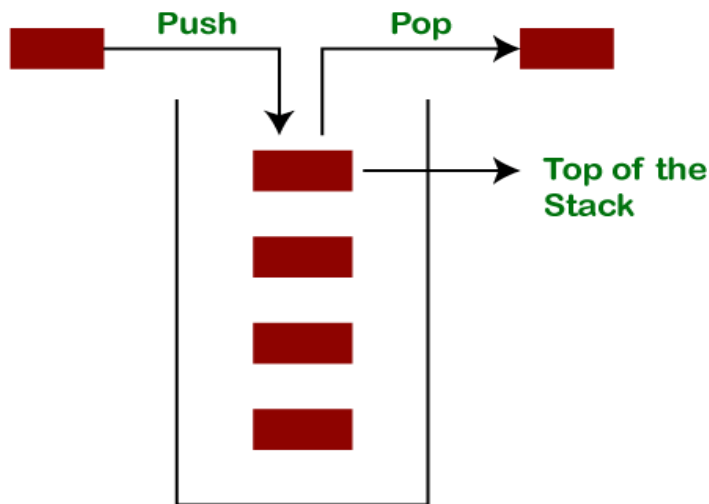
Sorting is not possible in Deletion. Using insertion we can perform insertion sort, using selection we can perform selection sort, using exchange we can perform the bubblesort (and other similar sorting methods). But no sorting method can be done just using deletion.

Q 9. Explain stack? Which are the applications of stack?

Definition: A Stack is a linear data structure that follows the LIFO (Last-In-First-Out) principle. A stack can be defined as a container in which insertion and deletion can be done from the one end known as the top of the stack.

Some key points related to stack:

- It is called as stack because it behaves like a real-world stack, piles of books, etc.
- A Stack is an abstract data type with a predefined capacity, which means that it can store the elements of a limited size.
- It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO (Last In First Out).



Standard Stack Operations:

- **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty():** It determines whether the stack is empty or not.
- **isFull():** It determines whether the stack is full or not.'
- **peek():** It returns the element at the given position.
- **count():** It returns the total number of elements available in a stack.
- **change():** It changes the element at the given position.
- **display():** It prints all the elements available in the stack.

Code:(Stack Implementation using Array)class

```
stackMethods{
private int size;
private int arr[];
int top;
stackMethods(int size){
```

```
this.size=size;

arr=new int[size];

top=-1;

}

void push(int a) {

if(isFull()) {

System.out.println("Stack is full");

return;

}

arr[++top]=a;

}

int pop() {

if(top== -1) { System.out.println("Stack is

Empty");

}

return top--;

}

boolean isEmpty() {

if(top== -1)

return true;

return false;

}
```

```
boolean isFull(){  
  
    if(top==size-1)  
  
        return true;  
  
        return false;  
  
}  
  
int peek() {  
  
    return arr[top];  
  
}  
  
void display() {  
  
    for(int i=0;i<=top;i++) {  
  
        System.out.println(arr[i]);  
  
    }  
  
}  
  
}
```

Real Time Examples:

- Expression evaluation
- Backtracking (game playing, finding paths, exhaustive searching)
- Memory management, run-time environment for nested language features.
- Back/Forward on browsers are performed using stacks.
- An "undo" mechanism in text editors; this operation is accomplished by keeping all text changes in a stack.

Q 10 . Convert a given infix expression into postfix and then evaluate it. (Write code)

ANS :-

The **infix** and **postfix** expressions can have the following operators: '+', '-', '%', '*', '/' and alphabets from a to z. The precedence of the operators (+, -) is lesser than the precedence of operators (*, /, %). Parenthesis has the highest precedence and the expression inside it must be converted first.

The stack is used to store the operators and parenthesis to enforce the precedence Start parsing the expression from left to right.

Eg. Infix Expression: $a/2+c*d-e*(f*g)$

Postfix Expression: $abc/de+*+f-$

Code to convert Infix to postfix Expression:import

```

java.io.*;
class Stack
{
char a[]=new char[100];
int top=-1;
void push(char c)
{
try
{
a[++top]= c;
}
catch(StringIndexOutOfBoundsException e)
{
System.out.println("Stack full, no room to push, size=100");System.exit(0);
}
}
char pop()
{
return a[top--];
}
boolean isEmpty()
{
return (top== -1)?true:false;
}
char peek()
{
return a[top];

```

```

}
}
public class InfixToPostfix
{
    static Stack operators = new Stack();
    public static void main(String argv[]) throws IOException
    {
        String infix;
        //create an input stream object
        BufferedReader keyboard = new BufferedReader (new
        InputStreamReader(System.in));
        //get input from user
        System.out.print("\nEnter the infix expression you want to convert: ");infix =
        keyboard.readLine();
        //output as postfix
        System.out.println("Postfix expression for the given infix expression is:" +
        toPostfix(infix));
    }
    private static String toPostfix(String infix)
    //converts an infix expression to postfix
    {
        char symbol;
        String postfix = "";
        for(int i=0;i<infix.length();++i)
        //while there is input to be read
        {
            symbol = infix.charAt(i);
            //if it's an operand, add it to the stringif
            (Character.isLetter(symbol)) postfix =
            postfix + symbol;
            else if (symbol=='(')
            //push (
            {
                operators.push(symbol);
            }
            else if (symbol==')')
            //push everything back to (
            {
                while (operators.peek() != '(')

```

```

{
postfix = postfix + operators.pop();
}
operators.pop();      //remove '('
}
else
//print operators occurring before it that have greater precedence
{
while (!operators.isEmpty() && !(operators.peek()=='(') && prec(symbol) <=
prec(operators.peek()))
postfix = postfix + operators.pop();
operators.push(symbol);
}
}
while (!operators.isEmpty()) postfix =
postfix + operators.pop();return
postfix;
}
static int prec(char x)
{
if (x == '+' || x == '-')
return 1;
if (x == '*' || x == '/' || x == '%')
return 2;
return 0;
}
}

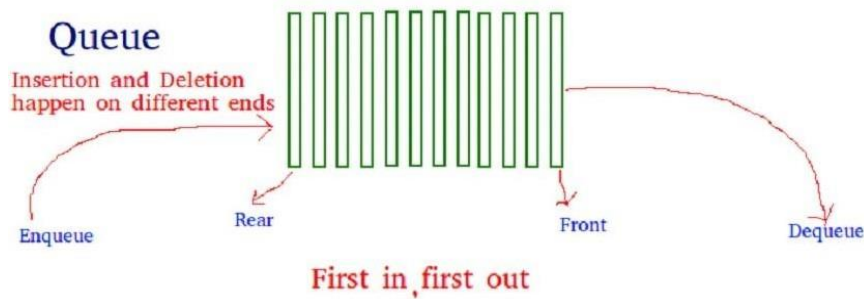
```

Q 11. Explain queue? Which are the types of queue ?

Definition:

A queue in the data structure can be considered similar to the queue in the real-world. A queue is a data structure in which whatever comes first will go out first. It follows the FIFO(First-In-First-Out) policy.

DIAGRAM:



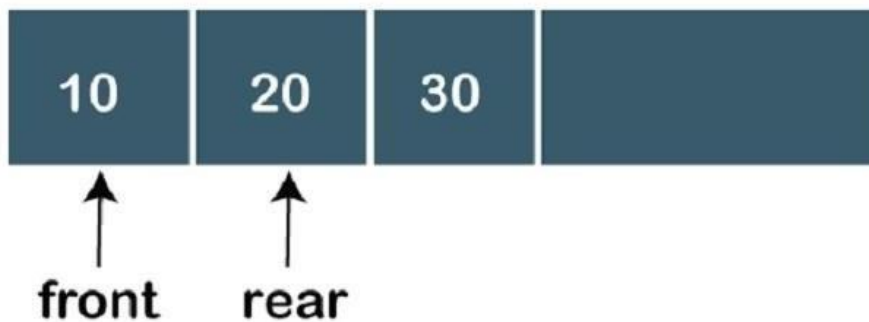
Explanation:

In Queue, the insertion is done from one end known as the rear end or the tail of the queue, whereas the deletion is done from another end known as the front end or the head of the queue. In other words, it can be defined as a list or a collection with a constraint that the insertion can be performed at one end called as the rear end or tail of the queue and deletion is performed on another end called as the front end or the head of the queue.

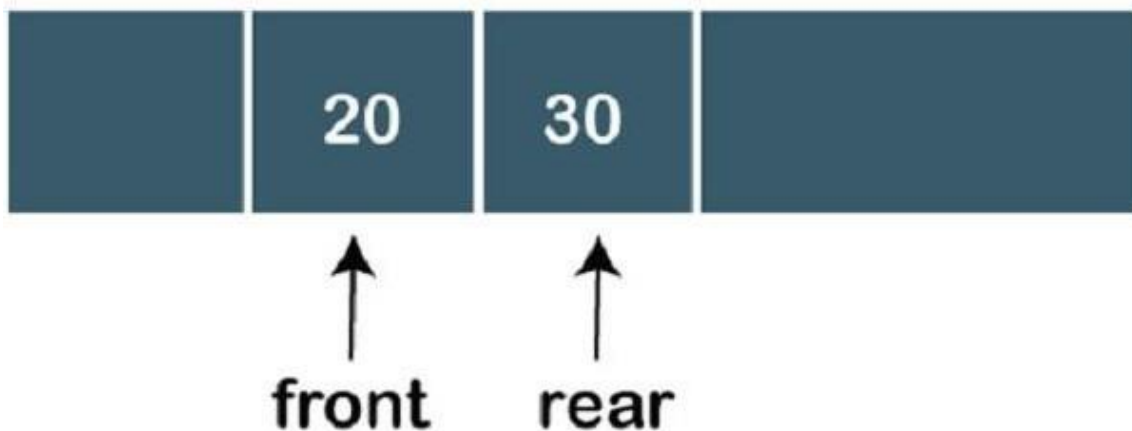
There are four types of Queues:

- **Linear Queue**

In Linear Queue, an insertion takes place from one end while the deletion occurs from another end. The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule. The linear Queue can be represented, as shown in the below figure:



The above figure shows that the elements are inserted from the rear end, and if we insert more elements in a Queue, then the rear value gets incremented on every insertion. If we want to show the deletion, then it can be represented as:

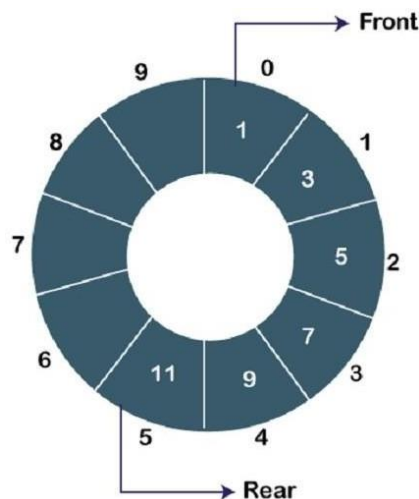


In the above figure, we can observe that the front pointer points to the next element, and the element which was previously pointed by the front pointer was deleted.

The major drawback of using a linear Queue is that insertion is done only from the rear end. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue. In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.

- **Circular Queue**

In Circular Queue, all the nodes are represented as circular. It is similar to the linear Queue except that the last element of the queue is connected to the first element. It is also known as Ring Buffer as all the ends are connected to another end. The circular queue can be represented as:



The drawback that occurs in a linear queue is overcome by using the circular queue. If the empty space is available in a circular queue, the new element can be added in an empty space by simply incrementing the value of rear.

•Priority Queue

A priority queue is another special type of Queue data structure in which each element has some priority associated with it. Based on the priority of the element, the elements are arranged in a priority queue. If the elements occur with the same priority, then they are served according to the FIFO principle.

In priority Queue, the insertion takes place based on the arrival while the deletion occurs based on the priority. The priority Queue can be shown as:

The above figure shows that the highest priority element comes first and the elements of the same priority are arranged based on FIFO structure.

•Deque

Both the Linear Queue and Deque are different as the linear queue follows the FIFO principle whereas, deque does not follow the FIFO principle. In Deque, the insertion and deletion can occur from both ends.

Code Example:

// Java program to implement a queue using an array class

```
Queue {
    private static int front, rear, capacity;
    private static int queue[];

    Queue(int c)
    {
        front = rear = 0;
        capacity = c;
        queue = new int[capacity];
    }

    // function to insert an element
    // at the rear of the queue
    static void queueEnqueue(int data)
    {
        // check queue is full or not if
        (capacity == rear) {
            System.out.printf("\nQueue is full\n");
            return;
        }

        // insert element at the rear
        else {
            queue[rear] = data;
            rear++;
        }
        return;
    }
}
```

```

// function to delete an element
// from the front of the queue
static void queueDequeue()
{
    // if queue is empty if
    (front == rear) {
        System.out.printf("\nQueue is empty\n");
        return;
    }

    // shift all the elements from index 2 till rear
    // to the right by one
    else {
        for (int i = 0; i < rear - 1; i++) {
            queue[i] = queue[i + 1];
        }

        // store 0 at rear indicating there's no element if
        (rear < capacity)
            queue[rear] = 0;

        // decrement rear
        rear--;
    }
    return;
}

```

```

// print queue elements
static void queueDisplay()
{
    int i;
    if (front == rear) {
        System.out.printf("\nQueue is Empty\n");
        return;
    }

    // traverse front to rear and print elements for
    (i = front; i < rear; i++) {
        System.out.printf(" %d <-- ", queue[i]);
    }
    return;
}

```

```

// print front of queue
static void queueFront()

```

```

    {
        if (front == rear) {
            System.out.printf("\nQueue is Empty\n");
            return;
        }
        System.out.printf("\nFront Element is: %d", queue[front]);
        return;
    }
}

```

```
public class StaticQueueinjava {
```

```
    // Driver code
```

```
    public static void main(String[] args)
    {
```

```
        // Create a queue of capacity 4
```

```
        Queue q = new Queue(4);
```

```
        // print Queue elements
```

```
        q.queueDisplay();
```

```
        // inserting elements in the queue
```

```
        q.queueEnqueue(20);
```

```
        q.queueEnqueue(30);
```

```
        q.queueEnqueue(40);
```

```
        q.queueEnqueue(50);
```

```
        // print Queue elements
```

```
        q.queueDisplay();
```

```
        // insert element in the queue
```

```
        q.queueEnqueue(60);
```

```
        // print Queue elements
```

```
        q.queueDisplay();
```

```
        q.queueDequeue();
```

```
        q.queueDequeue();
```

```
        System.out.printf("\n\nafter two node deletion\n\n");
```

```
        // print Queue elements
```

```
        q.queueDisplay();
```

```
        // print front of the queue
```

```
        q.queueFront();
```

```
    }
```

```
}
```

Output:

Queue is Empty

20 <-- 30 <-- 40 <-- 50 <--

Queue is full

20 <-- 30 <-- 40 <-- 50 <--

after two node deletion

40 <-- 50 <--

Front Element is: 40

Real life examples of queue are:

- A queue of people at ticket-window: The person who comes first gets the ticket first. The person who is coming last is getting the tickets in last. Therefore, it follows first-in-first-out (FIFO) strategy of queue.
- Vehicles on toll-tax bridge: The vehicle that comes first to the toll tax booth leaves the booth first. The vehicle that comes last leaves last. Therefore, it follows first-in-first-out (FIFO)

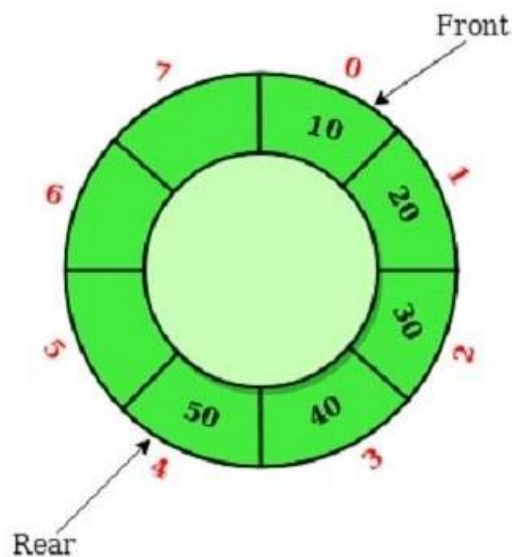
Q 12). Explain the queue full condition for circular queue?Ans

:-

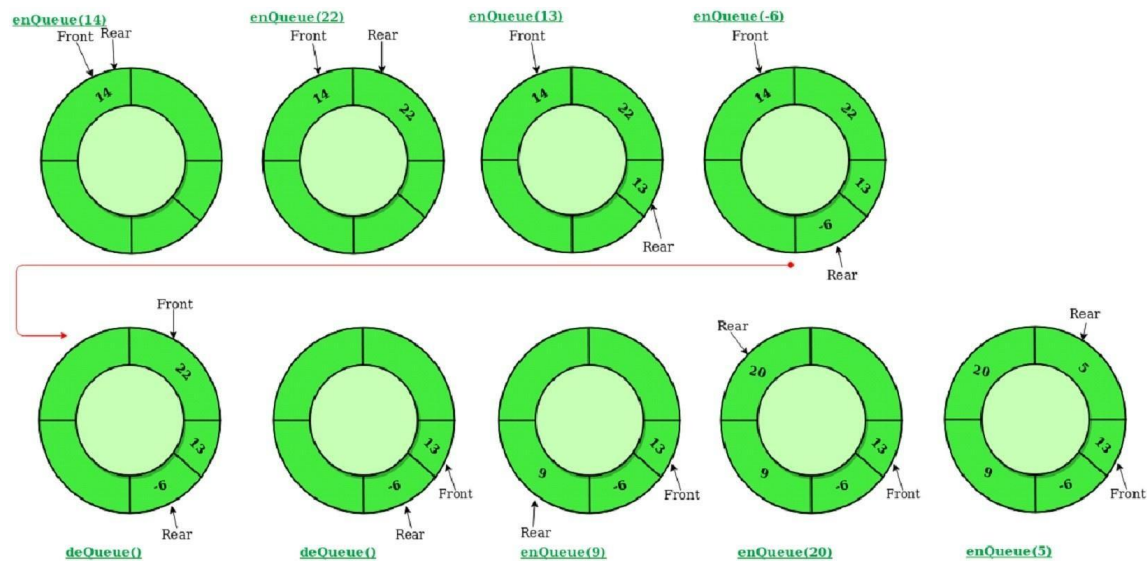
Definition:

Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle. It is also called 'Ring Buffer'.

Diagram:



Operations on Circular Queue:

**Explanation:**

The circular queue solves the major limitation of the normal queue. In a normal queue, after a bit of insertion and deletion, there will be non-usable empty space.

Circular Queue works by the process of circular increment i.e. when we try to increment the pointer and we reach the end of the queue, we start from the beginning of the queue. Here, the circular increment is performed by modulo division with the queue size. That is, if $REAR + 1 == 5$ (overflow!), $REAR = (REAR + 1) \% 5 = 0$ (start of queue)

Code:

```
// Java program for insertion and
// deletion in Circular Queue
import java.util.ArrayList;
class CircularQueue{
// Declaring the class variables.
private int size, front, rear;
// Declaring array list of integer type.
private ArrayList<Integer> queue = new ArrayList<Integer>();
// Constructor
CircularQueue(int size)
{
    this.size = size; this.front
    = this.rear = -1;
}
// Method to insert a new element in the queue.
public void enQueue(int data)
{
    // Condition if queue is full.
    if((front == 0 && rear == size - 1) ||
```

```

(rear == (front - 1) % (size - 1)))
{
    System.out.print("Queue is Full");
}
// condition for empty queue.
else if(front == -1)
{
    front = 0;
    rear = 0; queue.add(rear,
    data);
}
else if(rear == size - 1 && front != 0)
{
    rear = 0; queue.set(rear,
    data);
}
else
{
    rear = (rear + 1);
    // Adding a new element if
    if(front <= rear)
    {
        queue.add(rear, data);
    }
    // Else updating old value
    else
    {
        queue.set(rear, data);
    }
}
}
// Function to dequeue an element
// from the queue.
public int dequeue()
{
    int temp;
    // Condition for empty queue.
    if(front == -1)
    {
        System.out.print("Queue is Empty");
        // Return -1 in case of empty queue
        return -1;
    }
    temp = queue.get(front);
    // Condition for only one element
    if(front == rear)

```

```

    {
    front = -1;
    rear = -1;
    }
    else if(front == size - 1)
    {
    front = 0;
    }
    else
    {
    front = front + 1;
    }
    // Returns the dequeued element
    return temp;
}
// Method to display the elements of queue
public void displayQueue()
{
    // Condition for empty queue.
    if(front == -1)
    {
    System.out.print("Queue is Empty");
    return;
    }
    // If rear has not crossed the max size
    // or queue rear is still greater then
    // front.
    System.out.print("Elements in the " +
        "circular queue are: ");
    if(rear >= front)
    {
    // Loop to print elements from
    // front to rear.
    for(int i = front; i <= rear; i++)
    {
    System.out.print(queue.get(i));
    System.out.print(" ");
    }
    System.out.println();
    }
    // If rear crossed the max index and
    // indexing has started in loop
    else
    {
    // Loop for printing elements from
    // front to max size or last index

```

```

        for(int i = front; i < size; i++)
        {
            System.out.print(queue.get(i));
            System.out.print(" ");
        }
        // Loop for printing elements from
        // 0th index till rear position
        for(int i = 0; i <= rear; i++)
        {
            System.out.print(queue.get(i));
            System.out.print(" ");
        }
        System.out.println();
    }
}
// Driver code
public static void main(String[] args)
{
    // Initialising new object of
    // CircularQueue class.
    CircularQueue q = new CircularQueue(5);
    q.enqueue(14);
    q.enqueue(22);
    q.enqueue(13);
    q.enqueue(-6);
    q.displayQueue(); int
    x = q.dequeue();
    // Checking for empty queue.
    if(x != -1)
    {
        System.out.print("Deleted value = ");
        System.out.println(x);
    }
    x = q.dequeue();
    // Checking for empty queue.
    if(x != -1)
    {
        System.out.print("Deleted value = ");
        System.out.println(x);
    }
    q.displayQueue();
    q.enqueue(9);
    q.enqueue(20);
    q.enqueue(5);
    q.displayQueue();
    q.enqueue(20);
}

```

```

}
}

```

Output:

Elements in Circular Queue are: 14 22 13 -6

Deleted value = 14

Deleted value = 22

Elements in Circular Queue are: 13 -6 Elements

in Circular Queue are: 13 -6 9 20 5 Queue is Full

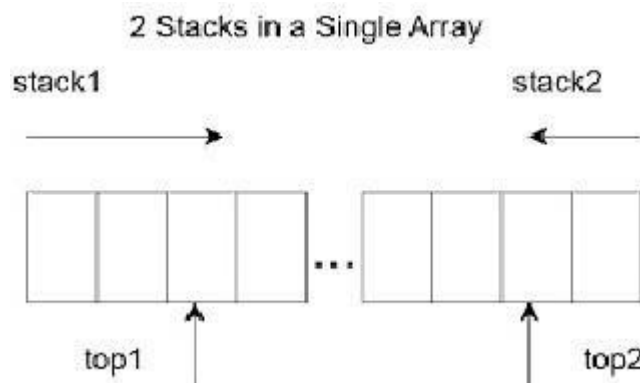
Real Time Example:-

- **Memory Management:** The unused memory locations in the case of ordinary queues can be utilized in circular queues.
- **Traffic system:** In computer controlled traffic system, circular queues are used to switch on the traffic lights one by one repeatedly as per the time set

Q 13). Write in brief the method to implement two stacks using single array? Ans:-

Stack- The stack is a linear data structure that is used to store the collection of objects. It is based on Last-In-First-Out (LIFO). Java collection framework provides many interfaces and classes to store the collection of objects.

Diagram-



Explanation- Create a data structure **twoStacks** that represents two stacks. Implementation of **twoStacks** should use only one array, i.e., both stacks should use the same array for storing elements. Following functions must be supported by **twoStacks**. **push1(int x)** → pushes **x** to first stack
push2(int x) → pushes **x** to second stack
pop1() → pops an element from first stack and return the popped element **pop2()** → pops an element from second stack and return the popped element
 Implementation of **twoStack** should be space efficient.

```

class TwoStacks {
    int size;

```

```
int top1, top2;
int arr[];
// Constructor
TwoStacks(int n)
{
    arr = new int[n];
    size = n;
    top1 = -1;
    top2 = size;
}
// Method to push an element x to stack1
void push1(int x)
{
    // There is at least one empty space for
    // new element
    if (top1 < top2 - 1) {
        top1++;
        arr[top1] = x;
    }
    else {
        System.out.println("Stack Overflow");
        System.exit(1);
    }
}
// Method to push an element x to stack2
void push2(int x)
{
    // There is at least one empty space for
    // new element
    if (top1 < top2 - 1) {
        top2--;
        arr[top2] = x;
    }
    else {
        System.out.println("Stack Overflow");
        System.exit(1);
    }
}
// Method to pop an element from first stack
int pop1()
{
    if (top1 >= 0) {
        int x = arr[top1];
        top1--;
        return x;
    }
}
```

```

        else {
            System.out.println("Stack Underflow");
            System.exit(1);
        }
        return 0;
    }
    // Method to pop an element from second stack
    pop2()
    {
        if (top2 < size) {
            int x = arr[top2];
            top2++;
            return x;
        }
        else {
            System.out.println("Stack Underflow");
            System.exit(1);
        }
        return 0;
    }
    // Driver program to test twoStack class
    public static void main(String args[])
    {
        TwoStacks ts = new TwoStacks(5);
        ts.push1(5);
        ts.push2(10);
        ts.push2(15);
        ts.push1(11);
        ts.push2(7);
        System.out.println("Popped element from" + " stack1 is " + ts.pop1());
        ts.push2(40);
        System.out.println("Popped element from" + " stack2 is " + ts.pop2());
    }
}

```

Output:- Popped element from stack1 is 11
 Popped element from stack2 is 40

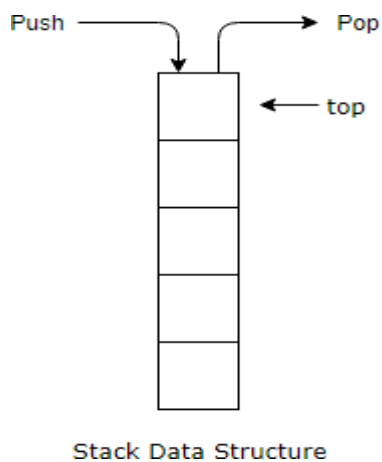
14. How can we implement stack and queue using linked list?ANS

:

STACK :

A stack is a linear data structure which allows the adding and removing of elements in a particular order. New elements are added at the top of the stack. If we want to remove an element from the stack, we can only remove the top element from the stack. Since it

allows insertion and deletion from only one end and the element to be inserted last will be the element to be deleted first, it is called the 'Last in First Out' data structure (LIFO).



Here we will define three operations on a stack:

- **Push** — it specifies adding an element to the stack. If we try to insert an element when the stack is full, then it is said to be a stack overflow condition.
- **Pop** — it specifies removing an element from the stack. Elements are always removed from the top of the stack. If we try to perform a pop operation on an empty stack, then it is said to be a stack underflow condition.
- **Peek** — it will show the element on the top of the stack (without removing it).

Stacks using linked list

A **stack** is a linear data structure that follows a specific order. The

order may be:

- LIFO (Last In First Out)
- FILO (First In Last Out)

Here, you can only retrieve whichever element you enter first at the end and not in between.

There are many ways to implement a stack data structure, but we are gonna do it using a linked list.

CODE ::

```
// Java program to Implement a stack using singly linked list
```

```
// import package
```

```

import static java.lang.System.exit;

// Create Stack Using Linked list

class StackUsingLinkedlist { // A linked list node

    private class Node {

        int data; // integer data

        Node link; // reference variable Node type }

    // create global top reference variable global

    Node top;

    // Constructor

    StackUsingLinkedlist()

    {

        this.top = null;

    }

    // Utility function to add an element x in the stack

    public void push(int x) // insert at the beginning

    {

        // create new node temp and allocate memory

        Node temp = new Node();

        // check if stack (heap) is full. Then inserting an element would lead to stack overflowif (temp

        == null) {

            System.out.print("\nHeap Overflow");

            return; }

        // initialize data into temp data field

        temp.data = x;

        // put top reference into temp link

```

```
        temp.link = top;

        // update top reference

        top = temp;
    }

    // Utility function to check if the stack is empty or not
    public boolean isEmpty()
    {
        return top == null;
    }

    // Utility function to return top element in a stack
    public int peek()
    {
        // check for empty stackif
        (!isEmpty()) {
            return top.data;
        }
        else {
            System.out.println("Stack is empty");
            return -1;
        }
    }

    // Utility function to pop top element from the stack
    public void pop() // remove at the beginning
    {
        // check for stack underflow
```

```
        if (top == null) {  
            System.out.print("\nStack Underflow");  
            return;  
        }  
        // update the top pointer to point to the next node  
        top = (top).link;  
    }  
    public void display()  
    {  
        // check for stack underflow  
        if (top == null) {  
            System.out.printf("\nStack Underflow");  
            exit(1);  
        }  
        else {  
            Node temp = top;  
            while (temp != null) {  
                // print node data  
                System.out.printf("%d->", temp.data);  
                // assign temp link to temp  
                temp = temp.link;  
            }  
        }  
    }  
}
```

```
// main class public
class GFG {

    public static void main(String[] args)
    {

        // create Object of Implementing class StackUsingLinkedlist
        obj = new StackUsingLinkedlist();

        // insert Stack value
        obj.push(11);
        obj.push(22);
        obj.push(33);
        obj.push(44);

        // print Stack elements
        obj.display();

        // print Top element of Stack
        System.out.printf("\nTop element is %d\n", obj.peek());

        // Delete top element of Stack
        obj.pop();
        obj.pop();

        // print Stack elements
        obj.display();

        // print Top element of Stack
        System.out.printf("\nTop element is %d\n", obj.peek());}}
```

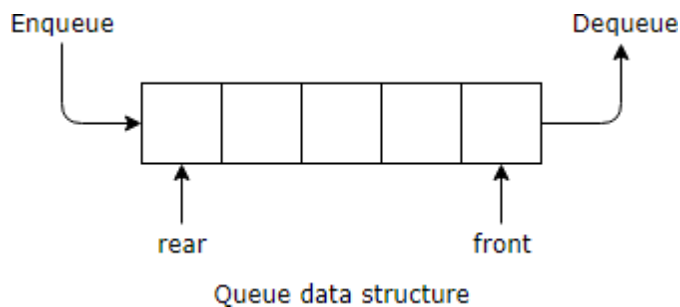
Uses of Stack

- Stack can be used to implement back/forward button in the browser.
- Undo features in the text editors are also implemented using stack.
- It is also used to implement recursion.

- Call and return mechanism for a method uses stack.
- It is also used to implement [backtracking](#).

QUEUE

A queue is also a linear data structure where insertions and deletions are performed from two different ends. A new element is added from the rear of the queue and the deletion of existing elements occurs from the front. Since we can access elements from both ends and the element inserted first will be the one to be deleted first, the queue is called the 'First in First Out' data structure (FIFO).



Here, we will define two operations on the queue.

- Enqueue – It specifies the insertion of a new element to the queue. Enqueue will always take place at the rear end of the queue.
- Dequeue — It specifies the deletion of an existing element from the queue. Dequeue will always take place at the front end of the queue.

Implementing Queue Functionalities Using Linked List

Similar to stack, the queue can also be implemented using both arrays and linked lists. But it also has the same drawback of limited size. Hence, we will be using a linked list to implement the queue.

Code ::

```
// Java program for linked-list implementation of queue
A linked list (LL) node to store a queue entry
```

```
class QNode {
    int key;
    QNode next;
```

```
// constructor to create a new linked list node
public QNode(int key)
{
    this.key = key;
    this.next = null;
}
}

// A class to represent a queue The queue, front stores the front node of LL and rear stores the
last node of LL
class Queue { QNode
    front, rear; public
    Queue()
    {
        this.front = this.rear = null;
    }

    // Method to add an key to the queue.
    void enqueue(int key)
    {
        // Create a new LL node
        QNode temp = new QNode(key);

        // If queue is empty, then new node is front and rear both if
        (this.rear == null) {
            this.front = this.rear = temp;
            return;
        }

        // Add the new node at the end of queue and change rear
```

```
        this.rear.next = temp;

        this.rear = temp;
    }

    // Method to remove an key from queue.
    void dequeue()
    {
        // If queue is empty, return NULL.if
        (this.front == null)

            return;

        // Store previous front and move front one node aheadQNode
        temp = this.front;
        this.front = this.front.next;

        // If front becomes NULL, then change rear also as NULLif
        (this.front == null)

            this.rear = null;
    }
}

// Driver class
public class Test {

    public static void main(String[] args)
    {
        Queue q = new Queue();

        q.enqueue(10);

        q.enqueue(20);

        q.dequeue();
    }
}
```

```

q.dequeue();

q.enqueue(30);

q.enqueue(40);

q.enqueue(50);

q.dequeue();

System.out.println("Queue Front : " + q.front.key);

System.out.println("Queue Rear : " + q.rear.key);

}

}

```

Uses of Queue

- CPU scheduling in operating systems uses queues. The processes that are ready to execute and the requests of CPU resources wait in a queue and the request is served on a first come first serve basis.
- [Data buffer](#) – a physical memory storage which is used to temporarily store data while it is being moved from one place to another is also implemented using queue.

15. Which are the shortcomings of array over linked list?

ANS:-

· **Size** — in Array we have to declare size of array at the declaration of array. After declaring the size of the array we cannot change its size in future. So while declaring an array we should know the upper limit of the array. Unlike array Linked list does not store data in contiguous location. In a linked list one node is connected to another node so we can change the size of the linked list any time.

· **Memory Wastage** — after declaration of array if there are some vacant spaces due to delete operation then there is Memory Wastage. In the linked list there is dynamic memory allocation so there is no wastage of memory.

· **Insertion/Deletion operation** - insertion and deletion operation in array is slower than Linked List. In an array we have to shift all elements for performing insertion/deletion operation. In insertion we have to create space for adding new elements so we need to shift all elements. In deletion operation after deleting any index value then also we have to shift elements. For insertion/deletion operations Linked List is faster than Array.

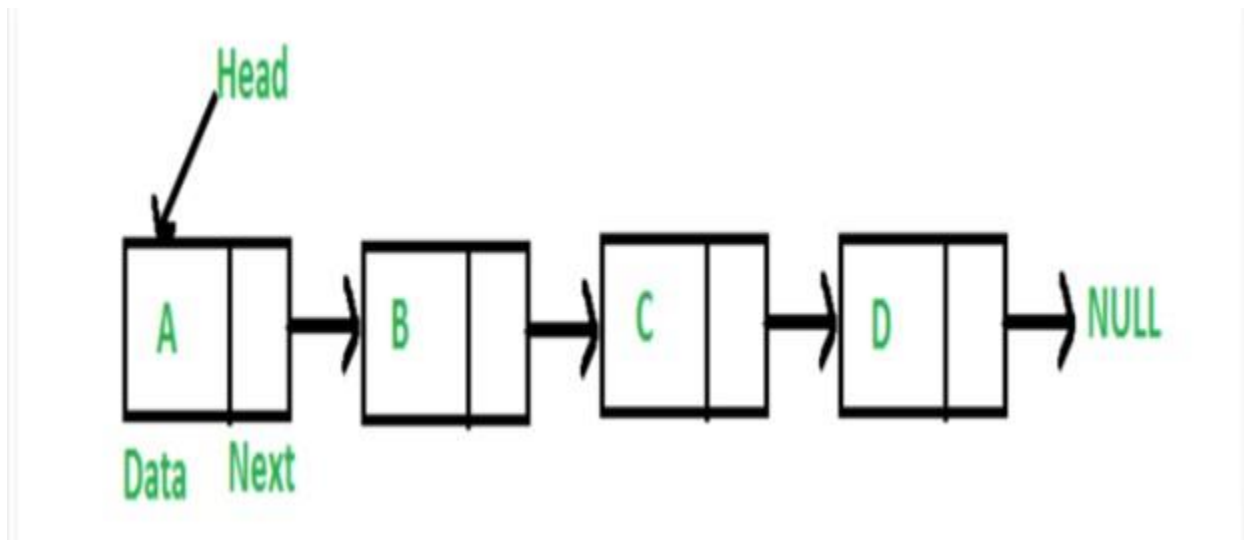
Q 16). Explain a linked list?

ANS:-

Linked List is a type of linear data structure. In linked list, data is not stored in contiguous location. Linked List consists of nodes where each node contains data and link to the next node.

• In nodes, there are two sections which contain data and address of the next node so with the help of address we access data in the linked list.

• There are 3 types of linked list – 1. Singly linked list 2. Doubly linked list 3. Circular linked list



• **Singly linked list** — in singly linked list each node contains data and address of next node so we can traverse in only one direction.

• **Doubly linked list** — in doubly linked list each node contains data and address of previous and next node. So we can traverse in forward as well as backward direction.

• **Circular linked list** — in circular linked list tail is connected to head. A circular linked list can be a Singly circular linked list or doubly circular linked list.

Advantages of Linked List

• Linked list is of dynamic size we do not have to specify size of linked list

·Insertion and deletion operation is faster unlike array we do not have to shift all the elements in insertion and deletion operation

Disadvantages of linked list

·Random access is not possible if we have to access any element then we have to traverse from head position to that position

·Not cache friendly. Since array elements are contiguous locations, there is locality of reference which is not there in case of linked list

Real life Example

- Implementation of stack and queues
- Dynamic memory allocation
- Maintaining directory of names
- Previous and next page in web browser
- Circular doubly linked list is used for implementation of advanced datastructure Fibonacci heap

Q 17). How to write a program to reverse linked list?

ANS :-

Using Stack:

Store the nodes(values and address) in the stack until all the values are entered.

Once all entries are done, Update the Head pointer to the last location(i.e the last value).

Start popping the nodes(value and address) and store them in the same order until the stack is empty.

Update the next pointer of last Node in the stack by NULL.

Code:-

// Java program for above approach

```
import java.util.*;
```

```
class GFG{
```

```
// Create a class Node to enter
// values and address in the list
static class Node
{
    int data;

    Node next;
};
static Node head = null;
// Function to reverse the
// linked list
static void reverseLL()
{
    // Create a stack "s"
    // of Node type
    Stack<Node> s = new Stack<>();
    Node temp = head;
    while (temp.next != null)
    {
        // Push all the nodes
        // in to stack
        s.add(temp); temp
        = temp.next;
    }
    head = temp;
    while (!s.isEmpty())
```

```
{  
    // Store the top value of  
    // stack in list  
    temp.next = s.peek();  
    // Pop the value from stack  
    s.pop();  
    // update the next pointer in the  
    // in the list  
    temp = temp.next;  
}  
temp.next = null;  
}  
  
// Function to Display  
// the elements in List  
static void printlist(Node temp)  
{  
    while (temp != null)  
    {  
        System.out.print(temp.data+ " ");  
        temp = temp.next;  
    }  
}  
  
// Program to insert back of the  
// linked list  
static void insert_back( int value)
```

```
{
    // we have used insertion at back method
    // to enter values in the list.(eg:
    // head.1.2.3.4.Null)
    Node temp = new Node();
    temp.data = value;
    temp.next = null;
    // If *head equals to nullif
    (head == null)
    {
        head = temp;
        return;
    }
    else
    {
        Node last_node = head; while
        (last_node.next != null)
        {
            last_node = last_node.next;
        }
        last_node.next = temp;
        return;
    }
}
```

// Driver Code

```

public static void main(String[] args)
{
    insert_back( 1);
    insert_back( 2);
    insert_back(3);
    insert_back( 4);
    System.out.print("Given linked list\n");
    printlist(head);
    reverseLL();
    System.out.print("\nReversed linked list\n");
    printlist(head);
}
}

```

Output

Given linked list

1 2 3 4

Reversed linked list

4 3 2 1

Q 18). How can we reverse linked list using single pointer? Write a program

ANS :

We have seen how to reverse a linked list in article reverse in iterative method we had used 3 pointers prev, cur and next. Below is an interesting approach that uses only two pointers. The idea is to use XOR to swap pointers.

Code :-

```
// Java program to reverse a linked
```

```
// list using two pointers.
import java.util.*;
class Main{
// Link list node
static class Node
{
    int data;
    Node next;
};
static Node head_ref = null;
// Function to reverse the linked
// list using 2 pointers
static void reverse()
{
    Node prev = null;
    Node current = head_ref;
    // At last prev points to new head
    while (current != null)
    {
        // This expression evaluates from left to right
        // current.next = prev, changes the link from
        // next to prev node
        // prev = current, moves prev to current node for
        // next reversal of node
        // This example of list will clear it more 1.2.3.4
    }
}
```

```

// initially prev = 1, current = 2

// Final expression will be current = 1^2^3^2^1,

// as we know that bitwise XOR of two same

// numbers will always be 0 i.e; 1^1 = 2^2 = 0

// After the evaluation of expression current = 3 that

// means it has been moved by one node from its

// previous position

Node next = current.next;

current.next = prev;

prev = current;

current = next;

}

head_ref = prev;
}

// Function to push a node static

void push(int new_data)

{

// Allocate node

Node new_node = new Node();

// Put in the data

new_node.data = new_data;

// Link the old list off the new node

new_node.next = (head_ref);

// Move the head to point to the new node

(head_ref) = new_node;

```

```
}

// Function to print linked list
static void printList()
{
    Node temp = head_ref;
    while (temp != null)
    {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
}

// Driver code
public static void main(String []args)
{
    push(20);
    push(4);
    push(15);
    push(85);
    System.out.print("Given linked list\n");
    printList();
    reverse();
    System.out.print("\nReversed Linked list \n");
    printList();
}
}
```

Output:

Given linked list

85 15 4 20

Reversed Linked list

20 4 15 85

19. If you are using C language to implement the heterogeneous linked list, what pointer type will you use?

ANS:-

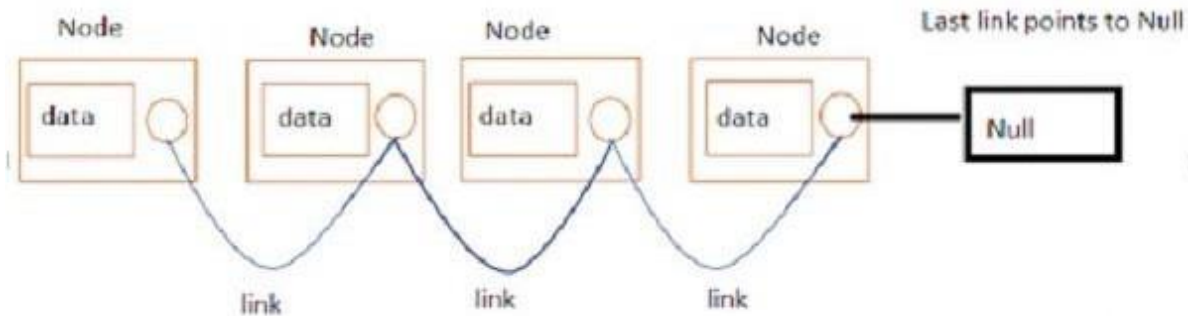
The heterogeneous linked list contains different data types in its nodes and we need a link, pointer to connect them. It is not possible to use ordinary pointers for this. So we go for void pointer. Void pointer is capable of storing pointer to any type as it is a generic pointer type.

Q 20). How to detect whether linked list is linear or circular?

ANS :-

Linked list is a linear data structure where each element called node, is a separate object. Unlike Arrays, the elements are not necessarily stored next to each other but store each other's memory addresses through pointers. Linked list's each node is divided into two parts :

- Contains data or information of element.
- Also called link field or next pointer field which contains nodes of the next node of the list.



Circular linked list is a variation of linked list where the last node is pointing to the first node's information part. Therefore the last node does not point to null.



When we traverse entire linked list:

- If any node seems to be pointing towards the head or starting node then the linked list is circular.
- If no node is pointing to null.

Then it means that the linked list is circular in nature.

Algorithm to find whether the given linked list is circular

A very simple way to determine whether the linked list is circular or not

- Traverse the linked list
- Check if the node is pointing to the head.
- If yes then it is circular.

Let's look at the snippet where we code this algorithm.

```
#include <iostream>
```

```
using namespace std;
```

```
//node structure
struct node{
    int data;
    struct node* next;
};

//function to find the circular linked list.
bool isCircular(node *head){
    node *temp=head;
    while(temp!=NULL)
    { //if temp points to head then it has completed a circle,thus a circular linked list.
        if(temp->next==head)
            return true;
        temp=temp->next;
    }
    return false;
}

//function for inserting new nodes.
node *newNode(int data){
    struct node *temp=new node;
    temp->data=data;
    temp->next=NULL;
}

int main() {
    //first case
    struct node* head=newNode(1);
```

```
head->next=newNode(2);
head->next->next=newNode(3);
    head->next->next->next=newNode(4);
    head->next->next->next->next=head;
if(isCircular(head))
cout<<"yes"<<endl;
else
cout<<"no"<<endl;

    //second case
struct node* head1=newNode(1);
head1->next=newNode(2);
if(isCircular(head1))
cout<<"yes";
else
cout<<"no";
return 0;
}
```

21. How to write a program to create Homogeneous linked list?

>>>

Expalnation:

A Homogeneous **linked list** is a linear collection of data elements whose order is not given by their physical placement in memory. Instead, each element points to the next. It is a data structure consisting of a collection of nodes which together represent a sequence. In its most basic form, each node contains: data, and a reference (in other words, a link) to the next node in the sequence. This structure allows for efficient insertion or removal of elements from any position in the sequence during iteration.

Program:

```
class LinkedList
{
    // create an object of Node class
    // represent the head of the linked list
    Node head;

    // static inner class
    static class Node
    {
        int value;
        // connect each node to next node
        Node next;

        Node(int d)
        {
            value = d;
            next = null;
        }
    }

    public static void main(String[] args)
    {

        // create an object of LinkedList
        LinkedList linkedList = new LinkedList();

        // assign values to each linked list node
        linkedList.head = new Node(1);
        Node second = new Node(2);
```

```

Node third = new Node(3);

// connect each node of linked list to next node
linkedList.head.next = second;
second.next = third;

// printing node-value
System.out.print("LinkedList: ");
while (linkedList.head != null)
{
    System.out.print(linkedList.head.value + " ");
    linkedList.head = linkedList.head.next;
}
}
}

```

22. Find out address of middle node in single traversal?

>>>

Explanaton:

- i) Use two pointer slow and fast. Initialized both the pointers to the head of a linked list.
- ii) Move **fast pointer by two step and slow pointer by one step** in each iteration.
- iii) When fast pointer reaches at the end of a linked list, The slow pointer points to themiddle element of a linked list

Program:

```

public class Node
{
    int data;
    Node next;

    public Node(int data)
    {
        this.data = data;
        this.next = null;
    }
}

* Main class
*/
public class MiddleElement

```

```
{  
  
    private Node head;  
  
    //In constructor, Initialize head attribute to null  
  
    public MiddleElement()  
    {  
        this.head = null;  
    }  
  
    public Node insert(int data)  
    {  
  
        if (head == null)  
        {  
            head = new Node(data);  
        }  
        else  
        {  
            // Create a new node  
            Node temp = new Node(data);  
  
            // New node points to head  
            temp.next = head;  
  
            // Head points to a new node  
            head = temp;  
        }  
  
        return head;  
    }  
  
    //Logic to print middle element of a linked list  
  
    public void printMiddleElement()  
    {  
  
        Node slow = head;  
        Node fast = head;  
  
        while (fast != null && fast.next != null)  
        {  
  
            slow = slow.next;
```

```
        fast = fast.next.next;
    }

    System.out.println(" Middle Element of a Linked List is " + slow.data);
}

public void print()
{
    Node temp = head;

    /*
     * Traverse a list and check if it not points to null. If it points to
     * null, It means there is no node present after that and we need to end
     * the loop.
     */
    while (temp != null) {
        System.out.println(temp.data);

        temp = temp.next;
    }
}

public static void main(String[] args)
{
    MiddleElement ll = new MiddleElement();

    ll.insert(6);
    ll.insert(5);
    ll.insert(8);
    ll.insert(9);
    ll.insert(15);

    ll.print();

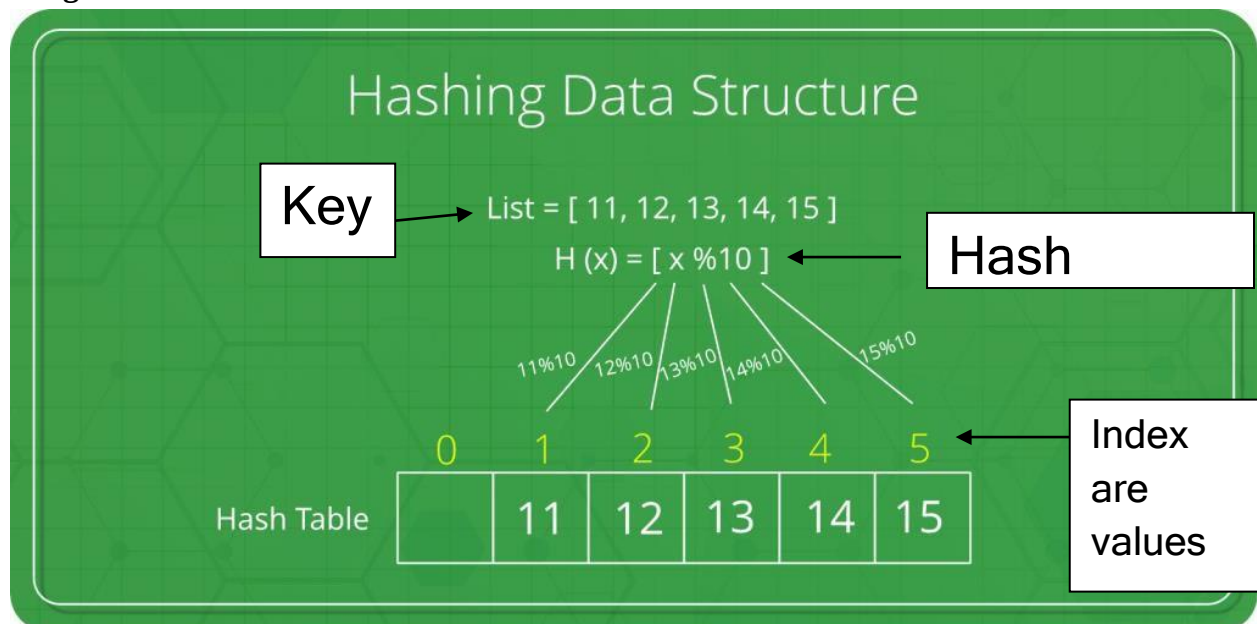
    ll.printMiddleElement();
}
```

23.Explain hash table? Explain hash function? How is it designed?Ans:-

Definition:- A hash table(hash map) is a type of data structure which is used for storing and accessing data very quickly. In a Big O(1) time.

Explanation :-A structure that can map keys to values. A hash table uses a hash function to compute an index, also called a hash code, into an array of buckets or slots, from which the desired value can be found. During lookup, the key is hashed and the resulting hash indicates where the corresponding value is store.

Diagram:-



Hash function:- It is used to generate the corresponding value to store the key. It is basically a mathematical function known as Hash function.

Some of hash functions are:-

1.Division method:- This is the easiest method to create a hash function. The hash function can be described as –

$$h(k) = k \bmod n$$

Here, $h(k)$ is the hash value obtained by dividing the key value k by size of hash table n using the remainder. It is best that n is a prime number as that makes sure the keys are distributed with more uniformity.

An example of the Division Method is as follows –

$k=1276$

$n=10$

$h(1276) = 1276 \bmod 10$

$= 6$

The hash value obtained is 6

2. Multiplication method

3. Mid Square method

Real time application: DataBase Indexing, Caches, Unique Data Representation, Sets

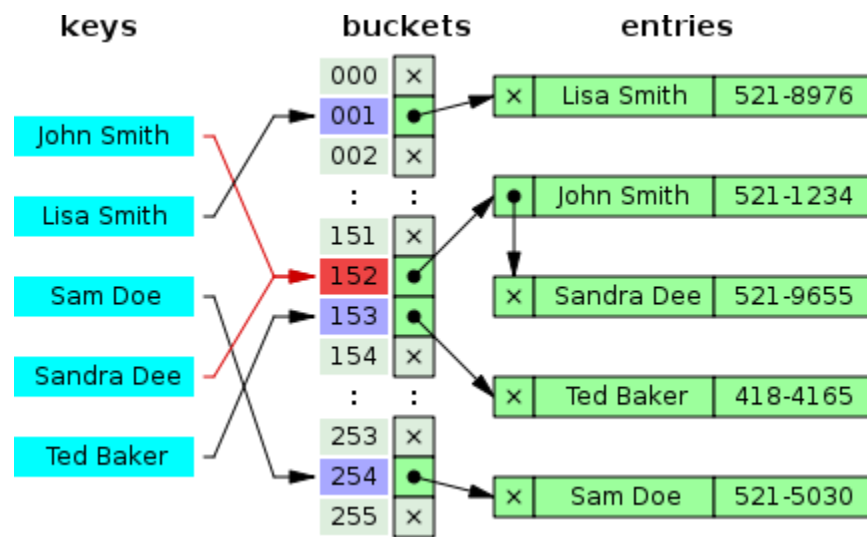
24. Explain collision and how to handle it?

Ans:-

Collision : Since a hash function gets us a small number for a big key, there is possibility that two keys result in same value. The situation where a newly inserted key maps to an already occupied slot in hash table is called **collision** and must be handled using some collision handling technique. Following are the ways to handle collisions:

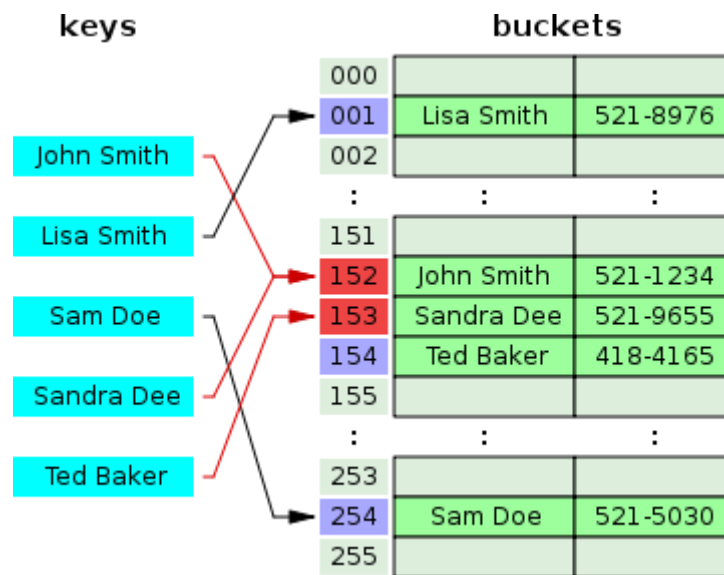
- **Separate Chaining:** The idea is to make each cell of hash table point to a linked list of records that have same hash function value. Chaining is simple, but requires additional memory outside the table.

Diagram:-



- **Open Addressing:** In open addressing, all elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we one by one examine table slots until the desired element is found or it is clear that the element is not in the table. It uses linear probing to store the data in hash table.

Diagram:-



Hash collision resolved by open addressing with linear probing (interval=1). Note that "Ted Baker" has a unique hash, but nevertheless collided with "Sandra Dee", that had previously collided with "John Smith".

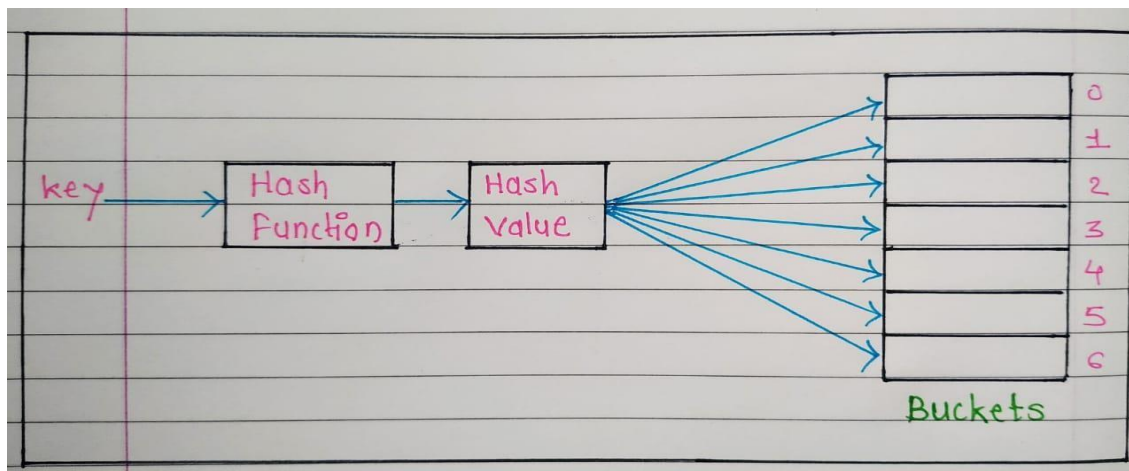
25] Explain Bucket ? Explain Load Factor ?

1. Definition :-

> **Bucket** :- Bucket is a hash file that stores the data in bucket format. It is a technique that determines an index or location for storage of an data.

> **Load Factor** :- Load Factor decides when to increase the number of buckets. The value n/b is called the **load factor**. where n are the entries and b is the size of array ; on each index.

2. Diagram :-

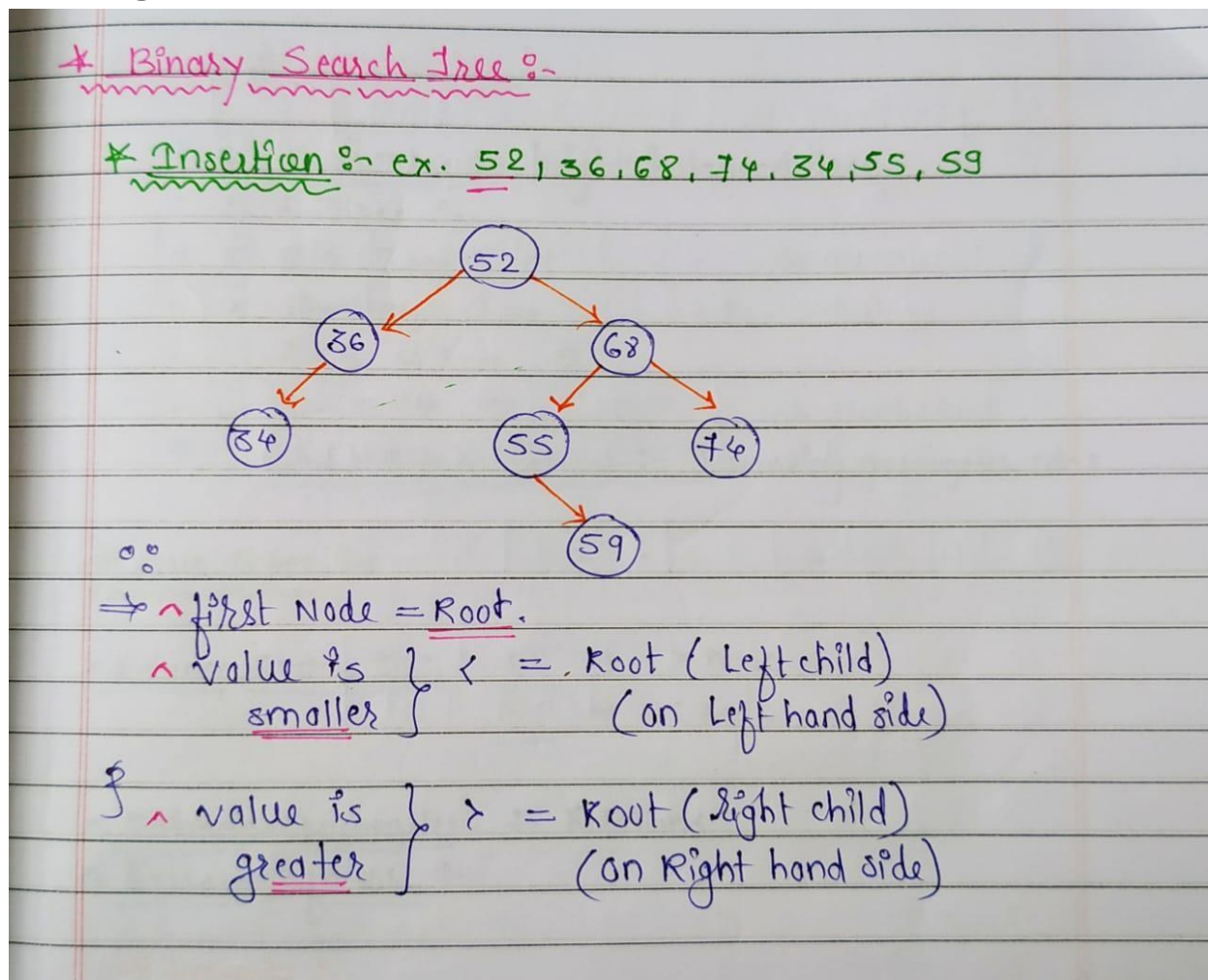


3. Explanation :- The Key receives the information for any location. When a (search) key value is provided, the hash function always computes the same address. When a record is required to be entered using hash value, the hash function computes the bucket address for search key, where the record will be stored.

26] Explain Binary Search Tree ?

1. Definition :- Binary Search Tree is also known as an ordered tree, it is a tree which contains the lesser number on the left hand side of tree and greater number on the right hand side. The left and right subtree each also be a binary search tree.

2. Diagram :-



3. Explanation :- As per the above diagram, 52 is known as the first node which is called as Root Node / Parent Node. 36 and 68 are the child nodes of the root node.

4. Code example :-

```

import java.util.*;
class BinarySTree
{
    Node root;
    static class Node
    {

```

```
        int data ;
        Node left , right;
        Node(int d)
        {
            data = d;
        }
    }
    BinarySTree()
    {
        root = null;
    }
    void insert(int key)
    {
        root = insertData(root , key);
    }
    Node insertData(Node root , int key)
    {
        if(root == null)
        {
            root = new Node(key);
            return root;
        }
        if(key < root.data)
            root.left = insertData(root.left, key);
        else if(key > root.data)
            root.right = insertData(root.right, key);
        return root;
    }
    void Inorder()
    {
        Inorder(root);
    }
```

```

    }
    void Inorder(Node root)
    {
        if(root != null)
        {
            Inorder(root.left);
            System.out.println(root.data);
            Inorder(root.right);
        }
    }
    public static void main(String args [])
    {
        BinarySTree b = new BinarySTree();
        b.insert(35);
        b.insert(25);
        b.insert(15);
        b.insert(45);
        b.insert(85);
        b.insert(54);
        b.insert(65);
        b.Inorder();
    }
}

```

5. Real time example :- We can use Binary Search Tree in many Search applications where we always insert and delete the data such as map and set objects in many Languages Libraries.

27) Minimum number of queues needed to implement the priority queue?

ans: Minimum number of queues needed to implement the priority queue are Two, Out of the 2 queues

1 queue is used for storing actual data and the other queue is used for storing priority data.

28) Define the notations used in Evaluation of Arithmetic Expressions using prefix and postfix forms?

ans: Polish and Reverse Polish are the notations used in Evaluation of Arithmetic Expressions using prefix and postfix forms. Polish notation is a notation form for expressing arithmetic, logic and algebraic equations. Its most basic distinguishing feature is that operators are placed on the left of their operands. If the operator has a defined fixed number of operands, the syntax does not require brackets or parenthesis to lessen ambiguity. Reverse Polish notation, also known as postfix notation, contrasts with the "infix notation" of standard arithmetic expressions in which the operator symbol appears between the operands. If a value appears next in the expression, push this value on to the stack.

30. Define the methods available in storing sequential files

The methods available in storing sequential files are:

- **Straight merging:** Perhaps the most straightforward sorting technique is the **straight merge**. The **straight merge distributes** the initial records onto two tapes, a and b, so that they contain the same number of records, or so that one tape contains only one more record than the other.
- **Natural merging :- Natural Merge Sort** is an optimization of Merge Sort: It identifies pre-sorted areas ("runs") in the input data and merges them. This prevents the unnecessary further dividing and **merging** of presorted subsequences. Input elements sorted entirely in ascending order are therefore sorted in $O(n)$.
- **Poly phase sort:-** A polyphase merge sort is a **variation of bottom up Merge sort** that sorts a list using an initial **uneven distribution of sub-lists (runs)**, primarily used for external sorting, and is more efficient than an ordinary merge sort when there are fewer than 8 external working files (such as a tape drive or a file on a hard drive). A polyphase merge sort is not a stable sort.
- **Distribution of Initial runs:-**

29. Convert the expression $((A=B)*C-(D-E)^(F+G))$ to equivalent Prefix and Postfix notations.

Infix to prefix

$$((A=B) * C - (D-E)^{(F+G)})$$

First we have to reverse the expression

$$((G+F)^{(E-D)} - C * (B=A))$$

character	stack	expression
(	(	
(	((	
G	((	G
+	((+	G
F	((+	GF
)	(	GF+
^	(^	GF+
(	(^ (	GF+
E	(^ (	GF+E
-	(^ (-	GF+E
D	(^ (-	GF+ED
)	(^	GF+ED -
-	(-	GF+ED - ^
(	(-	GF+ED - ^ (
*	(- *	GF+ED - ^ (
(	(- * (	GF+ED - ^ (
B	(- * (	GF+ED - ^ (B
=	(- * (=	GF+ED - ^ (B=
)	(- *	GF+ED - ^ (B=
)		* -

OTES

$GF+ED - ^ (B = * -$
The prefix notation is

2020

$$((A=B)*C-(D-E)^(F+G))$$

Infix to postfix.

character	stack	postfix.
(	(	
(	((	
A	((	A
=	((=	
B	((=	AB
)	(	AB=
*	(*	AB=
C	(*	AB=C
-	(-	AB=C*
(	(- (	AB=C*
D	(- (	AB=C* D
-	(- (-	AB=C* D
E	(- (-	AB=C* D E
)	(-	AB=C* D E -
^	(- ^	AB=C* D E -
(	(- ^ (	
F	(- ^ (	AB=C* D E - F
+	(- ^ (+	AB=C* D E - F +
G	(- ^ (+	AB=C* D E - F G
)	(- ^	AB=C* D E - F G + ^
)		

NOTES

The postfix notation is

$$AB=C*DE-FG+^ -$$

2020

31. Write in brief tree traversal techniques?

Definition of Tree Traversal Techniques

Tree traversal techniques are very important traversal techniques in computer science that can resolve many of the graph-based problems in real life. Tree traversal involves traversing each node of the tree at least once where the information in the form of key and value can be retrieved for easy identification. Tree traversal is a recursive process where the scenario might arise to visit the same node again therefore to avoid the situation a stack is maintained either in the form of a queue or stack to maintain the hierarchy of nodes within.

Tree Traversal Techniques

Tree traversal techniques play a significant role when it comes to computer science and graph theory concepts to be applied to real-life scenarios. Traversal in a tree within nodes is possible when certain operations are applied. The operations applied on these sub-trees are known as tree traversal techniques.

In order to get the tree traversal for Binary Tree Traversal Techniques four possible ways:

- Inorder Tree traversal
- Preorder Tree traversal
- Postorder Tree traversal
- Levelorder Tree traversal

In order to get the tree traversal for Recursive Traversal algorithms are as follows:

- Inorder Tree Traversal
- Preorder Tree Traversal
- Postorder Tree Traversal

In order to get the tree traversal for Non Recursive traversal algorithms are similar as above just what is different and segregated is as follows:

Level Order Tree Traversal

Binary Tree Traversal Techniques

Binary tree traversal techniques include these operations when in case there are scenarios related to them.

The inorder traversal, in this case, is the same where the left subtree of node is traversed before right subtree and a major difference can be seen on the root of the entire tree.

In the case of preorder and postorder traversal also the behavior is the same as in the order traversal technique.

Recursive Tree Traversal Techniques

Recursive Tree Traversal Techniques also involve inorder traversal technique in which the root of each subtree is visited once the left subtree is traversed with but before the right tree traversal gets initiated. For inorder traversal with respect to recursive tree, traversal involves certain procedures like :

At first when inorder traversal is followed then in that case the left subtree is visited followed by inorder traversal then manipulation is performed over the root and then finally the right subtree is visited for further manipulation.

Recursive tree traversal with respect to preorder traversal involves visiting each of the tree's roots once the left subtrees and right subtrees traversal is covered. Procedures followed are in a way where the root is visited first then the left subtree is visited followed by the right subtree thus covering the entire preorder traversal.

In the case of Postorder traversal with respect to recursive tree traversal techniques is as follows where the root is visited only once the left subtree and right subtree is covered for traversal. Thus procedure to cover the postorder traversal includes a visit to the first left sub-tree then a visit the right subtree to cover and once done then it lands to the last node which is the root.

Level order tree traversal is also one of the traversal techniques which are quite different when compared to Preorder, Inorder, and Postorder traversal technique where the traversal has a pattern that takes place level by level which gets initiated from the root and then travels from left to right where each traversal within a level requires a data structure as mentioned before queue or stack.

When a queue is present as a data structure within each level of traversal then in that case there exists one dependency factor which is related to the Inorder, Preorder, or Postorder traversal technique.

This dependency is not at all recommended thus it is made sure that in order to avoid this level order tree is segregated when compared with other traversal techniques including preorder, inorder, or postorder traversal technique.

Since it is not possible to run this level order tree with respect to recursive algorithms then it goes well with breadth-first search techniques thus can be used with those traversal algorithms efficiently.

Non-Recursive Tree Traversal Algorithm

Non-Recursive Traversal Algorithm for traversing again deals with mostly stack where the elements present in the node in form of information again needs inorder, preorder and postorder traversal but in different pattern again not including Low-level tree traversal technique.

Sometimes it is assumed that the flat traversal function will always solve the problem related to traversal as it consumes less space for stack but it is not necessary that it will happen so that the implementor will be using the flat version always.

At times while fetching and traversing within the trees and nodes the overhead arise can be more than expected then in that case it will always find some external pointer to make the functionality aligned for an expected manner.

Pre-order, inorder and postorder traversal techniques also get associated with the Non-recursive tree traversal technique which includes stack push and pops with setting root as vertex and simultaneously making the left subtree and right subtree traversal

occurring with the respective scenario making the root of the vertex at the down or edge of the stack.

Let's take a scenario where the inorder traversal is taking place than in that case the stack is pushed with zero initially and then the root is set as vertex following the same procedure until the stack is completely empty, then after that, the traversal moves to left subtree with vertex making as root and then the left subtree followed by right subtree.

In a similar format, both the pre-order and postorder traversal takes place in non-recursive tree traversal with respect to stack.

Conclusion

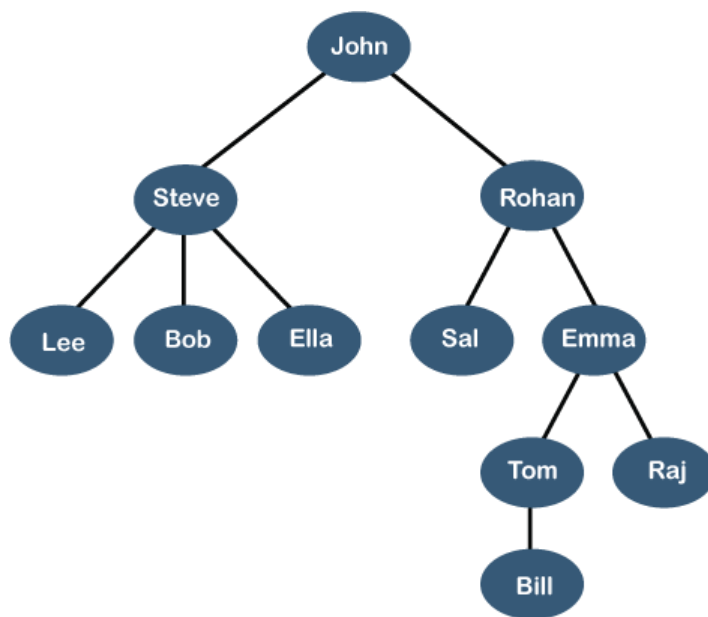
Tree traversal techniques play a pivotal role in solving any of the graph theory-related scenarios where the traversal happens with help of binary trees. Tree as a data structure provides a lot of flexibility in order to get the information stored at a node in each level from root to the depth of the entire tree.

32. Which are the types of tree? Write in brief in brief?

Defination :

A tree is a Hierarchical data structure that naturally hierarchically stores the information. The Tree data structure is one of the most efficient and mature. The nodes connected by the edges are represented.

Properties of Tree: Every tree has a specific root node. A root node can cross each tree node. It is called root, as the tree was the only root. Every child has only one parent, but the parent can have many children.



Types Of Tree :

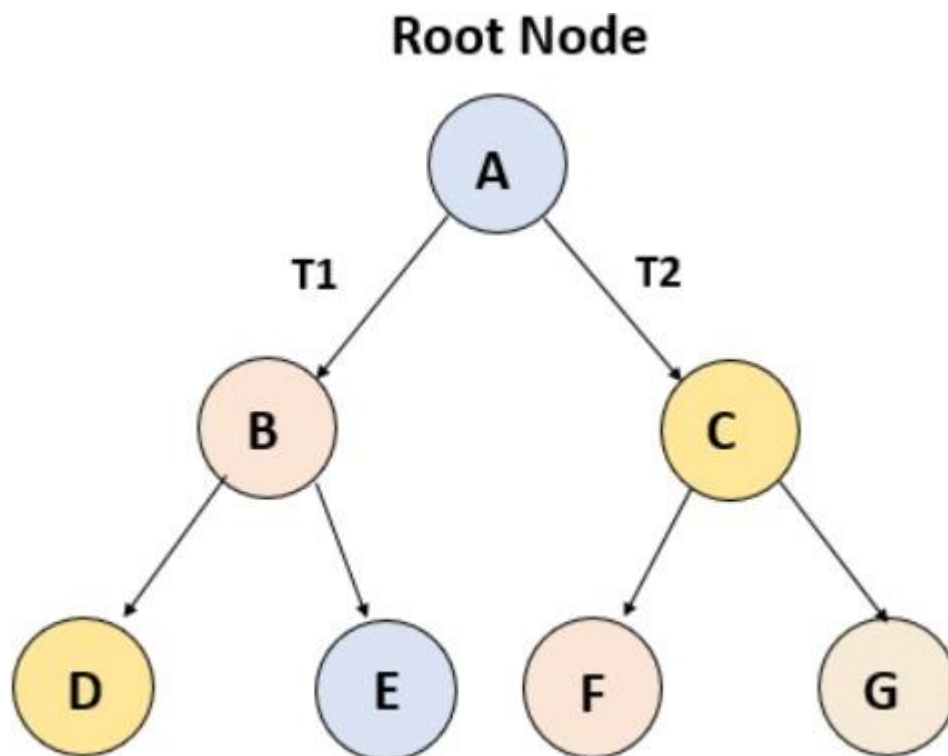
1. General Tree

If no constraint is placed on the tree's hierarchy, a tree is called a general tree. Every node may have infinite numbers of children in General Tree. The tree is the super-set of all other trees.

2. Binary Tree

The binary tree is the kind of tree in which most two children can be found for each parent. The kids are known as the left kid and right kid. This is more

popular than most other trees. When certain constraints and characteristics are applied in a Binary tree, a number of others such as AVL tree, BST (BinarySearch Tree), RBT tree, etc. are also used. When we move forward, we will explain all these styles in detail.

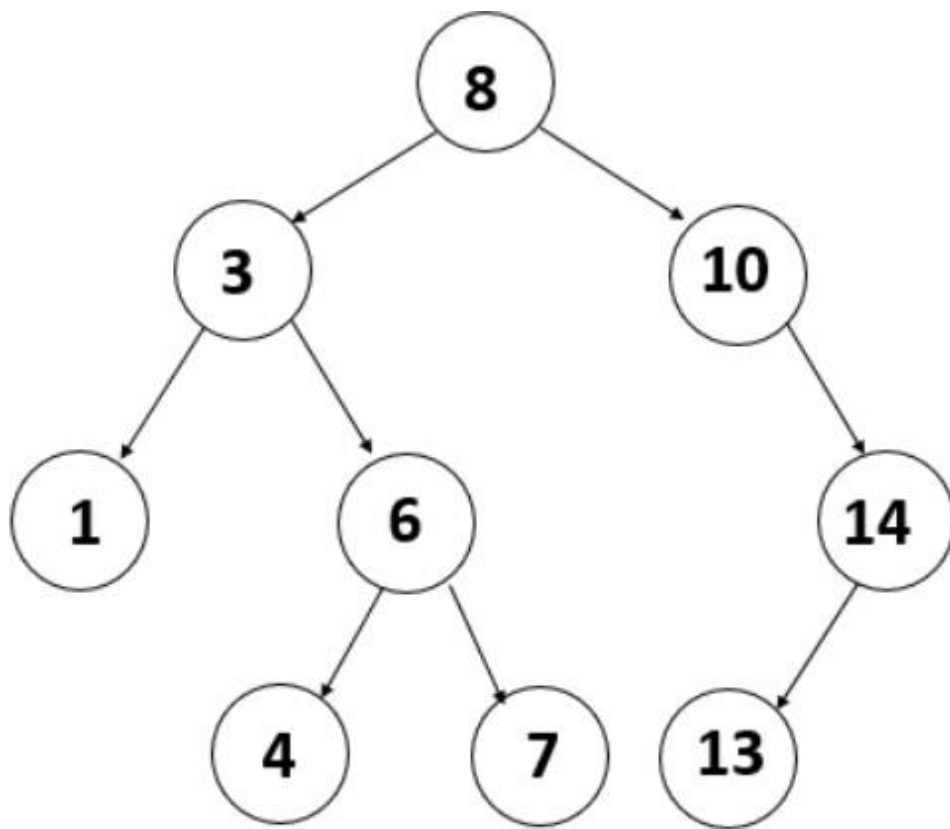


3. Binary Search Tree

Binary Search Tree (BST) is a binary tree extension with several optional restrictions.

The left child value of a node should in BST be less than or equal

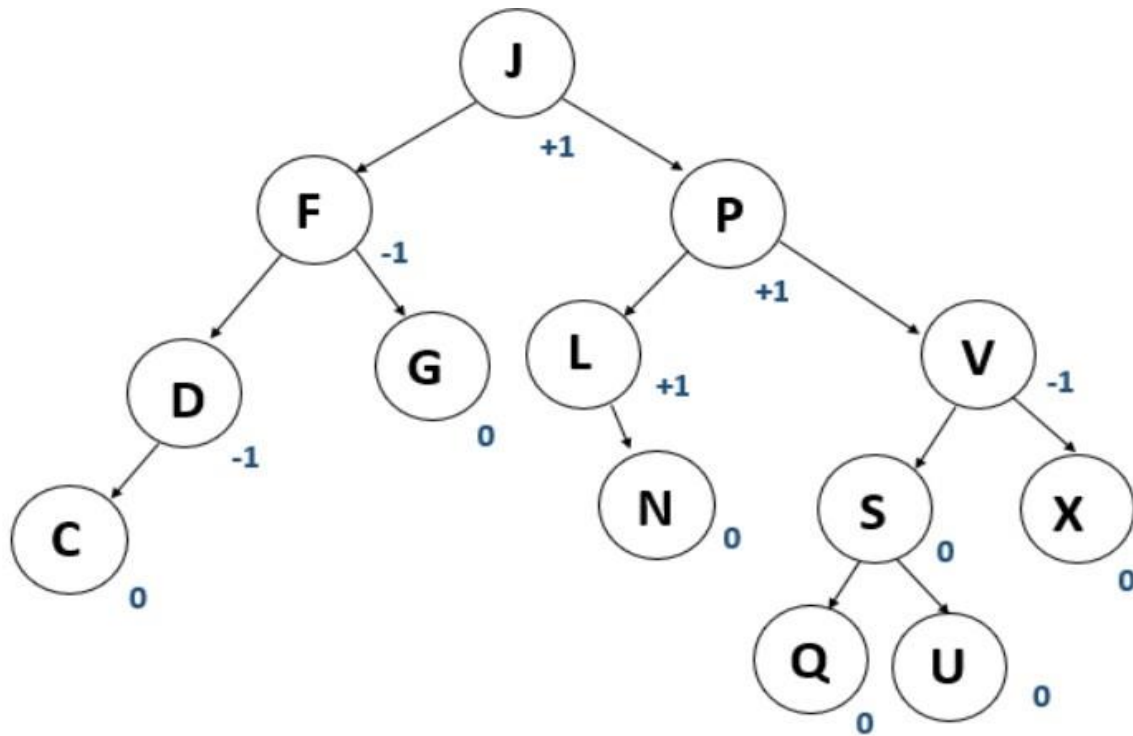
to the parent value, and the right child value should always be greater than or equal to the parent's value. This Binary Search Tree property makes it ideal for search operations since we can accurately determine at each node whether the value is in the left or right sub-tree. This is why the Search Tree is named.



www.educba.com

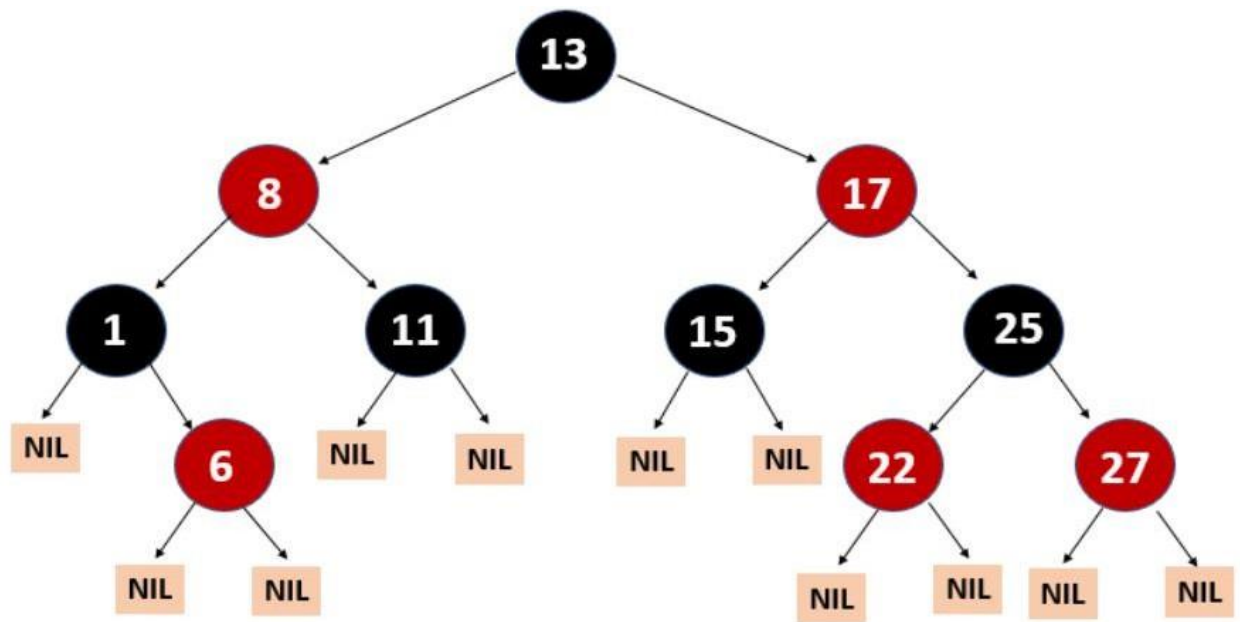
4. AVL Tree

AVL tree is a binary search tree self-balancing. On behalf of the inventors Adelson-Velshi and Landis, the name AVL is given. This was the first tree that balanced dynamically. A balancing factor is allocated for each node in the AVL tree, based on whether the tree is balanced or not. The height of the node kids is at most 1. AVL vine. In the AVL tree, the correct balance factor is 1, 0 and -1. If the tree has a new node, it will be rotated to ensure that it is balanced. It will then be rotated. Common operations such as viewing, insertion, and removal take $O(\log n)$ time in [the AVL tree](#). It is mostly applied when working with Lookups operations.



5. Red-Black Tree

Another kind of auto-balancing tree is red-black. According to the red-black tree's properties, the red-black name is given because the Red-black tree has either red or Black painted on each node. It maintains the balance of the forest. Even though this tree is not fully balanced, the searching operation only takes $O(\log n)$ time. When the new nodes are added in Red-Black Tree, nodes will be rotated to maintain the Red-Black Tree's properties.



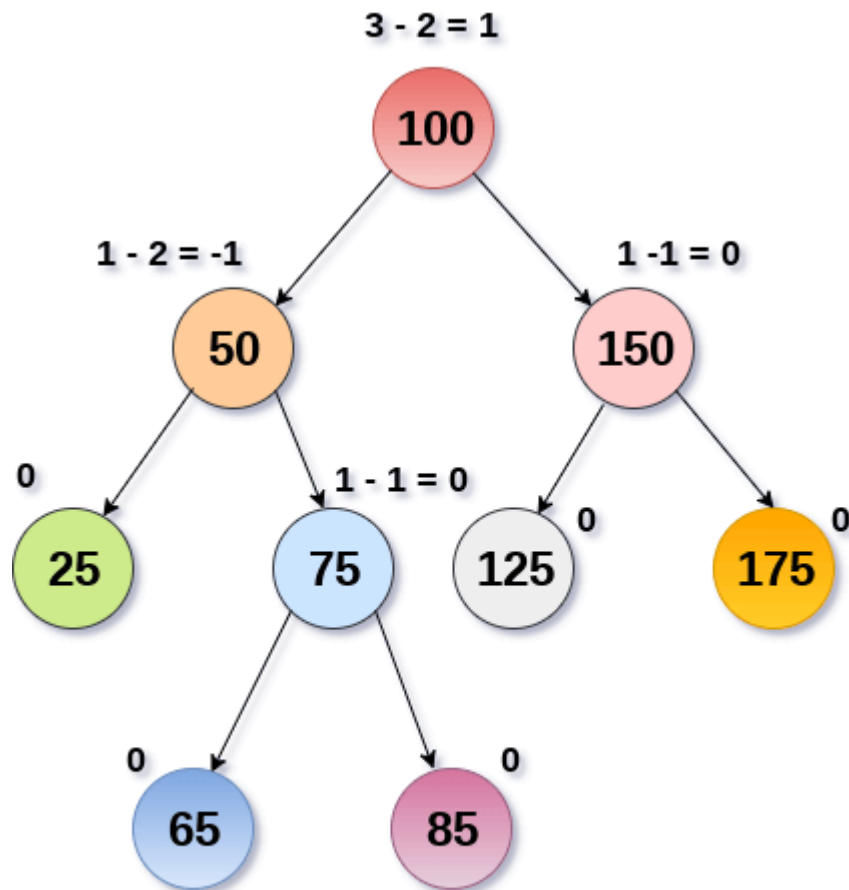
33. Explain AVL tree?

Defination :

AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its rightsub-tree from that of its left sub-tree.

AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.

Diagrams :



AVL Tree

Explanation :

Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

Balance Factor (k) = height (left(k)) - height (right(k))

If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

An AVL tree is given in the Above figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.

Operations on AVL tree :

SN	Operation	Description
1	Insertion	Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations.
2	Deletion	Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.

Why AVL Tree?

AVL tree controls the height of the binary search tree by not letting it to be skewed. The time taken for all operations in a binary search tree of height h is $O(h)$. However, it can be extended to $O(n)$ if the BST becomes skewed (i.e. worst case). By limiting this height to $\log n$, AVL tree imposes an upper bound on each operation to be $O(\log n)$ where n is the number of nodes.

AVL Rotations :

We perform rotation in AVL tree only in case if Balance Factor is other than -1, 0, and 1. There are basically four types of rotations which are as follows:

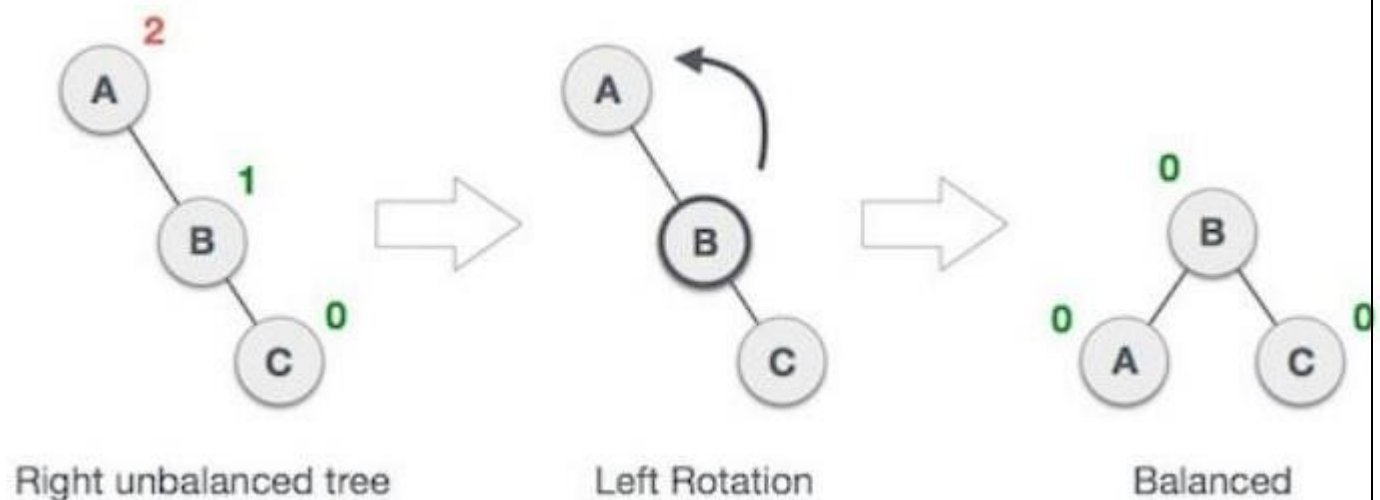
1. L L rotation: Inserted node is in the left subtree of left subtree of A
2. R R rotation : Inserted node is in the right subtree of right subtree of A
3. L R rotation : Inserted node is in the right subtree of left subtree of A
4. R L rotation : Inserted node is in the left subtree of right subtree of A

Where node A is the node whose balance Factor is other than -1, 0, 1.

The first two rotations LL and RR are single rotations and the next two rotations LR and RL are double rotations. For a tree to be unbalanced, minimum height must be at least 2, Let us understand each rotation

1. RR Rotation :

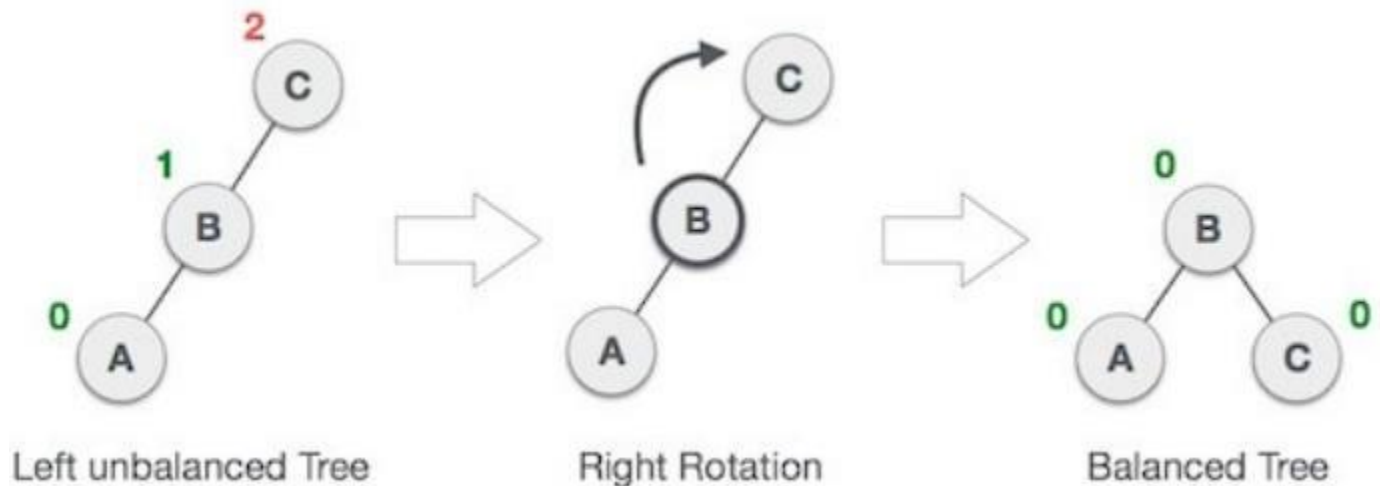
When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation, RR rotation is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2



In above example, node A has balance factor -2 because a node C is inserted in the right subtree of A right subtree. We perform the RR rotation on the edge below A.

2. LL Rotation :

When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, LL rotation is clockwise rotation, which is applied on the edge below a node having balance factor 2.



In above example, node C has balance factor 2 because a node A is inserted in the left subtree of C. We perform the LL rotation on the edge below A.

3. LR Rotation :

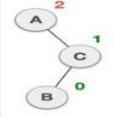
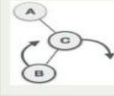
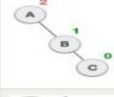
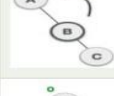
Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

Let us understand each and every step very clearly:

State	Action
	A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C
	As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node A, has become the left subtree of B.
	After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of C
	Now we perform LL clockwise rotation on full tree, i.e. on node C. node C has now become the right subtree of node B, A is left subtree of B
	Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.

4. RL Rotation

As already discussed, that double rotations are bit tougher than single rotation which has already explained above. R L rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by fulltree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

State	Action
	A node B has been inserted into the left subtree of C the right subtree of A , because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A .
	As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at C is performed first. By doing RR rotation, node C has become the right subtree of B .
	After performing LL rotation, node A is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node A .
	Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A . node C has now become the right subtree of node B , and node A has become the left subtree of B .
	Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.

Real Life Example :

Applications :

AVL trees are used when we need to perform Search operation more frequently than Insertion or Deletion (Otherwise Red-Black trees are preferred, due to the little overhead in AVL trees caused by Rotation).

1. They are used in Operating System kernels for searching address of processes during Preemption.
2. Language dictionaries, as they are loaded once and are frequently used for retrieval.
3. They may also be used in Huffman trees for data compression.
4. Expression solvers and parsers.
5. They are also used in implementations of `java.util.Set` in Java and `sets` and `maps` in C++ STL.

A couple of real life examples of Red-Black trees:

- Linux Completely Fair Scheduler
- Java Hashmap

34. Explain BFS and DFS?

- **Breadth First Traversal (BFS) Algorithm :**

Defination :

Breadth first search is a graph traversal algorithm that starts traversing the graph from root node and explores all the neighbouring nodes. Then, it selects the nearest node and explore all the unexplored nodes. The algorithm follows the same process for each of the nearest node until it finds the goal.

Algorithm :

Step 1: SET STATUS = 1 (ready state) for each node in G

Step 2: Enqueue the starting node A and set its STATUS = 2 (waiting state)

Step 3: Repeat Steps 4 and 5 until QUEUE is empty

Step 4: Dequeue a node N. Process it and set its STATUS = 3 (processed state).

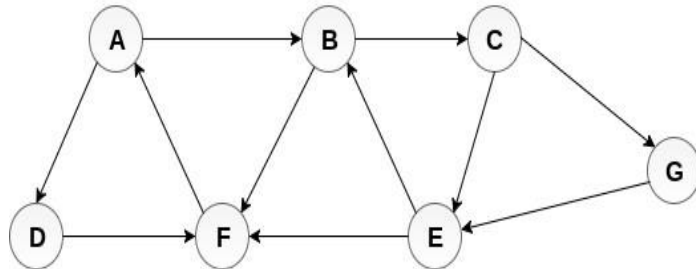
Step 5: Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)

[END OF LOOP]

Step 6: EXIT

Diagram :

Consider the graph G shown in the following image, calculate the minimum path p from node A to node E. Given that each edge has a length of 1.



Adjacency Lists

A : B, D

B : C, F

C : E, G

G : E

E : B, F

F : A

D : F

Explanation :

Minimum Path P can be found by applying breadth first search algorithm that will begin at node A and will end at E. the algorithm uses two queues, namely QUEUE1 and QUEUE2. QUEUE1 holds all the nodes that are to be processed while QUEUE2 holds all the nodes that are processed and deleted from QUEUE1.

Lets start examining the graph from Node A.

1. Add A to QUEUE1 and NULL to QUEUE2.

QUEUE1 = {A}
QUEUE2 = {NULL}

2. Delete the Node A from QUEUE1 and insert all its neighbours. Insert Node A into QUEUE2

QUEUE1 = {B, D}
QUEUE2 = {A}

3. Delete the node B from QUEUE1 and insert all its neighbours. Insert node B into QUEUE2.

QUEUE1 = {D, C, F}
QUEUE2 = {A, B}

4. Delete the node D from QUEUE1 and insert all its neighbours. Since F is the only neighbour of it which has been inserted, we will not insert it again. Insert node D into QUEUE2.

QUEUE1 = {C, F}
QUEUE2 = {A, B, D}

5. Delete the node C from QUEUE1 and insert all its neighbours. Add node C to QUEUE2.

QUEUE1 = {F, E, G}
 QUEUE2 = {A, B, D, C}

6. Remove F from QUEUE1 and add all its neighbours. Since all of its neighbours has already been added, we will not add them again. Add node F to QUEUE2.

QUEUE1 = {E, G}
 QUEUE2 = {A, B, D, C, F}

7. Remove E from QUEUE1, all of E's neighbours has already been added to QUEUE1 therefore we will not add them again. All the nodes are visited and the target node i.e. E is encountered into QUEUE2.

QUEUE1 = {G}
 QUEUE2 = {A, B, D, C, F, E}

Now, backtrack from E to A, using the nodes available in QUEUE2.

The minimum path will be **A → B → C → E**.

Real Life Example :

- Using GPS navigation system BFS is used to find neighboring places.
- In networking, when we want to broadcast some packets, we use the BFS algorithm.
- Path finding algorithm is based on BFS or DFS.
- BFS is used in Ford-Fulkerson algorithm to find maximum flow in a network.

- **Depth First Search (DFS) Algorithm :**

Defination :

Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children.

The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.

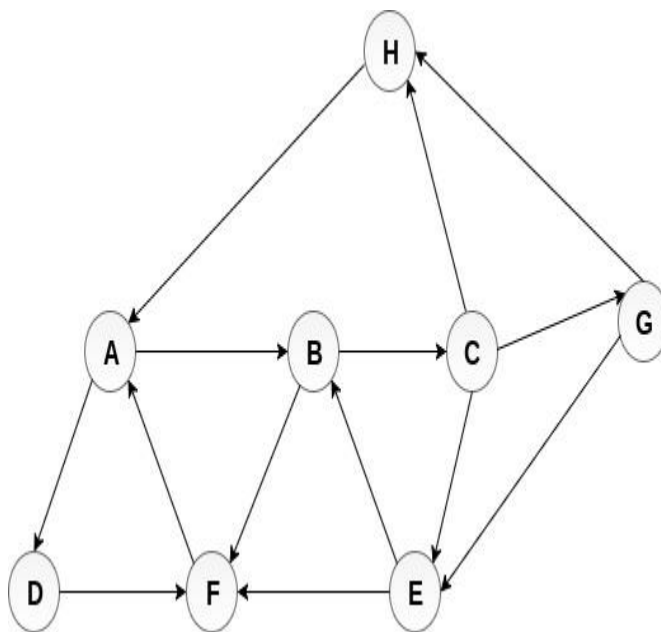
The data structure which is being used in DFS is stack. The process is similar to BFS algorithm. In DFS, the edges that leads to an unvisited node are called discovery edges while the edges that leads to an already visited node are called block edges.

Algorithm :

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Push the starting node A on the stack and set its STATUS = 2 (waitingstate)
- **Step 3:** Repeat Steps 4 and 5 until STACK is empty
- **Step 4:** Pop the top node N. Process it and set its STATUS = 3 (processed state)
- **Step 5:** Push on the stack all the neighbours of N that are in the ready state(whose STATUS = 1) and set their STATUS = 2 (waiting state)
[END OF LOOP]
- **Step 6:** EXIT

Diagram :

Consider the graph G along with its adjacency list, given in the figure below. Calculate the order to print all the nodes of the graph starting from node H, by using depth first search (DFS) algorithm.



Adjacency Lists

A : B, D

B : C, F

C : E, G, H

G : E, H

E : B, F

F : A

D : F

H : A

Explanation :

Push H onto the stack

STACK : H

POP the top element of the stack i.e. H, print it and push all the neighbours of H onto the stack that a

Print H

STACK : A

Pop the top element of the stack i.e. A, print it and push all the neighbours of A onto the stack that a

Print A

Stack : B, D

Pop the top element of the stack i.e. D, print it and push all the neighbours of D onto the stack that are in ready state.

Print D

Stack : B, F

Pop the top element of the stack i.e. F, print it and push all the neighbours of F onto the stack that are in ready state.

Print F

Stack : B

Pop the top of the stack i.e. B and push all the neighbours

Print B

Stack : C

Pop the top of the stack i.e. C and push all the neighbours.

Print C

Stack : E, G

Pop the top of the stack i.e. G and push all its neighbours.

Print G

Stack : E

Pop the top of the stack i.e. E and push all its neighbours.

Print E

Stack :

Hence, the stack now becomes empty and all the nodes of the graph have been traversed.

Print E

Stack :

Hence, the stack now becomes empty and all the nodes of the graph have been traversed.

The printing sequence of the graph will be :

$H \rightarrow A \rightarrow D \rightarrow F \rightarrow B \rightarrow C \rightarrow G \rightarrow E$

Real Life Example :

- Detecting cycle in a graph. ...
- Path Finding. ...
- Topological Sorting. ...
- To test if a graph is bipartite.

35. List out few of the Application of tree data structure?

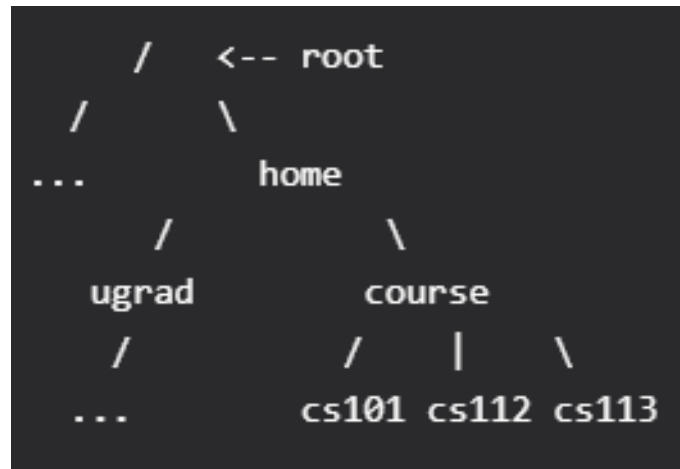
Answer: -

Unlike Array and Linked List, which are linear data structures, tree is hierarchical (or non-linear) data structure.

1. One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer:

FILE SYSTEM

2. If we form of a ordering search for moderate Linked List arrays). search Red-Black upper bound of $O(\log n)$ for search.



organize keys in tree (with some e.g., BST), we can a given key in time (quicker than and slower than Self-balancing trees like AVL and trees guarantee an

3. We can insert/delete keys in moderate time (quicker than Arrays and slower than Unordered Linked Lists). Self-balancing search trees like AVL and Red-Black trees guarantee an upper bound of $O(\log n)$ for insertion/deletion.
4. Like Linked Lists and unlike Arrays, Pointer implementation of trees don't have an upper limit on number of nodes as nodes are linked using pointers.

Other Applications:

1. Store hierarchical data, like folder structure, organization structure, XML/HTML data.
2. Binary Search Tree is a tree that allows fast search, Insert, delete on a sorted data. It also allows finding closest item
3. Heap is a tree data structure which is implemented using arrays and used to implement priority queues.

B-Tree and B+ Tree: They are used to implement indexing in databases

5. Syntax Tree: Used in Compilers.
6. K-D Tree: A space partitioning tree used to organize points in K dimensional space.
7. Trie: Used to implement dictionaries with prefix lookup.
8. Suffix Tree: For quick pattern searching in a fixed text.
9. Spanning Trees and shortest path trees are used in routers and bridges respectively in computer networks
10. As a workflow for compositing digital images for visual effects.

36. List out few of the applications that make use of multilinked structures?

Answer: -

1. Sparse matrix

Sparse matrices are those matrices that have the majority of their elements equal to zero. In other words, the sparse matrix can be defined as the matrix that has a greater number of zero elements than the non-zero elements.

The following are the advantages of using a sparse matrix:

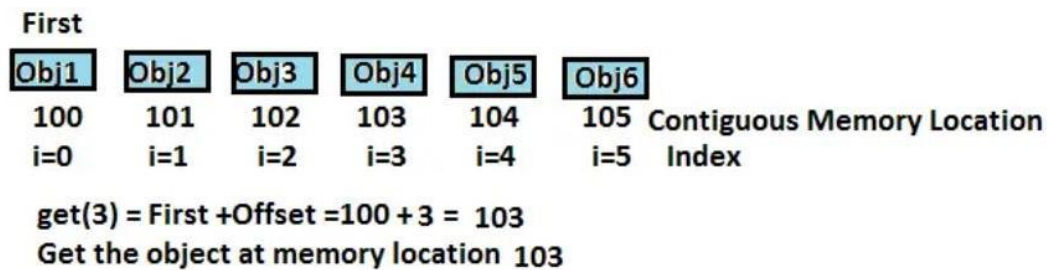
- **Storage:** As we know, a sparse matrix that contains lesser non-zero elements than zero so less memory can be used to store elements. It evaluates only the non-zero elements.
- **Computing time:** In the case of searching in a sparse matrix, we need to traverse only the non-zero elements rather than traversing all the sparse matrix elements. It saves computing time by logically designing a data structure traversing non-zero elements.

Representing a sparse matrix by a 2D array leads to the wastage of lots of memory. The zeroes in the matrix are of no use to store zeroes with non-zero elements. To avoid such wastage, we can store only non-zero elements. If we store only non-zero elements, it reduces the traversal time and the storage space.

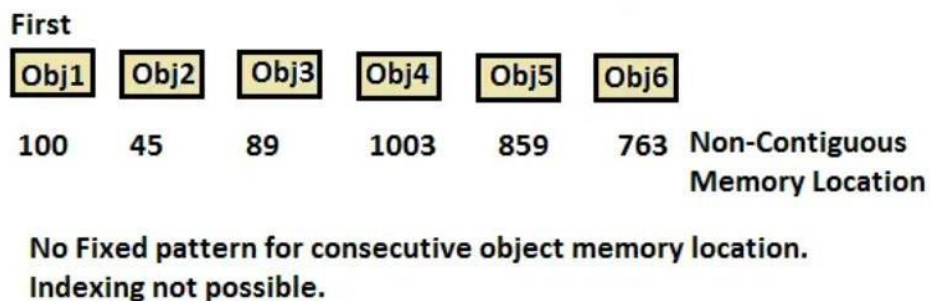
2. Index generation.

Linked list is a linear data structure whose order is not given by the physical location of object. Data stored at non-contiguous memory locations, so we cannot get the elements of linked list by passing the position. Refer the below graphical explanation.

Index Based Data Structures (Ex - Arrays)



Non-Indexed Data Structures (Ex - Linked List)



Since, there is no fixed pattern for consecutive object memory location, indexing is not possible in linked list. That's why accessing the elements in linked list is costly operation. But still we can access the elements by position in linked list through traversing the entire linked list.

37]. In tree construction which is the suitable efficient data structure? (Array, Linked list, Stack, Queue)

Linked list is the suitable efficient data structure.

Tree is hierarchical data structure. Like linked list tree also maintain pointers on nodes that's why Linked list is the suitable efficient data structure for tree's.

38. Explain the type of the algorithm used in solving the 8 Queens problem?

To solve this problem, we will make use of the **Backtracking algorithm**.

Definition :- The backtracking algorithm, in general checks all possible configurations and test whether the required result is obtained or not. For the given problem, we will explore all possible positions the queens can be relatively placed at. The solution will be correct when the number of placed queens = 8.

The good example of the use of backtracking is the eight queens puzzle, that asks for all arrangements of eight queens on a standard chessboard so that no queen attacks any other.

STEPS REVISITED – BACKTRACKING

1. Place the first queen in the left upper corner of the table.
2. Save the attacked positions.
3. Move to the next queen (which can only be placed to the next line).
4. Search for a valid position. If there is one go to step 8.
5. There is not a valid position for the queen. Delete it (the x coordinate is 0).

6. Move to the previous queen.
7. Go to step 4.8. Place it to the first valid position.
9. Save the attacked positions.
10. If the queen processed is the last stop otherwise go to step 3.

Que.39] In an AVL tree, at what condition the balancing is to be done?

AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.

Definition: AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

Explanation: Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

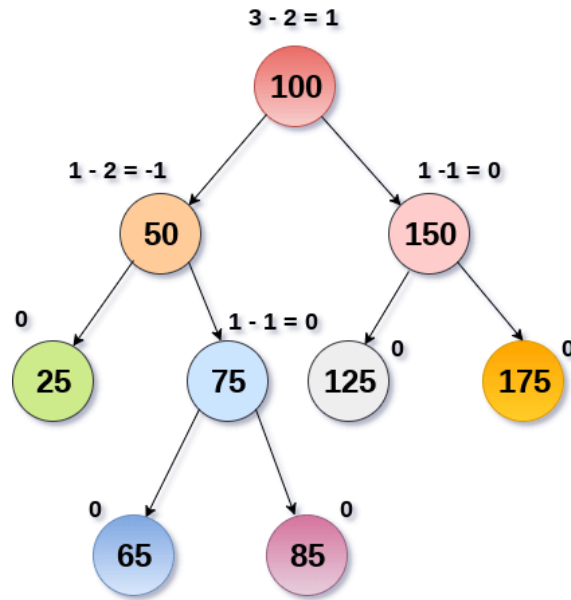
Balance Factor (k) = height (left(k)) - height (right(k))

If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

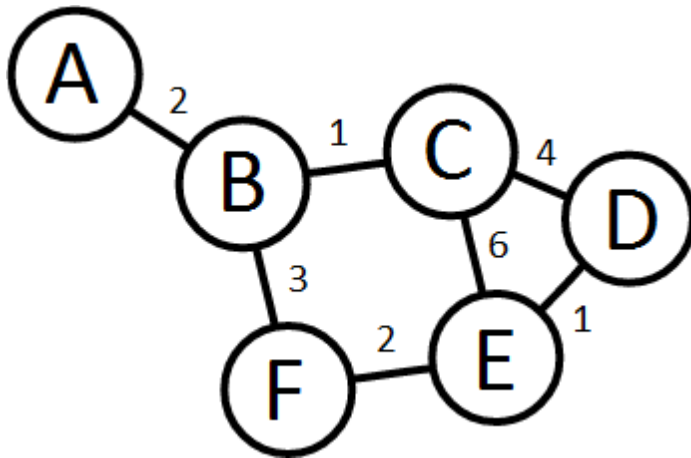
An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.



AVL Tree

Que.40] Does the minimum spanning tree of graph give the shortest distance among any 2 specified nodes?

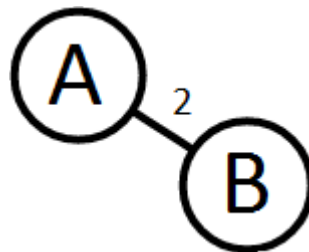
Minimum Spanning Trees:



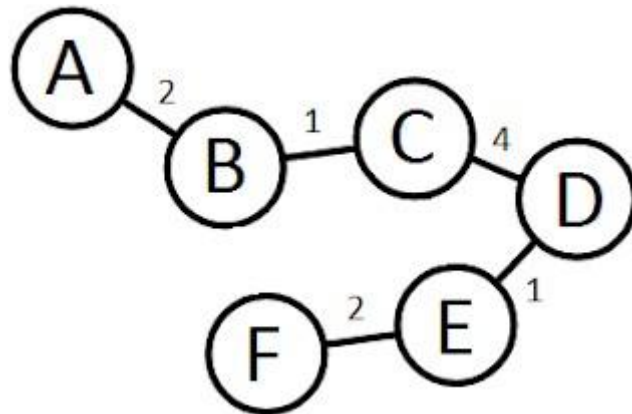
A graph is a way of representing relationships between multiple objects. One great analogy to help understand what this means is to think of the image above as a map of towns and the roads that connect them. (If you want to sound super smart for the rest of this post, just mentally replace the word “town” with “vertex” and “road” with “edge”.) Specifically, this graph above can be called an undirected, weighted, cyclic graph. Undirected means the roads are all two way roads. I can get from town A to town B and then back to town A if I wanted to. Weighted means that each of the roads has a certain distance associated with it. Traveling from town A to town B is two miles, while going from town B to town F is three miles and so on. Cyclic means that there is a connected chain of at least 3 towns. In this case it could be the chain C → D → E → C or the much longer chain of B → C → D → E → F → B.

What is minimum spanning tree?

In general, a tree is any part of the graph that doesn't have a cycle in it. Here are some examples of trees from the same graph.

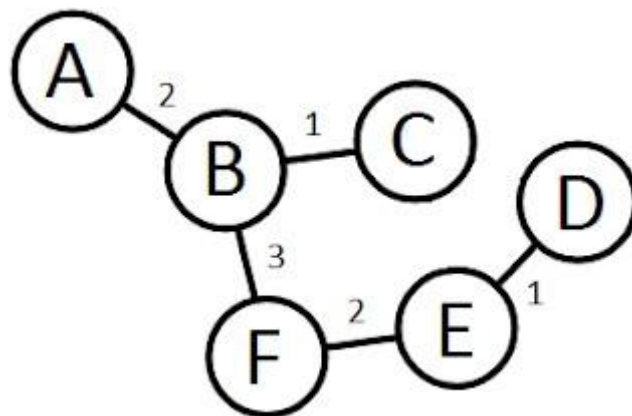


This one is a mini tree.



This second tree is a little more interesting, though, because it forms what is known as a **Spanning Tree**. This is a sub-graph that forms a tree while still including every town. The only thing that we removed were the roads which were causing the cycles.

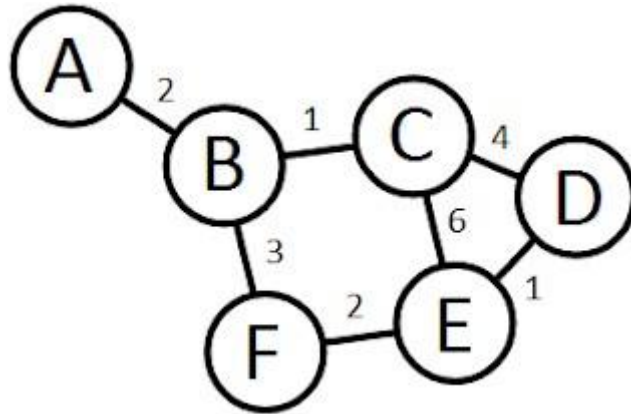
It should be noted that there can be multiple spanning trees in a single graph, depending on exactly which edges are removed. We can take this a step further by constructing something called a **Minimum Spanning Tree**, which is a spanning tree that has the lowest total distance of all the roads left. There are a couple of different ways to build this, but in general we start with a single town and connect the others by always choosing the smallest edge possible. Creating the minimum spanning tree from the graph given above results in this beauty:



In this case, the total distance of the roads is 9 miles. For reference the spanning tree above has a total distance of 10 miles. The cool part about minimum spanning tree is that they are super useful in modeling real world problems. Think about the last time you moved and had to wire up your cable modem. Finding the route which can physically connect all of your different devices using the least amount of Ethernet cable describes exactly what a minimum spanning tree is. In general these can model anything that has to connect a specific number of objects using the least amount of product. This could be road networks, computer wiring, indoor plumbing; the list goes on and on.

Shortest Path(graphs part two):

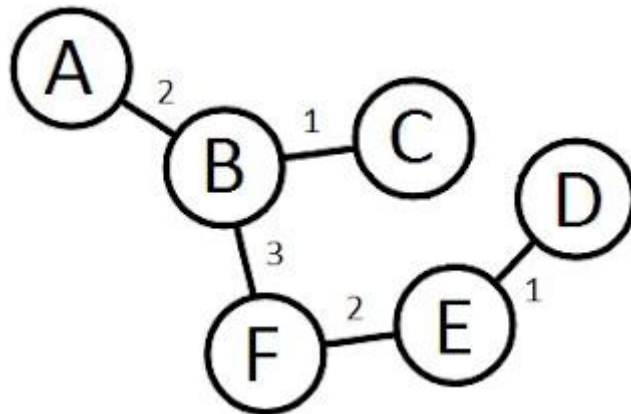
A lot of the time when we talk about towns and roads we want to focus on one specific problem: how can someone get from town A to town B as quickly as possible. The solution to this is known as the **Shortest Path**. Lets look at the original graph again.



Whats the shortest path from B to E? There are a couple of possibilities to consider. First, we could follow B -> F -> E and travel 5 miles. Another choice would be to go from B -> C -> E for a total of 7 miles. Finally, we could travel from B -> C -> D -> E for a total of 8 miles. In this case, the first option is the shortest path. There are a couple of different ways to find the shortest path between any two nodes, most notably [Dijkstra's](#) and [Bellman-Ford](#).

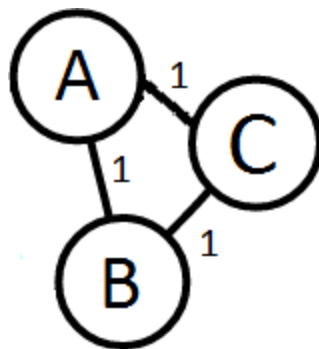
Does the Minimum Spanning Tree contain the Shortest Path between any two nodes?

Remember that when building the minimum spanning tree, we specifically chose the edges in order from smallest to largest. Like we talked about above, this means that the minimum spanning tree is guaranteed to have the lowest possible total cost of all the edges. Given this it seems reasonable to assume that it is also the shortest path between any two nodes. The tricky part is that it **is** the shortest path for some nodes, but we can't rely on it being always true. To see what I mean lets look at our minimum spanning tree again.



What is the shortest path from B to E? In the full graph there were three different ways to get there, with the shortest taking only 5 miles. In our minimum spanning tree, the same path exists and it is still 5 miles long. So, in general, the minimum spanning tree will hold **some** of the shortest paths.

Let's look at the path between C and D. In the original graph there was a direct connection C → D which had a distance of 4 miles. However, in the minimum spanning tree we need to move from C → B → F → E → D for a total distance of 7 miles! To understand why we can't guarantee every shortest path is included in the minimum spanning tree let's look at another super simple example.



Here we have a graph with a cycle in it. You can see that the distance between any two nodes in the original graph is one. To create a true Minimum Spanning Tree for this graph we just need to remove a single edge. No matter which edge we remove to do this, however, there will always be a pair of nodes which now has to take the long way around. The reason why this happens is the minimum spanning tree is only concerned with lowering the **total** cost of the edges. It doesn't optimize for specific node pairings. Infact, it can't optimize for these pairings, because if it did it wouldn't be able to guarantee we actually remove every cycle.

Conclusion:

As we've seen, the Minimum Spanning Tree doesn't contain the shortest path between any two arbitrary nodes, although it probably will contain the shortest path between a few nodes.

41. Which is the simplest file structure? (Sequential, Indexed, Random)

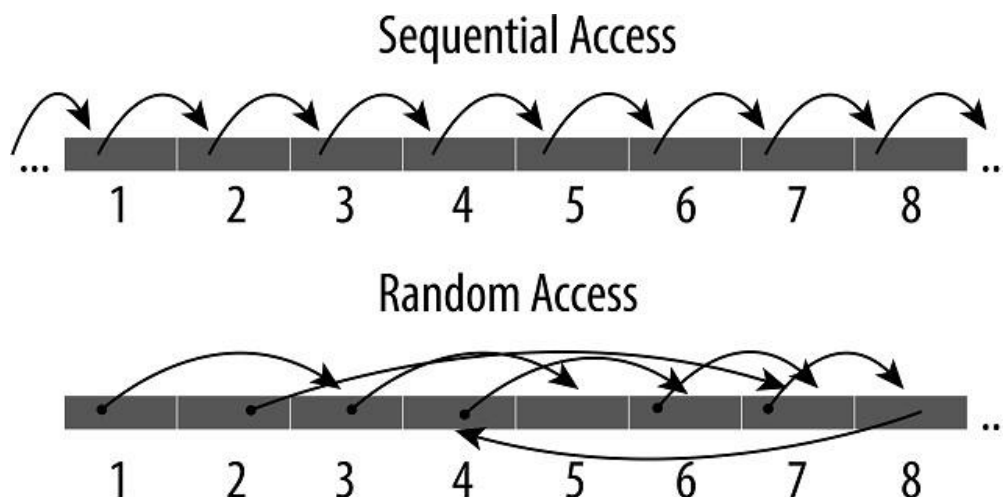
Sequential is the simplest file structure.

A sequential file contains records organized by the order in which they were entered. The order of the records is fixed.

Records in sequential files can be read or written only sequentially.

After you place a record into a sequential file, you cannot shorten, lengthen, or delete the record. However, you can update (REWRITE) a record if the length does not change. New records are added at the end of the file.

If the order in which you keep records in a file is not important, sequential organization is a good choice whether there are many records or only a few. Sequential output is also useful for printing reports.



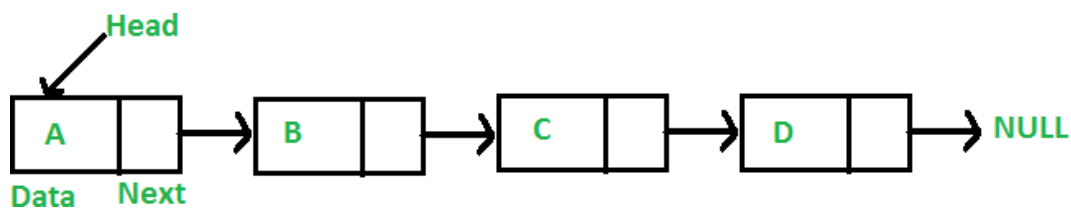
A common example of sequential access is with a [tape drive](#), where the device must move the tape's ribbon forward or backward to reach the desired information. The opposite would be [RAM](#) (Random Access Memory) going anywhere on the chip to access the information.

42. Whether Linked List is linear or Non-linear data structure?

Linked List is a linear data structure.

A linear data structure is a structure in which the elements are stored sequentially, and the elements are connected to the previous and the next element. As the elements are stored sequentially, so they can be traversed or accessed in a single run. The implementation of linear data structures is easier as the elements are sequentially organized in memory. The data elements in an array are traversed one after another and can access only one element at a time.

Linked list has data element and reference to the next node in the sequence hence it is a linear data structure.

**Applications of linked list in real world-**

1. *Image viewer* – Previous and next images are linked, hence can be accessed by next and previous button.
2. *Previous and next page in web browser* – We can access previous and next URL searched in web browser by pressing back and next button since, they are linked as linked list.
3. *Music Player* – Songs in music player are linked to previous and next song. you can play songs either from starting or ending of the list.

43. Explain the difference among the HEAP and the STACK?

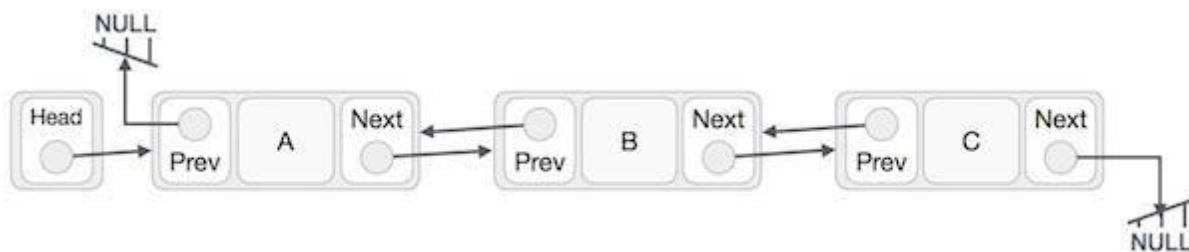
Stack Memory		Heap Space
Application	It stores items that have a very short life such as methods , variables , and reference variables of the objects.	It stores objects and Java Runtime Environment (JRE) classes.
Ordering	It follows the LIFO order.	It does not follow any order because it is a dynamic memory allocation and does not have any fixed pattern for allocation and deallocation of memory blocks.
Flexibility	It is not flexible because we cannot alter the allocated memory.	It is flexible because we can alter the allocated memory.
Efficiency	It has faster access, allocation, and deallocation.	It has slower access, allocation, and deallocation.
Memory Size	It is smaller in size.	It is larger in size.
Java Options Used	We can increase the stack size by using the JVM option <code>-Xss</code> .	We can increase or decrease the heap memory size by using the <code>-Xmx</code> and <code>-Xms</code> JVM options.
Visibility or Scope	The variables are visible only to the owner thread.	It is visible to all threads.
Generation of Space	When a thread is created, the operating system automatically allocates the stack.	To create the heap space for the application, the language first calls the operating system at run time.
Distribution	Separate stack is created for each object.	It is shared among all the threads.
Exception Throws	JVM throws the java.lang.StackOverflowError if the stack size is greater than the limit. To avoid this error, increase the stack size.	JVM throws the java.lang.OutOfMemoryError if the JVM is unable to create a new native method.
Allocation/Deallocation	It is done automatically by the compiler .	It is done manually by the programmer .
Cost	Its cost is less .	Its cost is more in comparison to stack.

Implementation	Its implementation is hard .	Its implementation is easy .
Order of allocation	Memory allocation is continuous .	Memory allocated in random order.
Thread-Safety	It is thread-safe because each thread has its own stack.	It is not thread-safe, so proper synchronization of code is required.

Q44 Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, either forward and backward easily as compared to SingleLinked List. Following are the important terms to understand the concept of doubly linked list.

- **Link** – Each link of a linked list can store a data called an element.
- **Next** – Each link of a linked list contains a link to the next link called Next.
- **Prev** – Each link of a linked list contains a link to the previous link called Prev.
- **LinkedList** – A Linked List contains the connection link to the first link called First and to the last link called Last.

Doubly Linked List Representation



As per the above illustration, following are the important points to be considered.

- Doubly Linked List contains a link element called first and last.
- Each link carries a data field(s) and two link fields called next and prev.
- Each link is linked with its next link using its next link.
- Each link is linked with its previous link using its previous link.
- The last link carries a link as null to mark the end of the list.

Basic Operations

Following are the basic operations supported by a list.

- **Insertion** – Adds an element at the beginning of the list.
- **Deletion** – Deletes an element at the beginning of the list.

- **Insert Last** – Adds an element at the end of the list.
- **Delete Last** – Deletes an element from the end of the list.
- **Insert After** – Adds an element after an item of the list.
- **Delete** – Deletes an element from the list using the key.
- **Display forward** – Displays the complete list in a forward manner.
- **Display backward** – Displays the complete list in a backward manner.

Insertion Operation

Following code demonstrates the insertion operation at the beginning of a doubly linkedlist.

Example

```
//insert link at the first location
void insertFirst(int key, int data){

//create a link
struct node *link = (struct node*) malloc(sizeof(struct node));
    link->key = key;
    link->data = data;

    if(isEmpty()){
//make it the last link
last = link;
    }else{
//update first prev link
        head->prev = link;
    }

//point it to old first link
    link->next = head;

//point first to new first link
    head = link;
}
```

Deletion Operation

Following code demonstrates the deletion operation at the beginning of a doubly linkedlist.

Example

```

//delete first item
struct node*deleteFirst(){

//save reference to first link
struct node *tempLink= head;

//if only one link
if(head->next== NULL){
last= NULL;
}else{
    head->next->prev= NULL;
}

    head = head->next;

//return the deleted link
returntempLink;
}

```

Insertion at the End of an Operation

Following code demonstrates the insertion operation at the last position of a doublylinked list.

Example

```

//insert link at the last location
voidinsertLast(int key,int data){

//create a link
struct node *link =(struct node*)malloc(sizeof(struct node));
    link->key = key;
    link->data = data;

if(isEmpty()){
//make it the last link
last= link;
}else{
//make link a new last link
last->next= link;

//mark old last node as prev of new link
    link->prev=last;
}

//point last to new last node

```

```
last= link;
}
```

A doubly linked list can be applied in various real-life scenarios and applications as discussed below.

- A Deck of cards in a game is a classic example of a doubly linked list. Given that each card in a deck has the previous card and next card arranged sequentially, this deck of cards can be easily represented using a doubly linked list.
- Browser history/cache — The browser cache has a collection of URLs and can be navigated using the forward and back buttons is another good example that can be represented as a doubly linked list.
- Most recently used (MRU) also can be represented as a doubly linked list.
- Other data structures like Stacks, hash table, the binary tree can also be constructed or programmed using a doubly linked list.

45. How do you insert a record among two nodes in double linked list?

Ans: In computer science, a doubly linked list is a linked data structure that consists of a set of sequentially linked records called nodes. Each node contains three fields: two link fields (references to the previous and to the next node in the sequence of nodes) and one data field. The beginning and ending nodes' previous and next links, respectively, point to some kind of terminator, typically a sentinel node or null, to facilitate traversal of the list. If there is only one sentinel node, then the list is circularly linked via the sentinel node. It can be conceptualized as two singly linked lists formed from the same data items, but in opposite sequential orders.

Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), pointer to the previous node (previous pointer). A sample node in a doubly linked list is shown in the figure.

2- Advantages over singly linked list -

1) A DLL can be traversed in both forward and backward direction.

- 2) The delete operation in DLL is more efficient if pointer to the node to be deleted is given.
- 3) We can quickly insert a new node before a given node.

In singly linked list, to delete a node, pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using previous pointer.

3-Insertion -

A node can be added in four ways

- 1) At the front of the DLL
- 2) After a given node.
- 3) At the end of the DLL
- 4) Before a given node.

4-/ Adding a node at the front of the list

```
public void push(int new_data)
{
    /* 1. allocate node
    * 2. put in the data */
    Node new_Node = new Node(new_data);

    /* 3. Make next of new node as head and previous as NULL */new_Node.next =
    head;
    new_Node.prev = null;

    /* 4. change prev of head node to new node */if
    (head != null)
        head.prev = new_Node;

    /* 5. move the head to point to the new node */head
    = new_Node;
}
```

EXAMPLE-1-A music player which has next and previous buttons.

2- Undo Redo operations.

CODE- Class for Doubly Linked List

```
public class DLL {
    Node head; // head of list

    /* Doubly Linked list Node*/
    class Node {
        int data;
        Node prev;
        Node next;
```

```

    // Constructor to create a new node
    // next and prev is by default initialized as null
    Node(int d) { data = d; }
}
}

```

46. In which data structure, elements can be added or removed at either end, but not in the middle?

Ans: INSERTION OF RECORD-

ALGO- Algorithm :

Step 1: IF ptr = NULL

Write OVERFLOW

Go to Step 9

[END OF IF]

Step 2: SET NEW_NODE = ptr

Step 3: SET ptr = ptr -> NEXT

Step 4: SET NEW_NODE -> DATA = VAL Step

5: SET NEW_NODE -> PREV = NULL Step 6:

SET NEW_NODE -> NEXT = START Step 7:

SET head -> PREV = NEW_NODE

Step 8: SET head = NEW_NODE

Step 9: EXIT

ANS 2 -Add a node after a given node-(A

7 steps process)

We are given pointer to a node as prev_node, and the new node is inserted after the given node.

dll_add_middle

```

/* Given a node as prev_node, insert a new node after the given node */public void

```

```

InsertAfter(Node prev_Node, int new_data)

```

```

{

```

```

    /*1. check if the given prev_node is NULL */if

```

```

    (prev_Node == null) {

```

```

        System.out.println("The given previous node cannot be NULL ");

```

```

        return;

```

```

    }

```

```

    /* 2. allocate node

```

```

* 3. put in the data */
Node new_node = new Node(new_data);

/* 4. Make next of new node as next of prev_node */new_node.next
= prev_Node.next;

/* 5. Make the next of prev_node as new_node */prev_Node.next =
new_node;

/* 6. Make prev_node as previous of new_node */
new_node.prev = prev_Node;

/* 7. Change previous of new_node's next node */if
(new_node.next != null)
    new_node.next.prev = new_node;
}

```

47. Which one is faster? A binary search of a ordered set of elements in an array or a sequential search of elements?

Ans: A binary search of a ordered set of elements in an array is faster than a sequential search of elements

	Binary Search	Sequential Search																													
Definition	Binary search is an efficient algorithm for finding an item from a sorted list of items. It works by repeatedly dividing in half the portion of the list that could contain the item, until you've narrowed down the possible locations to just one	a linear search or sequential search is a method for finding an element within a list. It sequentially checks each element of the list until a match is found or the whole list has been searched																													
Diagram	Search 23 0 1 2 3 4 5 6 7 8 9 <table><tr><td>2</td><td>5</td><td>8</td><td>12</td><td>16</td><td>23</td><td>38</td><td>56</td><td>72</td><td>91</td></tr></table> 23 > 16 take 2nd half L= 0 1 2 3 M=4 5 6 7 8 H= 9 <table><tr><td>2</td><td>5</td><td>8</td><td>12</td><td>16</td><td>23</td><td>38</td><td>56</td><td>72</td><td>91</td></tr></table> 23 > 56 Take 1st Half 0 1 2 3 4 L=5 6 M= 7 8 H=9 <table><tr><td>2</td><td>5</td><td>8</td><td>12</td><td>16</td><td>23</td><td>38</td><td>56</td><td>72</td><td>91</td></tr></table> Found 23	2	5	8	12	16	23	38	56	72	91	2	5	8	12	16	23	38	56	72	91	2	5	8	12	16	23	38	56	72	91
2	5	8	12	16	23	38	56	72	91																						
2	5	8	12	16	23	38	56	72	91																						
2	5	8	12	16	23	38	56	72	91																						

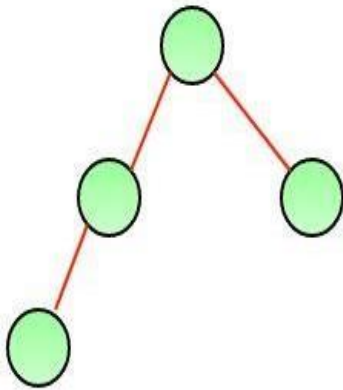
	$M = (L+H)/2$ where L is lowest index value and H is highest index value. If your key value not found on M then it compare with M and divide sequence as $M < K$ then $L = M+1$ and H is same and if $M > K$ then $H = M-1$ and L is same hence it is also called half-interval search ii. Time complexity is $O(\log n)$ iii. Find the key present at center position in constant time. iv. The element must be sorted in the container v. It cannot be implemented directly into the linked list. We need to change the basic rules of the list to implement this vi. Algorithm technique is Divide and Conquer. vii. Algorithm is slightly complex. It takes more amount of code to implement. viii. Log n number of comparisons are sufficient in worst case.	item one by one, in the order they existed in the list, until you found the item you were looking for. ii. Time complexity is $O(n)$ iii. Find the key present at first position in constant time. iv. Sequence of elements in the container does not affect v. Arrays and linked lists can be used to implement this vi. Algorithm is iterative in nature vii. Algorithm is easy to implement, and requires less amount of code viii. N number of comparisons are required for worst case.
Code Example	<pre> public class BinarySearch { static int bsearch(int a1[],int x,int l,int h) { if(h>=0) { int mid=((l+h)/2); if(a1[mid]==x) { return mid; } else if(a1[mid]>x) { return bsearch(a1,x,l,mid-1); } else return bsearch(a1,x,mid+1,h); } } return -1; } public static void main(String[] args) { </pre>	<pre> public static void Search(int arr[],int k) { int f=0; for(int i=0;i<arr.length;i++) { if(arr[i]==k) { f=1; break; } f=0; } if(f==1) System.out.println("Your number found"); else System.out.println("Your number not found"); } </pre>

	<pre> int arr[]={2,5,8,12,16,23,38,56,72,91}; int n=arr.length; int key=23; int result=bsearch(arr,key,0,n-1); if(result!=-1) System.out.println("Element present"); else System.out.println("Element found at index: "+result); } Output: Element found at index:5 </pre>	<pre> arr[]= } public static void main(String[] args) { int arr[]= {2,3,4,10,40,48,50}; int n=arr.length; int key=40; Search(arr,key); } Output:Your number found </pre>
Real time example	Find Page Number: This might be the most common real-life example of binary search. Suppose we want to go to page 345 in a book. We don't visit every page from 1 to 345. We open any random page on the book and check it's page number. If the current page number is greater than 345, then 345 is on the left side of the current page. If the current page number is smaller than 345, then 345 is on the right side of the current page. We keep repeating the process until we reach page 345.	Find Phone number: If you are asked to find the name of the person having phone number say "1234" With the help of telephone directory. Since the telephone directory is sorted by name not by numbers, we have to go through each and every number of the directory

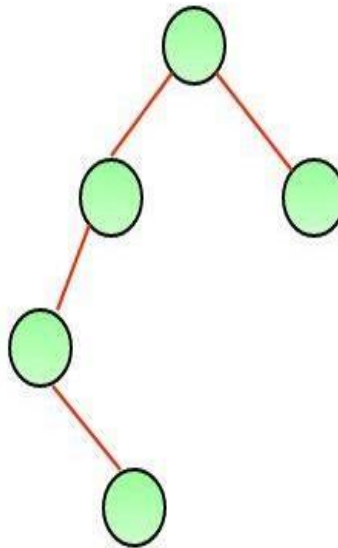
48. Explain a balanced tree?

Definition : A tree where no leaf is much farther away from the root than any other leaf. Different balancing schemes allow different definitions of “much farther and different amounts of work to keep them balanced.

Diagram:



A height balanced tree



Not a height balanced tree

Explain: Consider a height-balancing scheme where following conditions should be checked to determine if a binary tree is balanced.

An empty tree is height-balanced. A non- empty binary tree T is balanced if: 1) Left subtree of T is balanced

2) Right subtree of T is balanced

3) The difference between heights of left subtree and right subtree is not more than 1.

The above height-balancing scheme is used in AVL trees. The Diagram shows two trees, one of them is height-balanced and other is not. The second tree is not height-balanced because height of left subtree is 2 more than height of right subtree.

To check if a tree is height-balanced, get the height of left and right subtrees. Return true if difference between heights is not more than 1 and left and right subtrees are balanced, otherwise return false.

Code example:

```

class Node{
int data;

```

```

Node left,right;
Node(int d)
{
    data=d;
    left=right=null;
}
}
class BinaryTree{
Node root;
boolean isBalanced(Node node)
{
    int lh;
    int rh;
    if(node==null)
        return true;
    lh=height(node.left); //get height of left sub trees
    rh=height(node.right); //get height of right sub trees
    if(Math.abs(lh-rh)<=1
    && isBalanced(node.left)
    && isBalanced(node.right))
        return true;
    //if we reach here then tree is not height-balanced
    return false;
}
int height(Node node)
{
    if (node==null) //if tree is empty
        return 0;
    return 1+Math.max(height(node.left),height(node.right));/*if
    height=1+max of left height and right height*/
    tree is not empty
}

```

```

public static void main(String args[ ])
{
    BinaryTree tree=new BinaryTree();
    tree.root=new Node(1);
    tree.root.left=new Node(2);
    tree.root.right=new Node(3);
    tree.root.left.left=new Node(4);
    tree.root.left.right=new Node(5);
    tree.root.left.left.left=new Node(8);
    if(tree.isBalanced(tree.root))
        system.out.println("Tree is balanced");
    else
        system.out.println("Tree is not balanced");
}
}

```

Output: Tree is not balanced

Real time example:

Producers/consumers often use a balanced tree implementation to store a document in memory.

Computer chess games build a huge tree (training) which they prune at runtime using heuristics to reach an optimal move.

49. Which data structure is needed to convert infix notation to post fix notations? Ans :

Stack data structure is needed to convert infix notation to post fix notation

Defination : A stack is a container of objects that are inserted and removed according to LIFO(last in first out)

Infix:-

An infix expression is a single letter, or an operator, preceded by one infix string and followed by another infix string.

Postfix:-

A postfix expression (also called Reverse Polish Notation) is a single letter or an operator, preceded by two postfix strings. Every postfix string longer than a single variable contains first and second operands followed by an operator.

Diagram:

$a - (b + c * d) / e$ to		
ch	stack (bottom to top)	postfixExp
a		a
-	-	a
(	-(	a
b	-(	ab
+	-(+	ab
c	-(+	abc
*	-(+ *	abc
d	-(+ *	abcd
)	-(+	abcd*
	-(	abcd*+
	-	abcd*+
/	- /	abcd*+
e	- /	abcd*+e
		abcd*+e/-

Explanation:

problem can be resolved by following the operator precedence rules.

Operators	Symbols
Parenthesis	(), { }, []
Exponents	^
Multiplication and Division	*, /
Addition and Subtraction	+, -

The first preference is given to the parenthesis; then next preference is given to the exponents. In the case of multiple exponent operators, then the operation will be applied from right to left. After exponent, multiplication, and division operators are evaluated. The next preference is given to addition and subtraction. The operators that have the same precedence termed as **operator associativity**. If we go from left to right, then it is known as left-associative. If we go from right to left, then it is known as right-associative.

1. Print the operand as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the incoming operator on to the stack.

3. If the incoming symbol is '(', push it on to the stack.
4. If the incoming symbol is ')', pop the stack and print the operators until the left parenthesis is found.
5. If the incoming symbol has higher precedence than the top of the stack, push it on the stack.
6. If the incoming symbol has lower precedence than the top of the stack, pop and print the top of the stack. Then test the incoming operator against the new top of the stack.
7. If the incoming operator has the same precedence with the top of the stack then use the associativity rules. If the associativity is from left to right then pop and print the top of the stack then push the incoming operator. If the associativity is from right to left then push the incoming operator.
8. At the end of the expression, pop and print all the operators of the stack.

Infix expression: 2 + 3 * 4

Post fix expression: 234*+

Code:

```
import java.io.*;
class Stack
{
char a[]=new char[100];
int top=-1;
void push(char c)
{
try
{
a[++top]= c;
}
catch(StringIndexOutOfBoundsException e)
{
System.out.println("Stack full, no room to push, size=100");
System.exit(0);
}
}
char pop()
{
```

```

return a[top--];
}
boolean isEmpty()
{
return (top== -1)?true:false;
}
char peek()
{
return a[top];
}
}
public class InfixToPostfix
{
static Stack operators = new Stack();
public static void main(String argv[]) throws IOException
{
String infix;
//create an input stream object
BufferedReader keyboard = new BufferedReader (new InputStreamReader(System.in));
//get input from user
System.out.print("\nEnter the infix expression you want to convert: ");infix
= keyboard.readLine();
//output as postfix
System.out.println("Postfix expression for the given infix expression is:" + toPostfix(infix));
}
private static String toPostfix(String infix)
//converts an infix expression to postfix
{
char symbol; String
postfix = "";
for(int i=0;i<infix.length();++i)
//while there is input to be read
{
symbol = infix.charAt(i);
//if it's an operand, add it to the stringif
(Character.isLetter(symbol)) postfix =
postfix + symbol;
else if (symbol=='(')
//push (
{

```

```

operators.push(symbol);
}
else if (symbol=='')
//push everything back to (
{
while (operators.peek() != '(')
{
postfix = postfix + operators.pop();
}
operators.pop();      //remove '('
}
else
//print operators occurring before it that have greater precedence
{
while (!operators.isEmpty()    &&  !(operators.peek()=='(')    &&  prec(symbol)    <=
prec(operators.peek()))
postfix = postfix + operators.pop();
operators.push(symbol);
}
}
while (!operators.isEmpty()) postfix
= postfix + operators.pop();return
postfix;
}
static int prec(char x)
{
if (x == '+' || x == '-')
return 1;
if (x == '*' || x == '/' || x == '%')
return 2;
return 0;
}
}

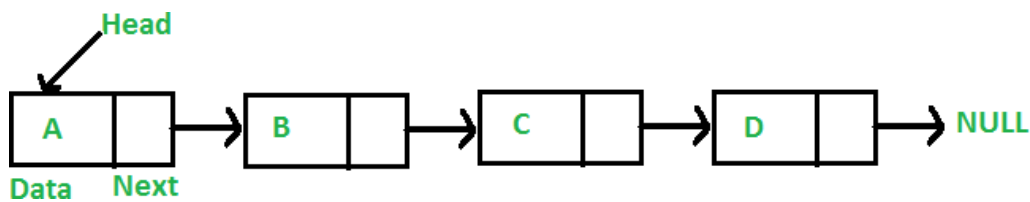
```

50. which data structures can we implement using linked list?

Ans: we can implement stack, queue and graph data structures using linked list

Definition: A linked list is a linear data structure, in which elements are not stored at contiguous memory location. The elements in a linked list are linked using pointers.

Diagram:



Explanation:

A linked list consists of nodes where each node contains a data field and a reference (link) to the next node in the list.

Code :

```

// Java program to Implement a stack
// using singly linked list
// import package
import static java.lang.System.exit;
// Create Stack Using Linked list
class StackUsingLinkedList {
    // A linked list node
    private class Node {
        int data; // integer data
        Node link; // reference variable Node type
    }
    // create global top reference variable global
    Node top;
    // Constructor
    StackUsingLinkedList()
  
```

```

{
    this.top = null;
}
// Utility function to add an element x in the stack
public void push(int x) // insert at the beginning
{
    // create new node temp and allocate memory
    Node temp = new Node();
    // check if stack (heap) is full. Then inserting an
    // element would lead to stack overflowif
    (temp == null) {
        System.out.print("\nHeap Overflow");
        return;
    }
    // initialize data into temp data field
    temp.data = x;
    // put top reference into temp link
    temp.link = top;
    // update top reference
    top = temp;
}
// Utility function to check if the stack is empty or not
public boolean isEmpty()
{
    return top == null;
}
// Utility function to return top element in a stack
public int peek()
{
    // check for empty stackif
    (!isEmpty()) {
        return top.data;
    }
    else {
        System.out.println("Stack is empty");
        return -1;
    }
}

// Utility function to pop top element from the stack
public void pop() // remove at the beginning

```

```

    {
        // check for stack underflowif
        (top == null) {
            System.out.print("\nStack Underflow");
            return;
        }
        // update the top pointer to point to the next node
        top = (top).link;
    }
    public void display()
    {
        // check for stack underflowif
        (top == null) {
            System.out.printf("\nStack Underflow");
            exit(1);
        }
        else {
            Node temp = top;
            while (temp != null) {
                // print node data
                System.out.printf("%d->", temp.data);
                // assign temp link to temp
                temp = temp.link;
            }
        }
    }
}

// main class public
class GFG {
    public static void main(String[] args)
    {
        // create Object of Implementing class StackUsingLinkedlist
        obj = new StackUsingLinkedlist();
        // insert Stack value
        obj.push(11);
        obj.push(22);
        obj.push(33);
        obj.push(44);
        // print Stack elements
        obj.display();
    }
}

```

```

        // print Top element of Stack
        System.out.printf("\nTop element is %d\n", obj.peek());
        // Delete top element of Stack
        obj.pop();
        obj.pop();
        // print Stack elements
        obj.display();
        // print Top element of Stack
        System.out.printf("\nTop element is %d\n", obj.peek());
    }
}

```

Real time Examples of linked list:

1. Previous and next page in web browser — We can access previous and next url searched in web browser by pressing back and next button since, they are linked as linked list.
2. Undo functionality in photoshop or microsoft word uses linked list.

51. Explain the bucket size, when the overlapping and collision occur at same time? Ans:

Definition:

- **Bucket** – A hash file stores data in bucket format. Bucket is considered a unit of storage. A bucket typically stores one complete disk block, which can store one or more records
- The condition of bucket-overflow is known as **collision**.

Example:

Let's take one example using the hash function $\text{key mod } 7$, insert the following

Sequence of key in the hash table-

50, 700, 76, 85, 92, 73 and 101

Step-1:

- Draw an empty hash table.
- For the given hash function, the possible range of hash values is $[0, 6]$.
- So, draw an empty hash table consisting of 7 buckets as-

0	
1	
2	
3	
4	
5	
6	

Step-2:

- Insert the given keys in the hash table one by one.
- The first key to be inserted in the hash table = 50.
- Bucket of the hash table to which key 50 maps = $50 \bmod 7 = 1$.
- So, key 50 will be inserted in bucket-1 of the hash table as-

0	
1	50
2	
3	
4	
5	
6	

Step-3:

- The next key to be inserted in the hash table = 700.
- Bucket of the hash table to which key 700 maps = $700 \bmod 7 = 0$.
- So, key 700 will be inserted in bucket-0 of the hash table as-

0	700
1	50
2	
3	
4	
5	
6	

Step-4:

- The next key to be inserted in the hash table = 76.
- Bucket of the hash table to which key 76 maps = $76 \bmod 7 = 6$.
- So, key 76 will be inserted in bucket-6 of the hash table as-

0	700
1	50
2	
3	
4	
5	
6	76

Step-5:

- The next key to be inserted in the hash table = 85.
- Bucket of the hash table to which key 85 maps = $85 \bmod 7 = 1$.
- Since bucket-1 is already occupied, so collision occurs.

If there is only one entry possible in the bucket, when the collision occurs, there is no way to accommodate the colliding value. This results in the **overlapping** of values.

52. Classify the Hashing functions based on the various methods by which the key value is found.

Ans:

Definition:

A hash function is a mathematical formula which, when applied to a key, produces an integer which can be used as an index for the key in the hash table. The main aim of a hash function is that elements should be relatively, randomly, and uniformly distributed. It produces a unique set of integers within some suitable range in order to reduce the number of collisions. In practice, there is no hash function that eliminates collisions completely. A good hash function can only minimize the number of collisions by spreading the elements uniformly throughout the array.

These are the hashing functions based on the various methods by which the key value is found.

1. Direct method
2. Modulo-Division method
3. Digit-Extraction method
4. Mid-Square method
5. Folding method

1.Direct Method:

In direct hashing the key is the address without any algorithmic manipulation.

Direct hashing is limited, but it can be very powerful because it guarantees that there are no synonyms and therefore no collision.

2.Modulo-Division method

This is also known as division remainder method.

This algorithm works with any list size, but a list size that is a prime number produces fewer collisions than other list sizes.

The formula to calculate the address is:

Address = key MODULO list size + 1

Where list size is the number of elements in the array.

Example:

Given data

Keys are: 137456 214562 140145

$137456 \% 19 + 1 = 11$

$214562 \% 19 + 1 = 15$

$140145 \% 19 + 1 = 2$

3.Digit-Extraction method:

Using digit extraction selected digits are extracted from the key and used as the address.Example:

Using six-digit employee number to hash to a three-digit address (000-999), we could select the first, third, and fourth digits (from the left) and use them as the address.

The keys are: 379452

-> 394

121267 -> 112

378845 -> 388

4. Mid-Square method:

In mid-square hashing the key is squared and the address is selected from the middle of the square number.

Limitation is the size of the key.

Example:

94522 = 89340304: address is 3403

5. Folding method:

Two folding methods are used they are:

1. Fold Shift:

In fold shift the key value is divided into parts whose size matches the size of the required address. Then the left and right parts are shifted and added with the middle part.

2. Fold boundary:

In fold boundary the left and right numbers are folded on a fixed boundary between them. The two outside values are thus reversed.

Example:

calculate the hash value using folding method for key 5678. Divided into parts = 56 and 78

Sum = 134

Hash value = 34 (ignore the last carry)

Q.53 Define the types of Collision Resolution Techniques and the methods used in each of the type?

What is Collision?

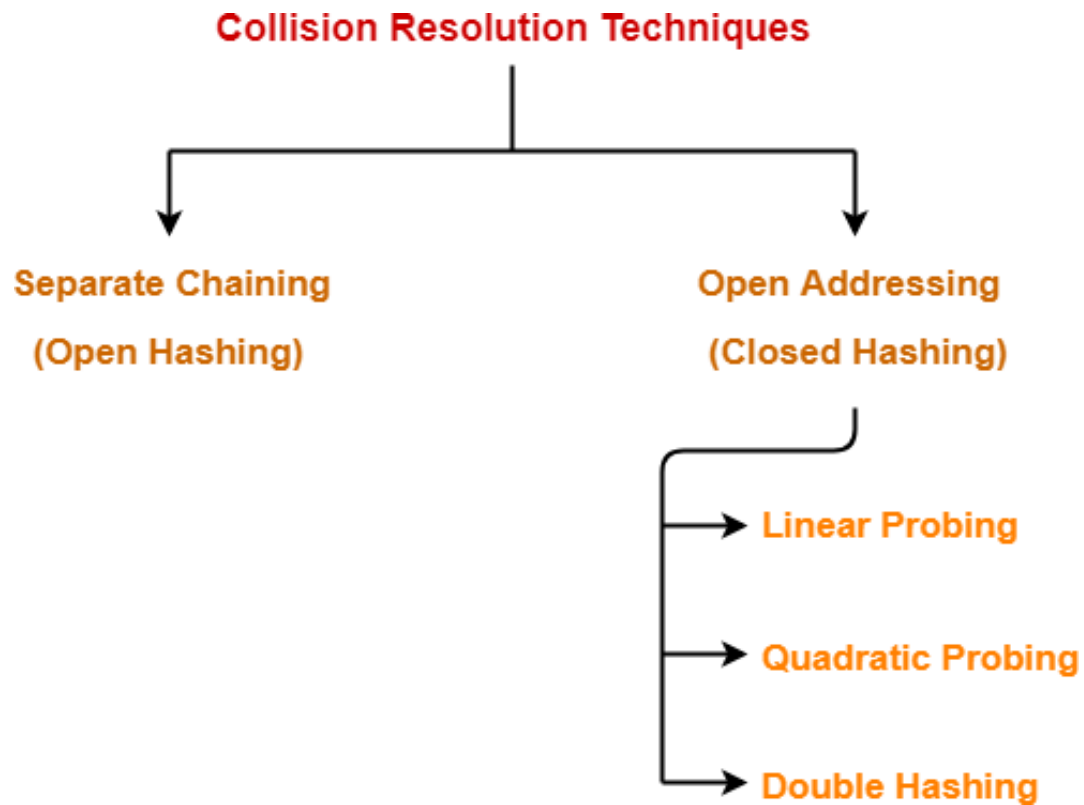
Since a hash function gets us a small number for a key which is a big integer or string, there is a possibility that two keys result in the same value. The situation where a newly inserted key maps to an already occupied slot in the hash table is called collision and must be handled using some collision handling technique.

How to handle Collisions?
There are mainly two methods to handle collision:

- 1)
- 2) Open Addressing

Separate

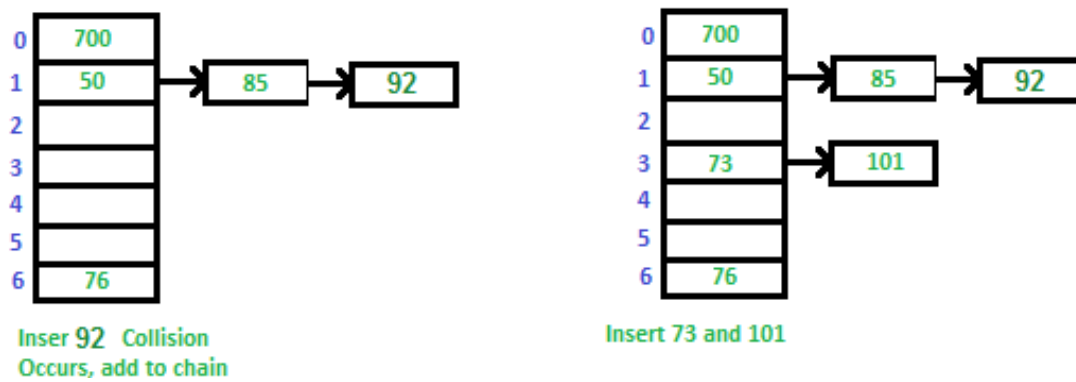
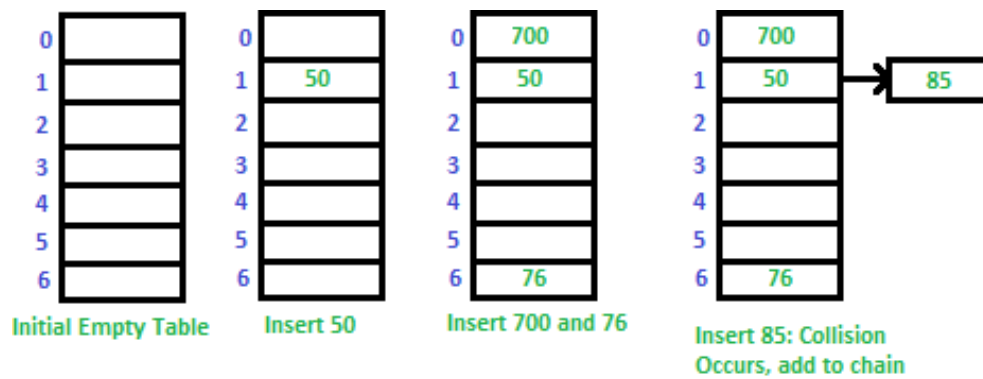
Chaining



A) Separate Chaining:

The idea is to make each cell of hash table point to a linked list of records that have same hash function value.

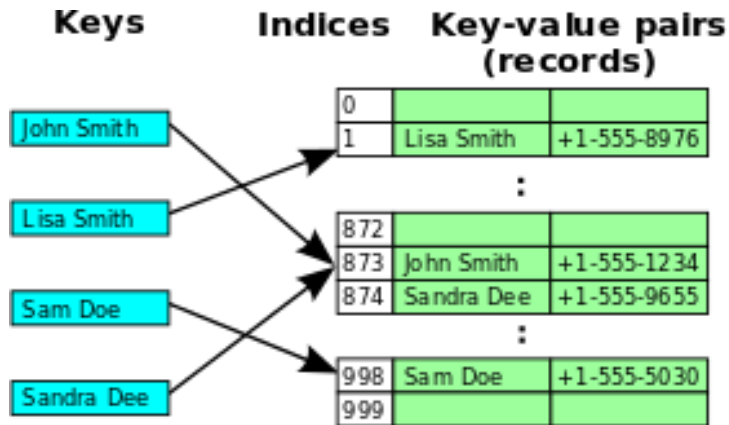
Let us consider a simple hash function as “**key mod 7**” and sequence of keys as 50, 700, 76, 85, 92, 73, and 101.



B) Open Addressing:

1) Linear Probing:

Linear probing is a scheme in computer programming for resolving collisions in hash tables, data structures for maintaining a collection of key–value pairs and looking up the value associated with a given key. So, linear probing basically does a linear search for an empty slot when there is a collision.



The collision between John Smith and Sandra Dee (both hashing to cell 873) is resolved by placing Sandra Dee at the next free location, cell 874.

Advantages:

- ☐ It is easy to implement.
- ☐ It has the best cache performance but suffers from clustering.
- ☐ It is easy to compute.

Disadvantages:

- ☐ The main problem with linear probing is clustering.
- ☐ Many consecutive elements form groups.
- ☐ Then, it takes time to search an element or to find an empty bucket.

In linear probing, we linearly probe for next slot. For example, the typical gap between two probes is 1 as seen in the example below.

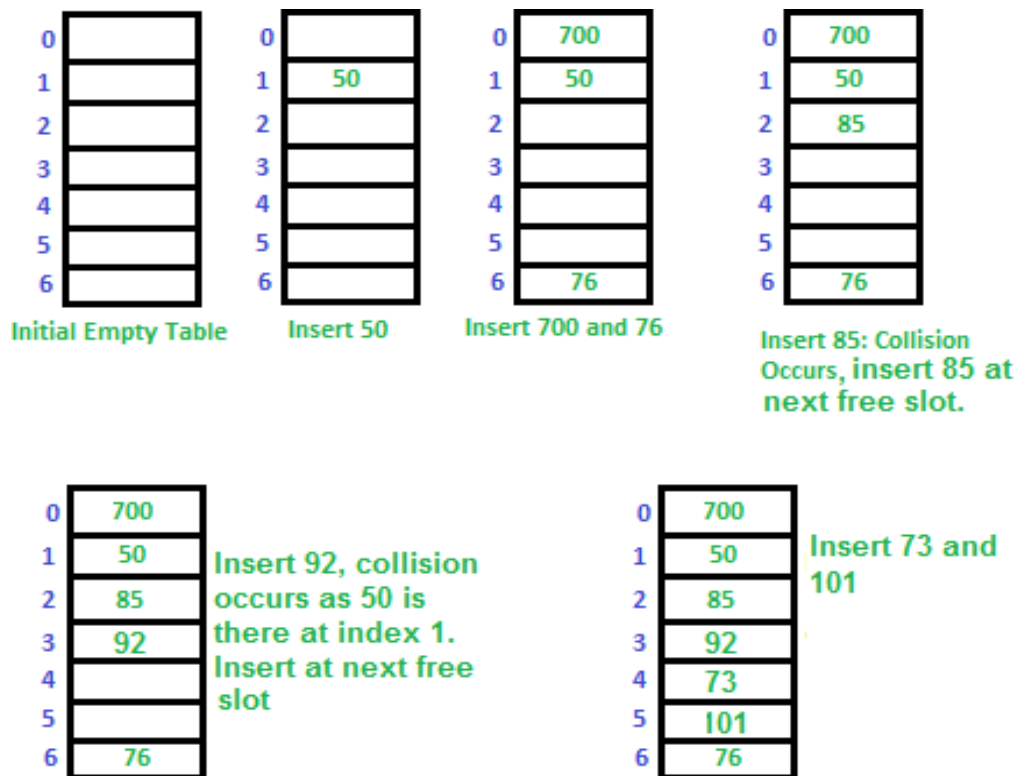
Let **hash(x)** be the slot index computed using a hash function and **S** be the table size. If slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1) \% S$

If $(\text{hash}(x) + 1) \% S$ is also full, then we try $(\text{hash}(x) + 2) \% S$

If $(\text{hash}(x) + 2) \% S$ is also full, then we try $(\text{hash}(x) + 3) \% S$

.....

Let us consider a simple hash function as “key mod 7” and a sequence of keys as 50, 700, 76, 85, 92, 73, and 101.



2) Doublehashing:

Double hashing is a computer programming technique used in conjunction with open- addressing in hash tables to resolve hash collisions, by using a secondary hash of the key as an offset when a collision occurs.

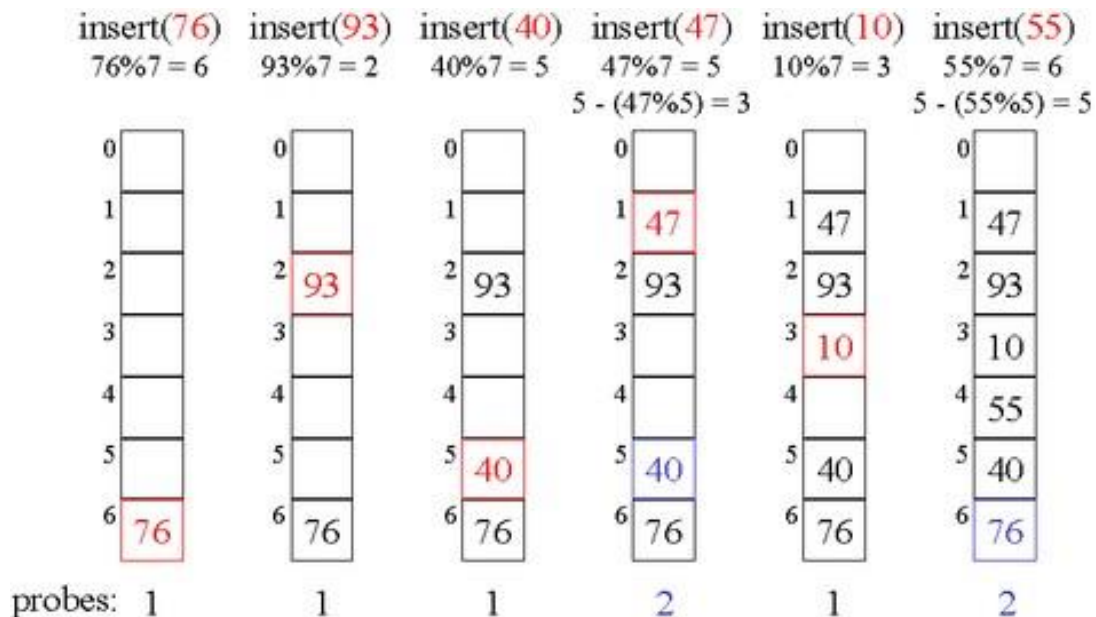
Double hashing is a collision resolving technique in Open Addressed Hash tables. Double hashing uses the idea of applying a second hash function to key when a collision occurs. Double hashing has poor cache performance but no clustering.

Double hashing uses the idea of applying a second hash function to the key when a collision occurs. The result of the second hash function will be the number of positions from the point of collision to insert.

There are a couple of requirements for the second function:

- ☐ it must never evaluate to 0
- ☐ must make sure that all cells can be probed

A popular second hash function is: $\text{Hash}_2(\text{key}) = R - (\text{key} \% R)$ where R is a prime number that is smaller than the size of the table.



Real life Example of Hashing:

1) Password Verification:

When you use any online website which requires a user login, you enter your E-mail and password to authenticate that the account you are trying to use belongs to you. When the password is entered, a hash of the password is computed which is then sent to the server for verification of the password. The passwords stored on the server are actually computed hash values of the original passwords. This is done to ensure that when the password is sent from client to server, no sniffing is there.

- 2) A filing cabinet is a real-life example of hashing. By applying the hash function to a piece of data, we obtain a hash key which specifies the correct pigeonhole. If the pigeonholes are elements of an array, the hash key will need to be an integer within that array's bounds.

3) Quadratic probing:

Quadratic probing is an open-addressing scheme where we look for i^2 th slot in i th iteration if the given hash value x collides in the hash table.

How Quadratic Probing is done?

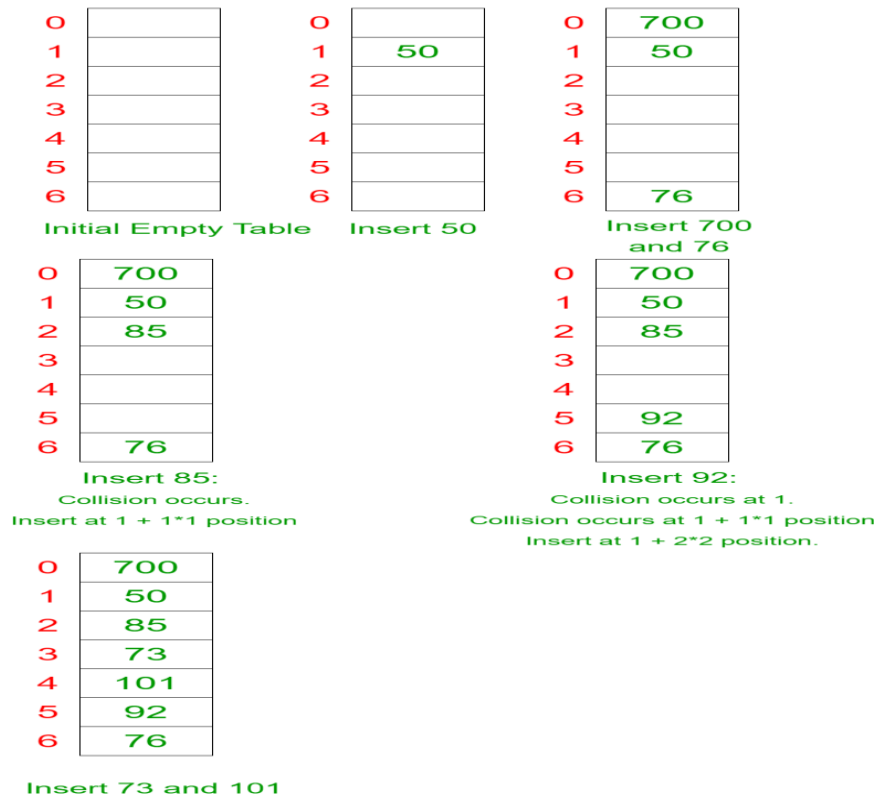
Let $\text{hash}(x)$ be the slot index computed using the hash function.

- If the slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1^2) \% S$.
- If $(\text{hash}(x) + 1^2) \% S$ is also full, then we try $(\text{hash}(x) + 2^2) \% S$.
- If $(\text{hash}(x) + 2^2) \% S$ is also full, then we try $(\text{hash}(x) + 3^2) \% S$.

- This process is repeated for all the values of i until an empty slot is found.

For example: Let us consider a simple hash function as “**key mod 7**” and sequence of keys as 50, 700, 76, 85, 92, 73, 101.

Quadratic Probing Example



Q.54. In RDBMS, what is the efficient data structure used in the internal storage representation?

B+ tree. Because in B+ tree, all the data is stored only in leaf nodes, that makes searching easier.

This corresponds to the records that shall be stored in leaf nodes.

Theory:

In order, to implement dynamic multilevel indexing, B-tree and B+ tree are generally employed. The drawback of B-tree used for indexing, however is that it stores the data pointer (a pointer to the disk file block containing the key value), corresponding to a particular key value, along with that key value in the node of a B-tree. This technique, greatly reduces the number of entries that can be packed into a node of a B-tree, thereby contributing to the increase in the number of levels in the B-tree, hence increasing the search time of a record.

B+ tree eliminates the above drawback by storing data pointers only at the leaf nodes of the tree. Thus, the structure of leaf nodes of a B+ tree is quite different from

the structure of internal nodes of the B tree. It may be noted here that, since data pointers are present only at the leaf nodes, the leaf nodes must necessarily store all the key values along with their corresponding data pointers to the disk file block, in order to access them. Moreover, the leaf nodes are linked to provide ordered access to the records. The leaf nodes, therefore form the first level of index, with the internal nodes forming the other levels of a multilevel index.

Some of the key values of the leaf nodes also appear in the internal nodes, to simply act as a medium to control the searching of a record.

B+ Tree

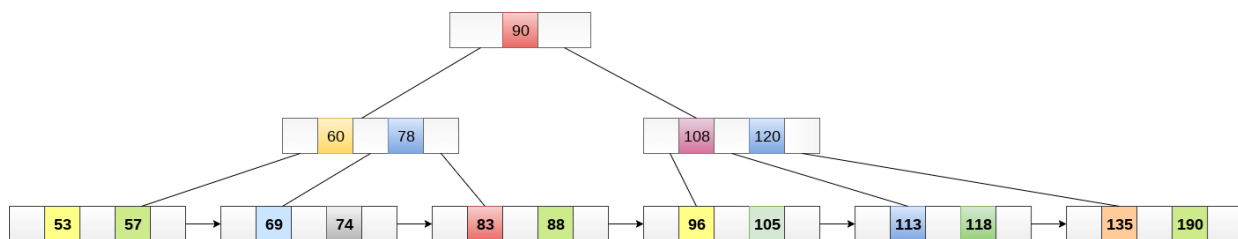
B+ Tree is an extension of B Tree which allows efficient insertion, deletion and search operations.

In B Tree, Keys and records both can be stored in the internal as well as leaf nodes. Whereas, in B+ tree, records (data) can only be stored on the leaf nodes while internal nodes can only store the key values.

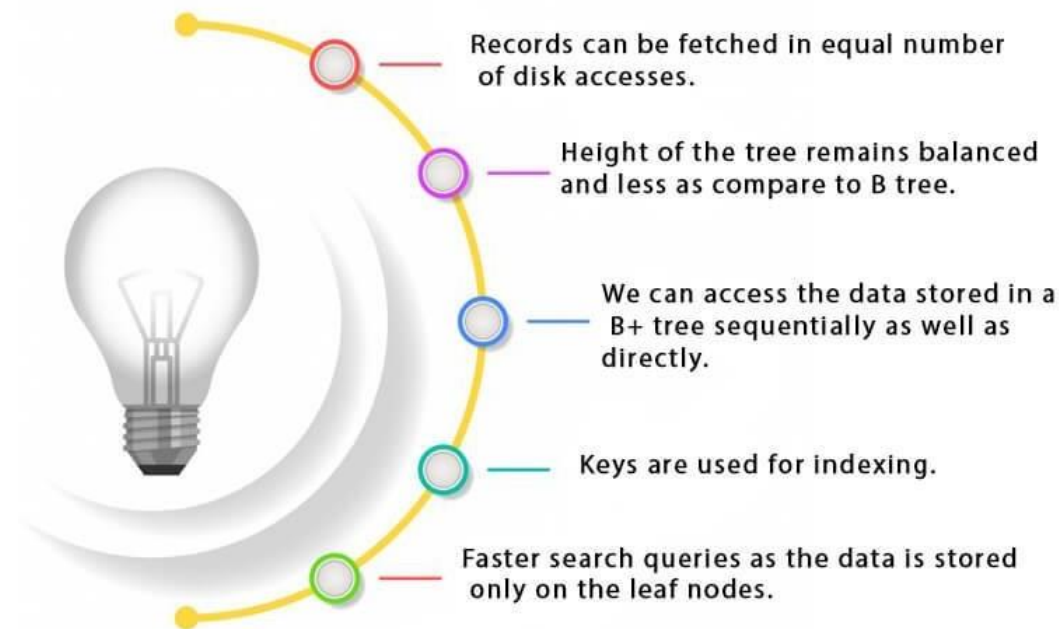
The leaf nodes of a B+ tree are linked together in the form of a singly linked lists to make the search queries more efficient.

B+ Tree are used to store the large amount of data which cannot be stored in the main memory. Due to the fact that, size of main memory is always limited, the internal nodes (keys to access records) of the B+ tree are stored in the main memory whereas, leaf nodes are stored in the secondary memory.

The internal nodes of B+ tree are often called index nodes. A B+ tree of order 3 is shown in the following figure.



Advantages of B+ Tree



Applications of tree data structures:

1. Folders in Operating system.
2. HTML Document Object Model (DOM).
3. Network Routing.
4. Auto corrector and spell checker.
5. Next Move in games.

55. List different types of data structures.

Ans: Data Structure can be defined as the group of data elements which provides an efficient way of storing and organising data in the computer so that it can be used efficiently. Some examples of Data Structures are arrays, Linked List, Stack, Queue, etc. Data Structures are widely used in almost every aspect of Computer Science i.e. Operating System, Compiler Design, Artificial intelligence, Graphics and many more.

Different data structures (DS):

Array:

An array stores a collection of items at adjoining memory locations. Items that are the same type are stored together so the position of each element can be calculated or retrieved easily by an index. Arrays can be fixed or flexible in length.

Linked List

A linked list stores a collection of items in a linear order. Each element, or node, in a linked list contains a data item, as well as a reference, or link, to the next item in the list.

Stack

A stack stores a collection of items in the linear order that operations are applied. This order could be last in, first out (LIFO) or first in, first out (FIFO).

Queue

A queue stores a collection of items like a stack; however, the operation order can only be first in, first out.

Tree

A tree stores a collection of items in an abstract, hierarchical way. Each node is associated with a key value, with parent nodes linked to child nodes -- or subnodes. There is one root node that is the ancestor of all the nodes in the tree.

Binary Search Tree

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties –

- The value of the key of the left sub-tree is less than the value of its parent (root) node's key.
- The value of the key of the right sub-tree is greater than or equal to the value of its parent (root) node's key.

Heap

A heap is a tree-based structure in which each parent node's associated key value is greater than or equal to the key values of any of its children's key values.

Hashing

Hashing is a technique or process of mapping keys, values into the hash table by using a hash function. It is done for faster access to elements. The efficiency of mapping depends on the efficiency of the hash function used.

Graph

A graph stores a collection of items in a nonlinear fashion. Graphs are made up of a finite set of nodes, also known as vertices, and lines that connect them, also known as edges. These are useful for representing real-world systems such as computer networks.

56. Find nth last node in linked list in one iteration. Ans

Find Nth node from the end of Linked List

----Problem Description:---

We have two approaches to identify Nth node from the end. In

First Approach:

First iterate through linked list and identify length.

Then again iterate (length-N) nodes from head node.

The Node after the above iteration is the Nth node from the end. In

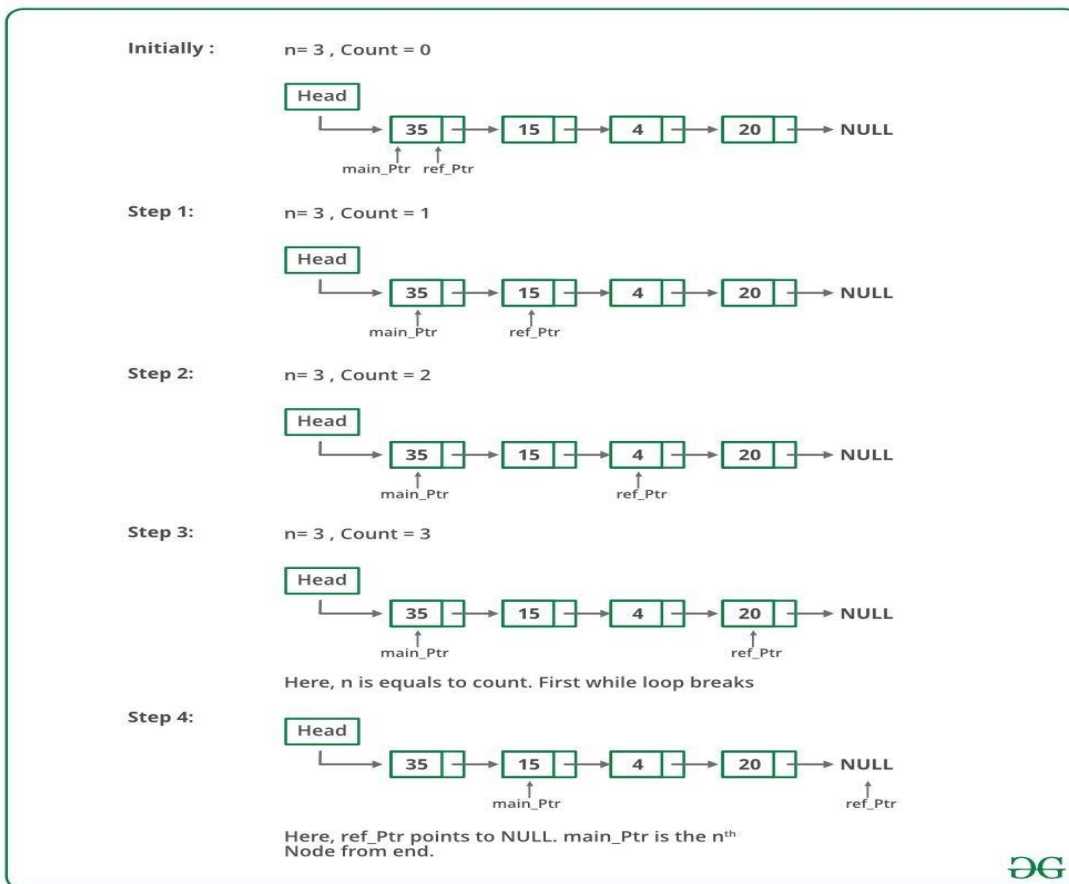
Second Approach:

We will define two temp Node references and they will be initially pointed to head node. The two temp nodes are called as reference node & main node.

First iterate reference node to N nodes.

Then iterate both temp nodes one step at a time, till reference node is null.

Once after the above iteration, the main node represents the Nth node from the end n



-----PROGRAM-----

```
public class L1NthNodeFromLast {

    static class Node {
```

```

    int data;
    Node next;
    Node(int tmp) {data = tmp;}
}

static Node findNthNodeFromLastV1(Node head, int n) {

    /**
     * In this approach, first identify the length of the list.
     * Then do length-n iterations from head to reach nth node from last.
     */
    int length = 0; Node
    temp = head;
    while(temp != null) {
        temp = temp.next;
        length++;
    }
    System.out.println("length: "+length);
    for(int i=0;i<length-n;i++) {
        head = head.next;
    }
    return head;
}

static Node findNthNodeFromLastV2(Node head, int n) {

    /**
     * In this approach, we will use two pointers like refNode & mainNode
     * First we will traverse refNode by n times.
     * Then we will traverse both nodes one step at a time till refNode reaches null
     * The mainNode is the one we are looking for.
     */
    Node refNode = head;
    Node mainNode = head;
    if(head == null) {
        return null;
    }
    for(int i=0;i<n;i++) {
        if(refNode == null) {
            System.out.println("The n input size is more than linked list size");
            return null;
        }
        refNode = refNode.next;
    }

    while(refNode != null) {

```

```

        mainNode = mainNode.next;
        refNode = refNode.next;
    }

    return mainNode;
}

public static void main(String[] a) {

    Node n1 = new Node(23);
    Node n2 = new Node(29);
    Node n3 = new Node(69);
    Node n4 = new Node(85);
    Node n5 = new Node(40);
    Node n6 = new Node(45);
    Node n7 = new Node(54);
    n1.next = n2;
    n2.next = n3;
    n3.next = n4;
    n4.next = n5;
    n5.next = n6;
    n6.next = n7;

    Node nthNode = findNthNodeFromLastV2(n1, 3);
    System.out.println("Nth node value: "+nthNode.data);
}
}

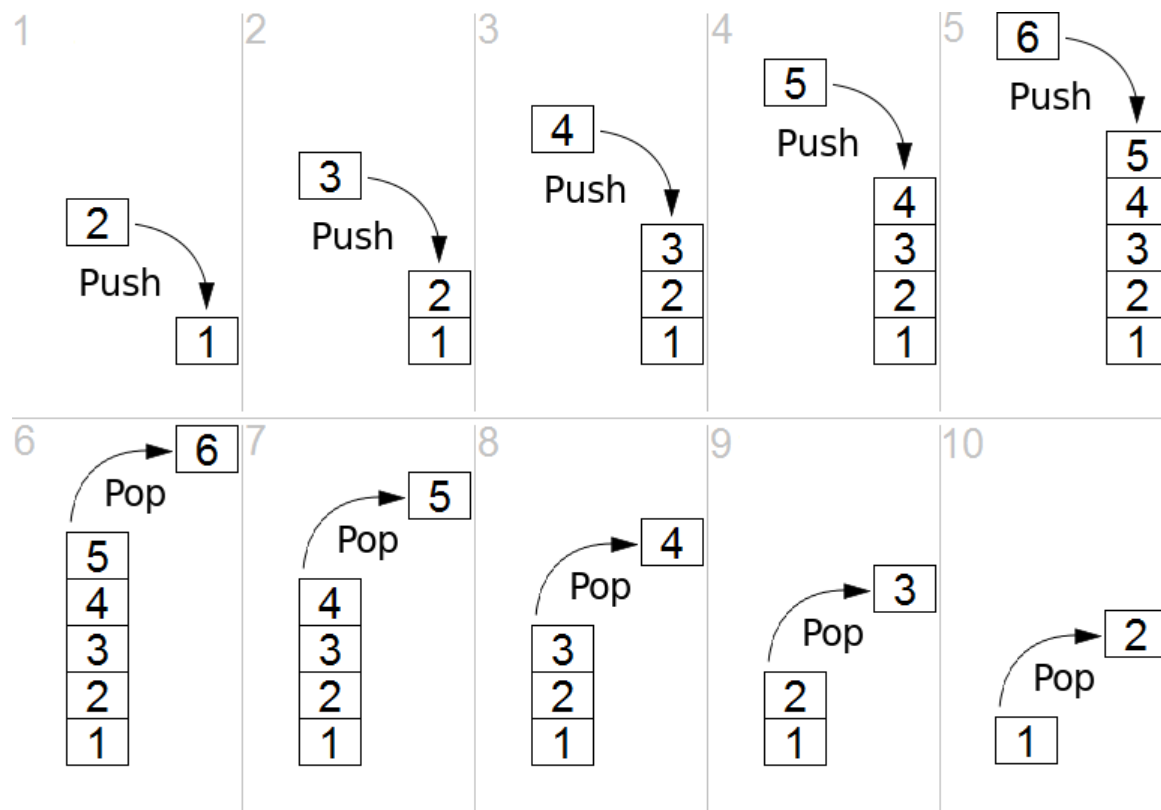
```

57. Explain The Difference Among A Stack And An Array?

STACK-

Def-- A Stack is a linear data structure in which elements can be inserted and deleted only from one side of the list, called the **top**. A stack follows the **LIFO** (Last In First Out) principle, i.e., the element inserted at the last is the first element to come out. The insertion of an element into the stack is called **push** operation, and the deletion of an element from the stack is called **pop** operation. In stack, we always keep track of the last element present in the list with a pointer called **top**.

Diagram-



Expl-

In the above dia. We can see that in first 1-5 the values are pushed in from 1 to 5 so the last value is 5 and in the 6-10 the values are popping means the values are removing from the stack. So the process is going like the first we remove the value 5 and after that we remove 4, and 3, 2, 1 as we removing (popping) the values from stack.

1. Stacks are based on the LIFO principle, i.e., the element inserted at the last, is the first element to come out of the list.
2. Stack has a dynamic and fixed size.
3. The stack can contain elements of the different data types.
4. Stack has only one type.

Code-

```
import java.io.*;
```

```
import java.util.*;

class Test

{

    // Pushing element on the top of the stack

    static void stack_push(Stack<Integer> stack)

    {

        for(int i = 0; i < 5; i++)

        {

            stack.push(i);

        }

    }


    // Popping element from the top of the stack

    static void stack_pop(Stack<Integer> stack)

    {

        System.out.println("Pop Operation:");


        for(int i = 0; i < 5; i++)

        {
```

```
        Integer y = (Integer) stack.pop();

        System.out.println(y);

    }

}

// Displaying element on the top of the stack

static void stack_peek(Stack<Integer> stack)

{

    Integer element = (Integer) stack.peek();

    System.out.println("Element on stack top: "+ element);

}

// Searching element in the stack

static void stack_search(Stack<Integer> stack, intelement)

{

    Integer pos = (Integer) stack.search(element);

    if(pos == -1)

        System.out.println("Element not found");

}
```

```
        else

            System.out.println("Element is found at position: "+ pos);

    }

    public static void main (String[] args)

    {

        Stack<Integer> stack = new Stack<Integer>();

        stack_push(stack);

        stack_pop(stack);

        stack_push(stack);

        stack_peek(stack);

        stack_search(stack, 2);

        stack_search(stack, 6);

    }

}
```

For example-

The undo operation in a text processing / word processing software like Notepad, Microsoft Word is an example where the Stack data structure is used.

ARRAY-

An Array is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together. This makes it easier to calculate the position of each element by simply adding an offset to a base value, i.e., the memory location of the first element of the array (generally denoted by the name of the array).

DIA-

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8

EXPL-

1. In the array the elements belong to indexes, i.e., if you want to get into the fourth element you have to write the variable name with its index or location within the square bracket eg-arr[4].
2. Insertion and deletion in array can be done at any index in the array.
3. Array has a fixed size
4. Array contains elements of same data type.
5. Different types of Arrays are 1D, 2D, etc

Code-

//Java Program to illustrate how to declare, instantiate, initialize

//and traverse the Java array.

```

class Testarray{
public static void main(String args[]){
int a[]=new int[5];//declaration and instantiation
a[0]=10;//initialization
a[1]=20;
a[2]=70;
a[3]=40;
a[4]=50;

```

```
//traversing array
for(int i=0;i<a.length;i++)//length is the property of array
System.out.println(a[i]);
}
}
```

For example-

Collecting database records into ArrayList JDBC is used to connect to the database and perform the operations on the tables. Every application needs to save user data and activities into the database.

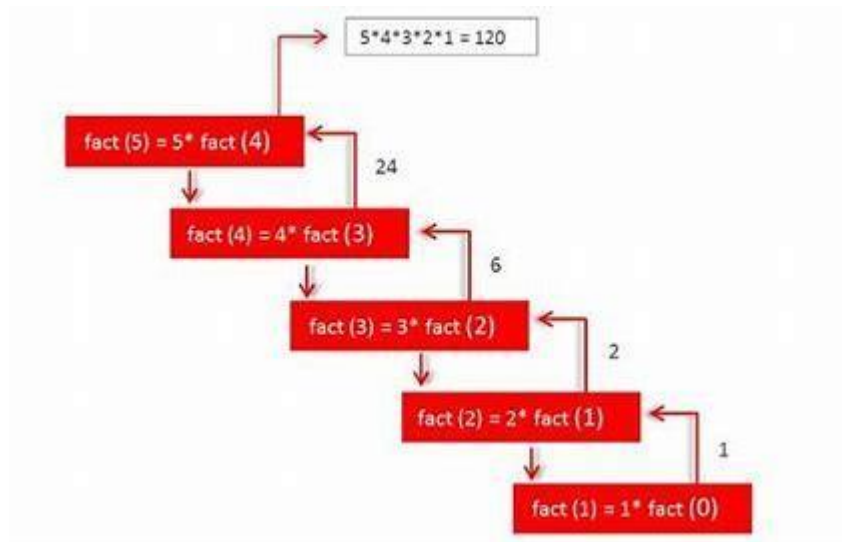
.In School management application, You have a list of students stored in Database.

58 .What Do You Mean By Recursive Definition?

Def-

A recursive is a type of function or expression **stating some concept or property of one or more variables**, which is specified by a procedure that yields values or instances of that function by repeatedly applying a given relation or routine operation to known values of the function.

Dia-



Explanation-

The main purpose for using the recursive approach is that once you understand it, it can be clearer to read. There is actually no performance benefit to using recursion. The iterative approach with loops can sometimes be faster. But mainly the simplicity of recursion is sometimes preferred.

Code-

```
public class RecursionExample2 {
    static int count=0;
    static void p(){
        count++;
        if(count<=5){
            System.out.println("hello "+count);
            p();
        }
    }
    public static void main(String[] args) {
        p();
    }
}
```

Real time example-

For example we have a folder structure and in this one folder sample it has 2 files and one folder, then the outer folder has 2 files and one folder, InnerFolder1, has again 2 files and one folder, InnerFolder2, is the innermost folder with only two files, no other folders. For instance, looping a video in VLC player can be considered an infinite loop which is simply an iteration that does not terminate.

59. Define The Benefits Of Linked List Over Array (static Data Structure)?

Ans: Both Arrays and Linked List can be used to store linear data of similar types but Array is a static data structure while Linked list is a dynamic data structure so it can grow and shrink at runtime by allocating and deallocating memory. So there is no need to give initial size of linked list.

Insertion and Deletion

Insertion and deletion of nodes are really easier. Unlike array here we don't have to shift elements after insertion or deletion of an element. In linked list we just have to update the address present in next pointer of a node.

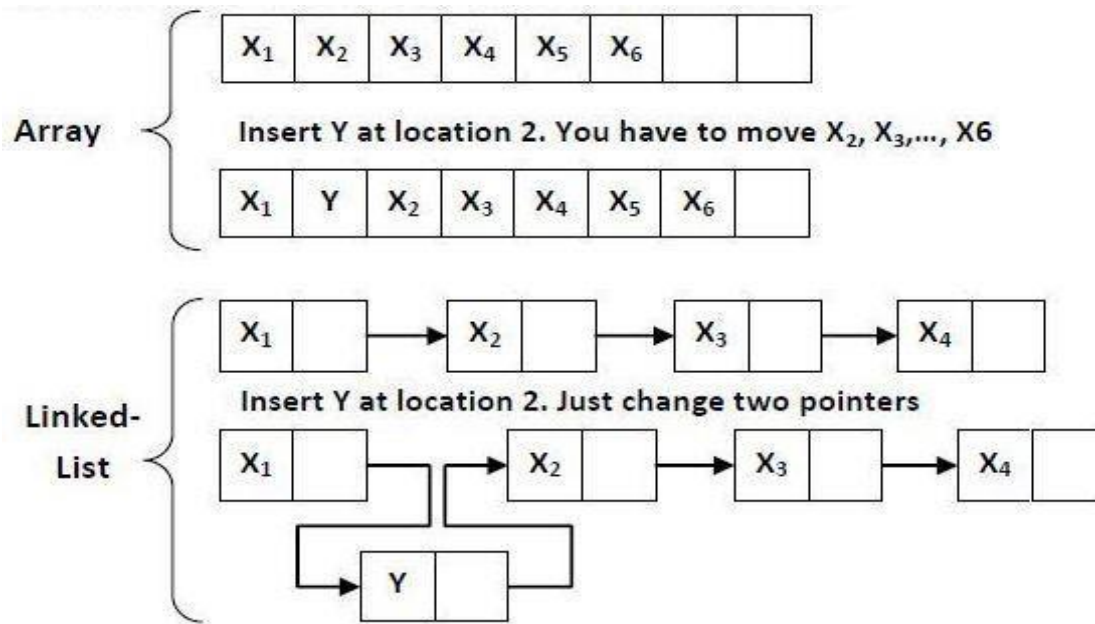
Inserting a new element in an array of elements is expensive, because room has to be created for the new elements and to create room existing elements have to be shifted.

For example, suppose we maintain a sorted list of IDs in an array `id[id[]]` =

[1000, 1010, 1050, 2000, 2040,].

And if we want to insert a new ID 1005, then to maintain the sorted order, we have to move all the elements after 1000 (excluding 1000).

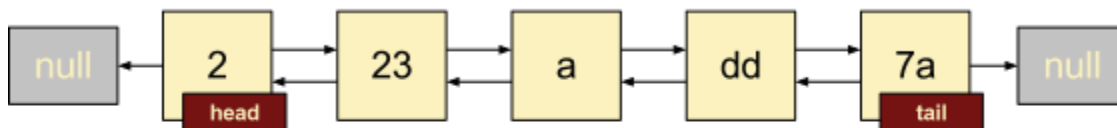
Deletion is also expensive with arrays unless some special techniques are used. For example, to delete 1010 in `id[]`, everything after 1010 has to be moved.



No Memory Wastage

The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage, and in practical uses, upper limit is rarely reached, but in case of linked list size can increase or decrease at run time so there is no memory wastage. In case of array there is lot of memory wastage, like if we declare an array of size 10 and store only 6 elements in it then space of 4 elements are wasted. There is no such problem in linked list as memory is allocated only when required.

Linked List

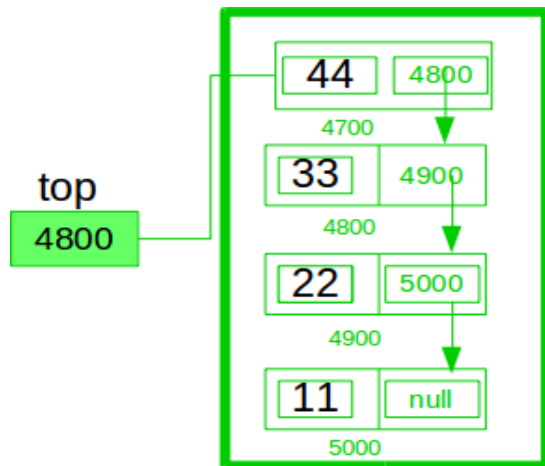


Array

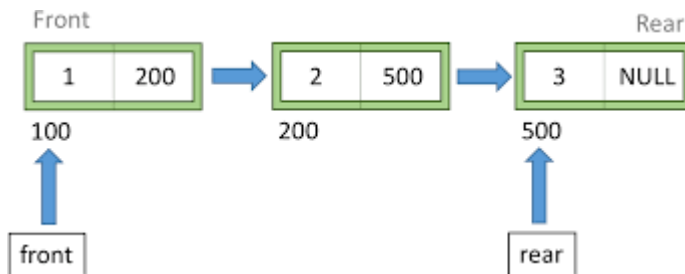


Implementation

Data structures such as stack and queues can be easily implemented using linked list.



Implementation of Stack using Linked List



Implementation of Queue using Linked List

60. We Apply Binary Search Algorithm To A Sorted Linked List, Why?

Ans: Binary search is possible on the linked list if the list is ordered and you know the count of elements in list. But While sorting the list, you can access a single element at a time through a pointer to that node i.e. either a previous node or next node. This increases the traversal steps per element in linked list just to find the middle element. This makes it slow and inefficient. Whereas the binary search on an array is fast and efficient because of its ability to access any element in the array in constant time. You can get to the middle of the array just by saying "array[middle]". Now, can you do the same with a linked list? The answer is No. You will have to write your own, possibly inefficient algorithm to get the value of the middle node of a linked list. In a linked list, you lose the ability to get the value of any node in a constant time.

Binary search is possible on linked list data structure but there is no point using it, because at the end it would take same time as normal linear search.

Binary search reduces the number of elements to be searched by filtering out almost half the number of elements at each iteration. For the best implementation of Binary search we need two prerequisites:

1. The list must be in sorted order.
2. Any random element can be accessed in constant time.

With linked list the second prerequisite is not satisfied as any random element in linked list cannot be accessed in constant time, but it must be traversed completely.

It must be traversed and there is no other way because each element in linked list is not stored contiguously but in random locations and it could be accessed only by traversing the previous element. (The previous element stores the address of the next element)

In an array basically any random element is accessed as,

Address of any random element = Base address of the array + index of the element * size of the element

This also answers the question why to have elements of array of the same datatype. The above calculation makes it possible to access any element in an array in constant time. So binary search is preferred in array list, but not in linked list.

Qu.61 Define the DisBenifits of Linked Array implementations of Linked List ?

Sol:-

1. **Memory Usage:** More memory is required to store elements in linked list as compared to array. Because in linked list each node contains a pointer and it requires extra memory for itself.
2. **Traversal:** Elements or nodes traversal is difficult in linked list. We can not randomly access any element as we do in array by index. For example if we want to access a node at position n then we have to traverse all the nodes before it. So, time required to access a node is large.

3. **Reverse Traversing:** In linked list reverse traversing is really difficult. In case of doubly linked list its easier but extra memory is required for back pointer hence wastage of memory.

4. You'll have to keep track of the free entries - after a few insertions and deletions, these free entries could be anywhere.

5. Using an array will impose an upper limit on the size of the linked list.

Qu.62 Define the DisBenifits of representing a stack or Queue by a Linked List ?

Sol:-

1. A node in a linked list (info and next field) occupies more storage than a corresponding element in an array.

2. Additional time spent in managing the available list.

3. In linked list we need to maintain pointers between the nodes, which uses more memory than an equivalent array would require. And since insertions always occur at the tail and extractions at the head, you must also maintain pointers to both nodes to permit constant time access to those nodes. However, the additional memory is more than compensated for by the fact the list can grow and shrink dynamically, without the need to copy and reallocate any nodes.

Q 63. Is it necessary to sort a file before searching a particular item?

Sol:-

if less work is involved in searching a element than to sort and then extract then we don't go for sort

if frequent use of the file is required for the purpose of retrieving specific element it is more efficient to sort the file thus it depends on situation.

Q 64. Parenthesis are never needed in prefix or postfix expressions. Why?

Sol:-

Evaluation of postfix or prefix notations are done with the help of stacks. So there is no need for brackets.

In other words, infix notation is human-readable format, which needs brackets based on BODMAS rule to understand the order of execution. The postfix or prefix are

machine-readable format. The conversion and evaluation of postfix and prefix are applications of stack model.

Let's assume the below

- ☐ Operands are real numbers in single digits.
- ☐ Permitted operators: +, -, *, /, ^ (exponentiation)
- ☐ Blanks are NOT permitted in expression.
- ☐ Parenthesis are permitted

Example:

Postfix: 54+

Output: 9

Explanation: Infix expression of above postfix is: $5 + 4$ which resolves to 9

Postfix: 2536+**5/2-

Output: 16

Explanation: Infix expression of above postfix is: $2 * (5 * (3 + 6)) / 5 - 2$ which resolves to 16

Let's consider the following table

Postfix Expression : 2536+**5/2-		
Token	Action	Stack
2	Push 2 to stack	[2]
5	Push 5 to stack	[2, 5]
3	Push 3 to stack	[2, 5, 3]
6	Push 6 to stack	[2, 5, 3, 6]
+	Pop 6 from stack	[2, 5, 3]
	Pop 3 from stack	[2, 5]
	Push 3+6 =9 to stack	[2, 5, 9]
*	Pop 9 from stack	[2, 5]
	Pop 5 from stack	[2]
	Push 5*9=45 to stack	[2, 45]
*	Pop 45 from stack	[2]
	Pop 2 from stack	[]
	Push 2*45=90 to stack	[90]
5	Push 5 to stack	[90, 5]
/	Pop 5 from stack	[90]
	Pop 90 from stack	[]
	Push 90/5=18 to stack	[18]
2	Push 2 to stack	[18, 2]
-	Pop 2 from stack	[18]
	Pop 18 from stack	[]
	Push 18-2=16 to stack	[16]
Result : 16		

65. List Out The Areas In Which Data Structures Are Applied Extensively?

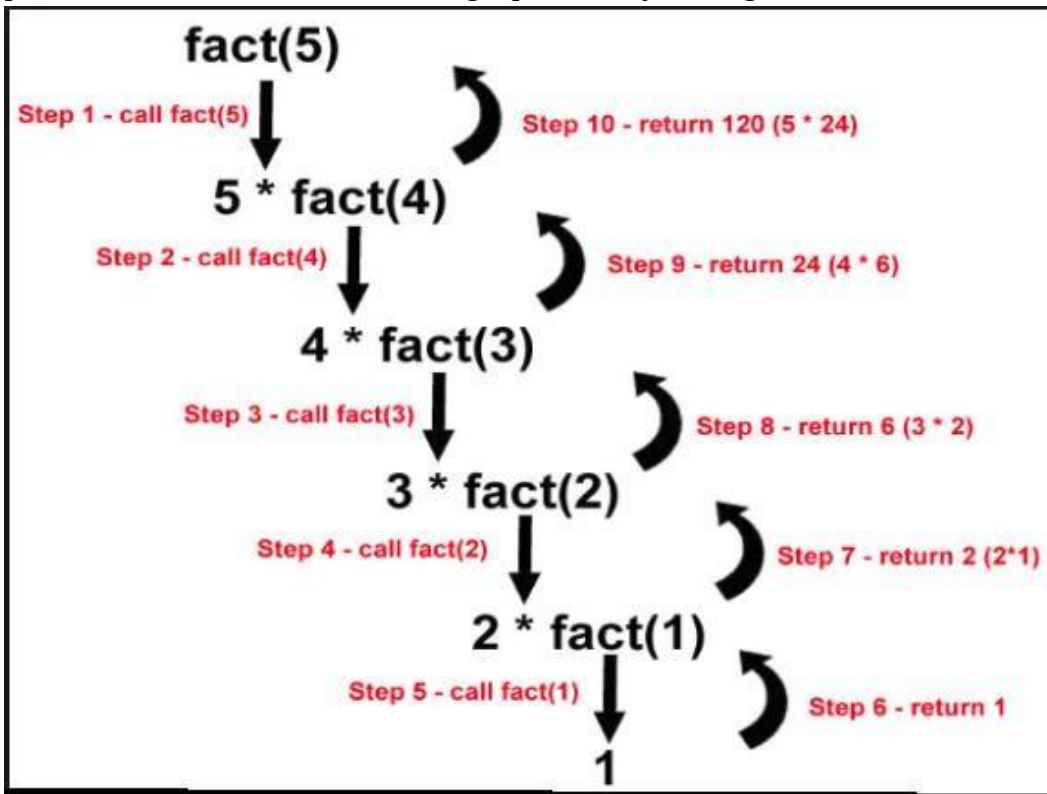
Sol:-

1. Compiler Design,
2. Operating System,
3. Database Management System,
4. Statistical analysis package,
5. Numerical Analysis,
6. Graphics,
7. Artificial Intelligence,
8. Simulation

66. Explain The Data Structures Used To Perform Recursion?

Sol:-

Recursion is a function or method has the ability of calling itself to solve the problem. The process of recursion involves solving a problem by turning it into smaller varieties of itself.



Explanation:-

The concept of recursion is established on the idea that a problem can be solved much easily and in lesser time if it is represented in one or smaller versions. Adding base conditions to stop recursion is another important part of using this algorithm to solve a problem.

A recursive subroutine is defined as one that directly or indirectly calls itself. Calling a subroutine directly signifies that the definition of the subroutine already has the call statement of calling the subroutine that has been defined.

On the other hand, the indirect calling of a subroutine happens when a subroutine calls another subroutine, which then calls the original subroutine. Recursion can use a few lines of code to describe a very complex task.

Program:-

```

static int fact(int n)
{
    if(n<=1)
        return 1;
    else
        return n*fact(n-1);
}
  
```

```
}  
public static void main(String [] arg)  
{  
    System.out.println(fact(5));  
}  
}
```

Output:- 120

Realtime example:-

The **Tower Of Hanoi** is an ancient puzzle said to have originated in India or Vietnam. It involves moving various sized rings or disks around on three poles. The goal in this puzzle is to move all of the rings on one pole to another while keeping the order of the rings intact. You must follow the rules of the puzzle however, and this is that only one ring can be moved at a time, and no ring may be placed on top of a smaller sized ring. This puzzle can be solved using recursion.

67. List Out Few Of The Application Of Tree Data-structure?

Sol:-

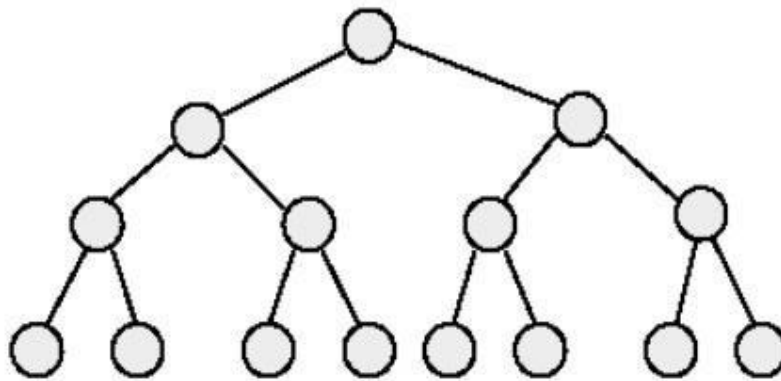
A tree is a hierarchical data structure defined as a collection of nodes. Nodes represent value and nodes are connected by edges.

Tree Terminology

1. The tree has one node called root. ...
2. Each node has one parent only but can have multiple children.
3. Each node is connected to its children via edge.

Trees

Definition:- A tree is a non-linear data structure in which items are arranged in a sorted sequence. it is a very flexible and powerful data structure that can be used for a variety of applications.



APPLICATION :-

- 1) Manipulate hierarchical data
- 2) Make information easy to search (see tree traversal).
- 3) Manipulate sorted lists of data
- 4) Router algorithms
- 5) Form of a multi-stage decision-making, like Chess Game.
- 6) As a workflow for compositing digital images for visual effects.

Some applications of the trees are: XML Parser uses tree algorithms. Decision-based algorithm is used in machine learning which works upon the algorithm of tree.

Databases also uses tree data structures for indexing

68) There Are 8, 15, 13, 14 Nodes Were There In 4 Different Trees. Which Of Them ?

Sol:-15.

In general:

There are $2n-1$ nodes in a full binary tree. By

the method of elimination:

Full binary trees contain odd number of nodes. So there cannot be full binary trees with 8 or 14 nodes, so rejected. With 13 nodes you can form a complete binary tree but not a full binary tree. So the correct answer is 15.

Q.69 There are 8, 15, 13, 14 nodes were there in 4 different trees. Which of them could have formed a full binary tree?

Sol:-

In general:

There are $2n-1$ nodes in a full binary tree.

By the method of elimination:

Full binary trees contain odd number of nodes. So there cannot be full binary trees with 8 or 14 nodes, so rejected. With 13 nodes you can form a complete binary tree but not a full binary tree. So the correct answer is 15.

Q.70 Explain A Spanning Tree?

Sol:-

We will learn about the *spanning tree* and *minimum spanning tree*. Before knowing about the spanning trees, we should know about some graphs.

The following are the types of graphs:

- Undirected graph: An undirected graph is a graph in which all the edges do not point to any particular direction, i.e., they are not unidirectional; they are bidirectional. It can also be defined as a graph in which set of V vertices and set of E edges, each edge connecting two different vertices.
- Connected graph: A connected graph is a graph in which a path always exists from a vertex to any other vertex.
A graph is connected if we can reach any vertex from any other vertex by following edges in either direction.
- Directed graph: A directed graph is defined as a graph in which set of V vertices and set of E edges, each connecting two different vertices, but it is not mandatory that node points in the opposite direction also.

Spanning tree:

If we have a graph containing V vertices and E edges, then the graph can be represented as:

 $G(V, E)$

If we create the spanning tree from the above graph, then the spanning tree would have the same number of vertices as the graph, but the vertices are not equal. The

edges in the spanning tree would be equal to the number of edges in the graph minus 1.

Suppose the spanning tree is represented as:

$$G'(V', E')$$

where,

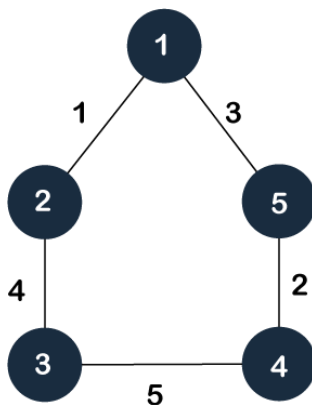
$$V = V'$$

$$E' \in E - 1$$

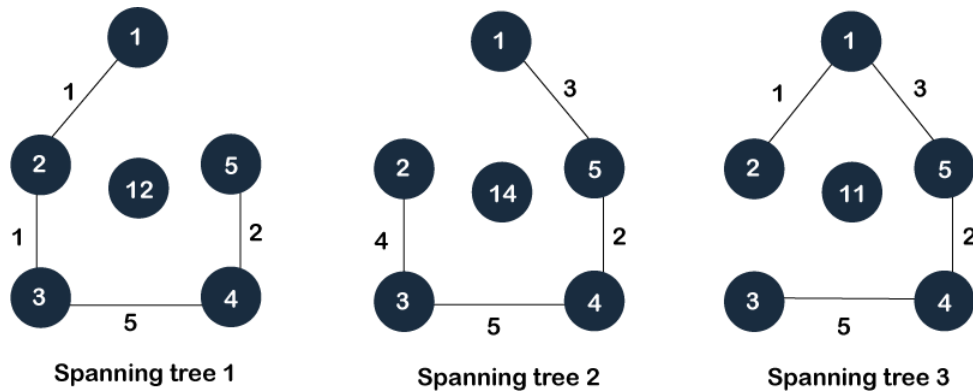
$$E' = |V| - 1$$

Let's understand through an example.

Suppose we want to create the spanning tree of the graph, which is shown below:

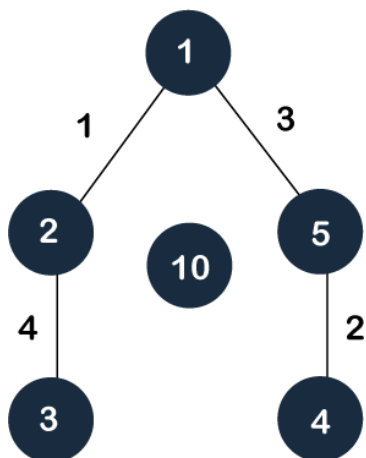


As we know, that spanning tree contains the same number of vertices as the graph, so the total number of vertices in the graph is 5; therefore, the spanning tree will also contain the 5 vertices. The edges in the spanning tree are equal to the number of vertices in the graph minus 1; therefore, the number of edges is 4. Three spanning trees can be created, which are shown below:



Minimum Spanning Trees:

The minimum spanning tree is the tree whose sum of the edge weights is minimum. From the above spanning trees, the total edge weight of the spanning tree 1 is 12, the total edge weight of the spanning tree 2 is 14, and the total edge weight of the spanning tree 3 is 11; therefore, the total edge weight of the spanning tree 3 is minimum. From the above graph, we can also create one more spanning tree as shown below:



In the above tree, the total edge weight is 10 which is less than the above spanning trees; therefore, the minimum spanning tree is a tree which is having an edge weight, i.e., 10.

Applications of Spanning trees

The following are the applications of the spanning trees:

- **Building a network:** Suppose there are many routers in the network connected to each other, so there might be a possibility that it forms a loop. So, to avoid the

formation of the loop, we use the tree data structure to connect the routers, and a minimum spanning tree is used to minimally connect them.

- Clustering: Here, clustering means that grouping the set of objects in such a way that similar objects belong to the same group than to the different group. Our goal is to divide the n objects into k groups such that the distance between the different groups gets maximized.

How can clustering be achieved?

First, we divide the n objects into k groups.

Secondly, we will combine these clusters iteratively by adding an edge between them.

Stop when we reach the k clusters.

Approach-

One of the approaches that can be used to obtain clustering is to compute a minimum spanning tree. The minimum spanning tree can be simply calculated by dropping the $k-1$ most heavily weighted edge from the graph.

Q.71 Difference among the malloc and calloc ?

Sol:-

Malloc() :- It is a function which is used to allocate a block of memory dynamically. It reserves memory space of specified size and returns the null pointer pointing to the memory location.

The pointer returned is usually of type void. It means that we can assign malloc function to any pointer. The full form of malloc is memory allocation.

Calloc() :- function is used to allocate multiple blocks of memory. It is a dynamic memory allocation function which is used to allocate the memory to complex data structures such as arrays and structures. If this function fails to allocate enough space as specified, it returns will null pointer. The full form of calloc function is contiguous allocation.

Differences of malloc and calloc :

malloc()	calloc()
Malloc() function will create a single block of memory of size specified by the user.	Calloc() function can assign multiple blocks of memory for a variable.
Malloc function contains garbage value.	The memory block allocated by a calloc function is always initialized to zero.
Number of arguments is 2.	Number of argument is 1.
Calloc is slower than malloc.	Malloc is faster than calloc.
It is not secure as compared to calloc.	It is secure to use compared to malloc.
Time efficiency is higher than calloc().	Time efficiency is lower than malloc().
Malloc() function returns only starting address and does not make it zero.	Before allocating the address, Calloc() function returns the starting address and makes it zero.
It does not perform initialization of memory.	It performs memory initialization.

Q.72 why do we use multidimensional array?

Sol:-

A multidimensional array can be useful to organize subgroups of data within an array. In addition to organizing data stored in elements of an array, a multidimensional array can store memory addresses of data in a pointer array and an array of pointers.

Qu.73 Explain the difference among null pointer and void pointer?

Sol:-

NULL pointer • A null pointer means it is not pointing to anything. If, there is no address that is assigned to a pointer, then set it to null. • Null pointer is a value • The syntax are: <code><data type> *<variable name> = NULL;</code> • For example, <code>int *p = NULL; char *p = '\0';</code> • Program example: <pre>#include<stdio.h> int main(){ printf("EDAC 2021"); int *p = NULL; // ptr is a NULL pointer printf("\n The value of pointer is: %x", p); return 0;}</pre> Output : EDAC 2021 The value of pointer is: 0	Void Pointer • A void pointer is nothing but the one who does not have any data type with it. It is also called as a general purpose pointer. It can hold the addresses of any data type. • Void pointer is a type • Thee syntax is as follows: <code>void *<data type>;</code> • For example, <code>void *p; int a; char c;</code> • Program example: <pre>#include<stdio.h> int main(){ int a = 10; void *ptr = &a; printf("%d", *(int *)ptr); return 0;}</pre> Output: 10
---	--

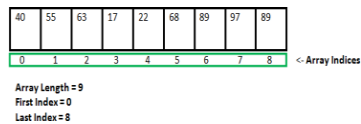
74) Explain the difference among Array and Stack?

Sol:-

Array:

- Arrays are defined as the collection of similar type of data items stored at contiguous memory locations.

- Diagram:



- In the array the elements belong to indexes, i.e., if you want to get into the fourth element you have to write the variable name with its index or location within the square bracket eg. arr[4]

- Insertion and deletion in array can be done at any index in the array.

- Array are of fixed size.

- Array contains elements of same data type.

- We can do both linear and Binary search.

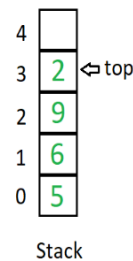
- Program example:

```
class Testarray{
public static void main(String args[]){
int a[]=new int[5]; //declaration and
instantiation
a[0]=10; //initialization
a[1]=20;
```

Stack:

- A Stack is a linear data structure that follows the LIFO (Last-In-First-Out) principle.

- Diagram:



- Stacks are based on the LIFO principle, i.e., the element inserted at the last, is the first element to come out of the list.

- Insertion and deletion in stacks takes place only from one end of the list called the top.

- Stack has a dynamic size.

- Stack can contain elements of different data type.

- We can do only linear search.

- Program example:

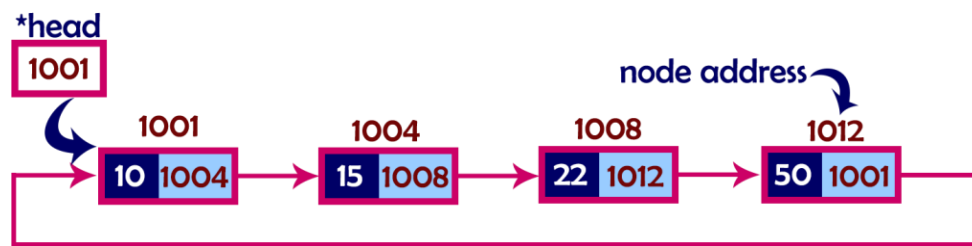
```
public MyStack(int s) {
maxSize = s;
stackArray = new long[maxSize];
top = -1;
}
public void push(long j) {
stackArray[++top] = j;
}
public long pop() {
```

<pre> a[2]=70; a[3]=40; a[4]=50; //traversing array for(int i=0;i<a.length;i++)//length is the property of array System.out.println(a[i]); }} Output: 10 20 70 40 50 </pre>	<pre> return stackArray[top--]; } public long peek() { return stackArray[top]; } public boolean isEmpty() { return (top == -1); } public boolean isFull() { return (top == maxSize - 1); } public static void main(String[] args) { MyStack theStack = new MyStack(10); theStack.push(10); theStack.push(20); theStack.push(30); theStack.push(40); theStack.push(50); while (!theStack.isEmpty()) { long value = theStack.pop(); System.out.print(value); System.out.print(" "); } System.out.println(""); } } Output: 50 40 30 20 10 </pre>
--	---

Qu.75 Tell How to Check Whether a Linked List Is Circular?

Sol:-

Circular linked list is a sequence of elements in which every element has link to its next element in the sequence and the last element has a link to the first element in the sequence.



Java program to check if linked list is circular

```

import java.util.*;
class Main
{
/* Link list Node */
static class Node
{
int data;
Node next;
}
/*This function returns true if given linked
list is circular, else false. */
static boolean isCircular( Node head)
{
// An empty linked list is circularif
(head == null)
return true;
// Next of head
Node node = head.next;
// This loop would stop in both cases (1) If
// Circular (2) Not circular
while (node != null && node != head)
node = node.next;
// If loop stopped because of circular
// condition
return (node == head);
}
// Utility function to create a new node.
static Node newNode(int data)
{
Node temp = new Node();
temp.data = data;

```

```

temp.next = null;
return temp;
}
/* Driver code*/
public static void main(String args[])
{
/* Start with the empty list */
Node head = newNode(1);
head.next = newNode(2);
head.next.next = newNode(3);
head.next.next.next = newNode(4);
System.out.print(isCircular(head)? "Yes\n" : "No\n" );
// Making linked list circular head.next.next.next.next =
head; System.out.print(isCircular(head)? "Yes\n" : "No\n"
);
}
}

```

76. How Can I Search for Data in A Linked List?

Sol:-

Searching is performed in order to find the location of a particular element in the list. Searching any element in the list needs traversing through the list and make the comparison of every element of the list with the specified element. If the element is matched with any of the list element, then the location of the element is returned from the function.

Algorithm

- Step 1: SET PTR = HEAD
- Step 2: Set I = 0
- STEP 3: IF PTR = NULL

WRITE "EMPTYLIST"

GOTO STEP 8

END OF IF

- STEP 4: REPEAT STEP 5 TO 7 UNTIL PTR != NULL
- STEP 5: if ptr → data = item

Write i+1

End of IF

- STEP 6: $I = I + 1$
- STEP 7: $PTR = PTR \rightarrow NEXT$

[END OF LOOP]

- STEP 8: EXIT

// Iterative Java program to search an element in linked list

//Node class

class Node

{

int data;

Node next;

Node(int d)

{

data = d; next

= null;

}

}

//Linked list class

class LinkedList

{

Node head; //Head of list

//Inserts a new node at the front of the list

public void push(int new_data)

{

//Allocate new node and putting data Node

new_node = new Node(new_data);

//Make next of new node as head

new_node.next = head;

//Move the head to point to new Node

```

head = new_node;
}
//Checks whether the value x is present in linked list
public boolean search(Node head, int x)
{
    Node current = head; //Initialize current
    while (current != null)
    {
        if (current.data == x)
            return true; //data found
        current = current.next;
    }
    return false; //data not found
}

```

```

//Driver function to test the above functions
public static void main(String args[])
{

```

```

//Start with the empty list LinkedList
l1 = new LinkedList();

```

```

/*Use push() to construct below list14-
>21->11->30->10 */

```

```

l1.push(10);
l1.push(30);
l1.push(11);
l1.push(21);
l1.push(14);

```

```

if (l1.search(l1.head, 21))
    System.out.println("Yes");
else System.out.println("No");
}
}

```

Qu.77 Define The Goals Of Data Structure?

Sol:-

A data structure is a data organization, management and storage format that enable efficient access and modification. More precisely, a data structure is a collection of data

values, the relationships among them, and the functions or operations that can be applied to the data.

- 1) Focus on tradeoffs, and reinforce the concept that there are costs and benefits associated with every data structure or algorithm. This is done by describing, for each data structure, the amount of space and time required for typical operations.
- 2) For each algorithm, we examine the time required for key input types. It will teach you how to measure the effectiveness of a data structure or algorithm. Only through such measurement can you determine which data structure in your toolkit is most appropriate for a new problem. The techniques presented also allow you to judge the merits of new data structures that you or others might invent.
- 3) To design an algorithm that is easy to understand, code, and debug.
- 4) To design an algorithm that makes efficient use of the computer's resources.

Qu.78 What Does Abstract Data Type Mean?

Sol:-

The abstract datatype is special kind of datatype, whose behavior is defined by a set of values and set of operations. The keyword “Abstract” is used as we can use these datatypes, we can perform different operations. But how those operations are working that is totally hidden from the user. The ADT is made of with primitive datatypes, but operation logics are hidden.

Some examples of ADT are Stack, Queue, List etc. Let us

see some operations of those mentioned ADT –

- Stack –
 - isFull(), This is used to check whether stack is full or not
 - isEmpty(), This is used to check whether stack is empty or not
 - push(x), This is used to push x into the stack
 - pop(), This is used to delete one element from top of the stack
 - peek(), This is used to get the top most element of the stack
 - size(), this function is used to get number of elements present into the stack
- Queue –
 - isFull(), This is used to check whether queue is full or not
 - isEmpty(), This is used to check whether queue is empty or not
 - insert(x), This is used to add x into the queue at the rear end
 - delete(), This is used to delete one element from the front end of the queue
 - size(), this function is used to get number of elements present into the queue
- List –
 - size(), this function is used to get number of elements present into the list

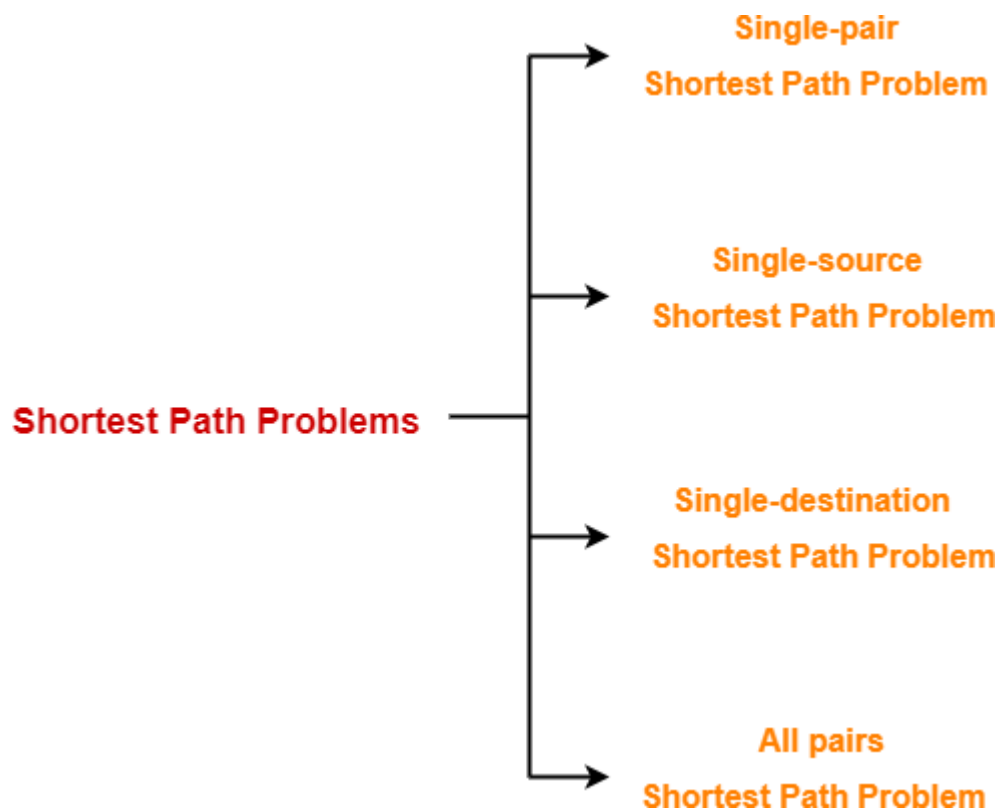
- insert(x), this function is used to insert one element into the list
- remove(x), this function is used to remove given element from the list
- get(i), this function is used to get element at position i
- replace(x, y), this function is used to replace x with y value

Qu 79. What Do You Mean By Shortest Path in data structure?

- Sol:- Shortest path problem is a problem of finding the shortest path(s) between vertices of a given graph.
- Shortest path between two vertices is a path that has the least cost as compared to all other existing paths
- Shortest path algorithms are a family of algorithms used for solving the shortest path problem.

Types of Shortest Path Problem-

Various types of shortest path problem are-



1. Single-pair shortest path problem

2. Single-source shortest path problem
3. Single-destination shortest path problem
4. All pairs shortest path problem

□ Single-Pair Shortest Path Problem-

- It is a shortest path problem where the shortest path between a given pair of vertices is computed.
- A* Search Algorithm is a famous algorithm used for solving single-pair shortest path problem.

□ Single-Source Shortest Path Problem-

- It is a shortest path problem where the shortest path from a given source vertex to all other remaining vertices is computed.
- Dijkstra's Algorithm and Bellman Ford Algorithm are the famous algorithms used for solving single-source shortest path problem.

□ Single-Destination Shortest Path Problem-

- It is a shortest path problem where the shortest path from all the vertices to a single destination vertex is computed.
- By reversing the direction of each edge in the graph, this problem reduces to single-source shortest path problem.
- Dijkstra's Algorithm is a famous algorithm adapted for solving single-destination shortest path problem.

□ All Pairs Shortest Path Problem-

- It is a shortest path problem where the shortest path between every pair of vertices is computed.
- Floyd-Warshall Algorithm and Johnson's Algorithm are the famous algorithms used for solving All pairs shortest path problem.

Qu.80 State the difference among Primitive and Non-primitive Data Types?

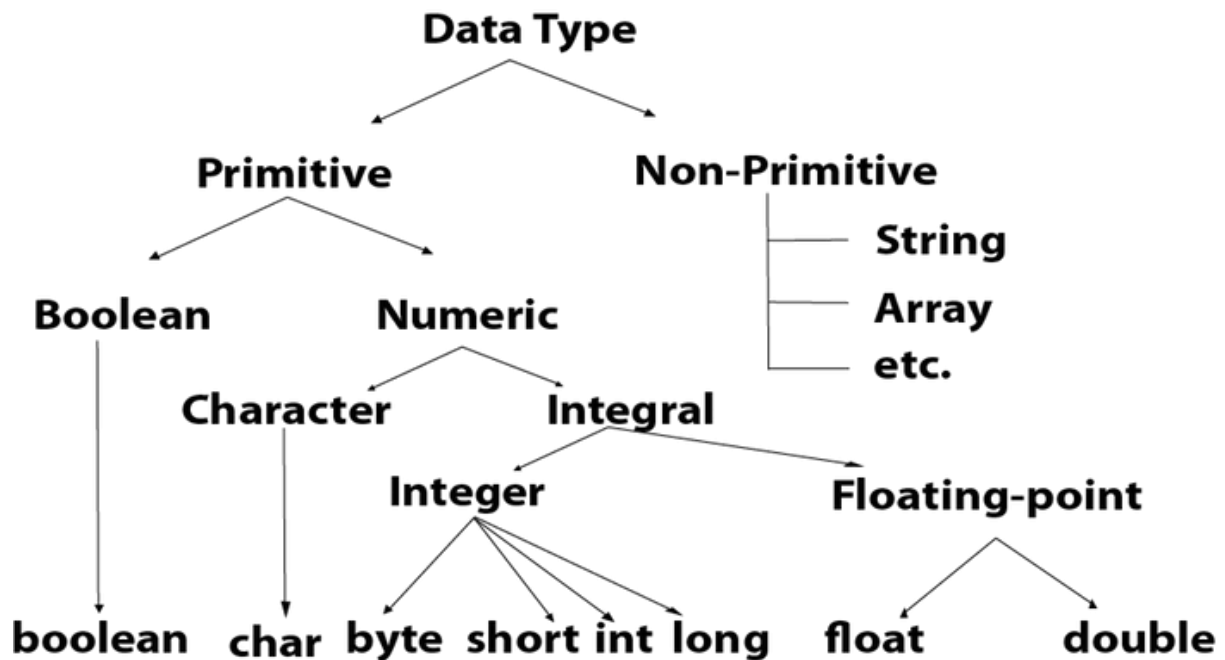
Sol:-

Primitive Data Types

- Primitive data types are predefined types of data, which are supported by the programming language. For example, integer, character, and string are all primitive data types.
- A primitive data type is one that fits the base architecture of the underlying computer such as int, float, and pointer, and all of the variations, thereof such as char short long unsigned float double and etc, are a primitive data type.
- Primitive data are only single values, they have not special capabilities.
- The integer, real, logic data, character data, pointer, and reference are primitive data structures. Data structures that normally are directly operated upon by machine-level instructions are known as the primitive structure and data type.

Non-primitive Data Types

- Non-primitive data types are not defined by the programming language, but are instead created by the programmer. They are sometimes called “reference variables,” or “object references,” since they reference a memory location, which stores the data.
- A non-primitive data type is something else such as an array, structure, or class, known as the non-primitive data type.
- The data type that is derived from primary data types is known as a non-primitive data type.
- Examples of the non-primitive data types are Array, structure, union, link list, stacks, queue etc...



81. Define the Types of Queues?

Ans- There are four different types of queues: -

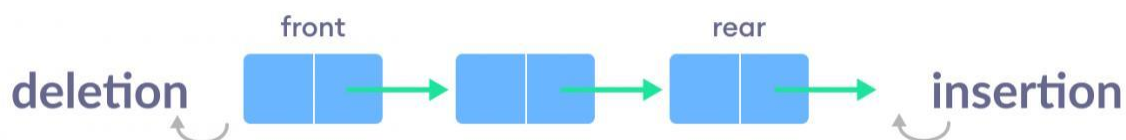
A queue is a useful data structure in programming. It is similar to the ticket queue outside a cinema hall, where the first person entering the queue is the first person who gets the ticket.

There are four different types of queues:

- Simple Queue
- Circular Queue
- Priority Queue
- Double Ended Queue

Simple Queue

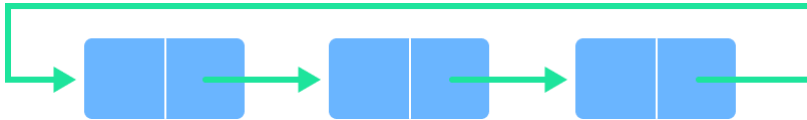
In a simple queue, insertion takes place at the rear and removal occurs at the front. It strictly follows the FIFO (First in First out) rule.



Simple Queue Representation

Circular Queue

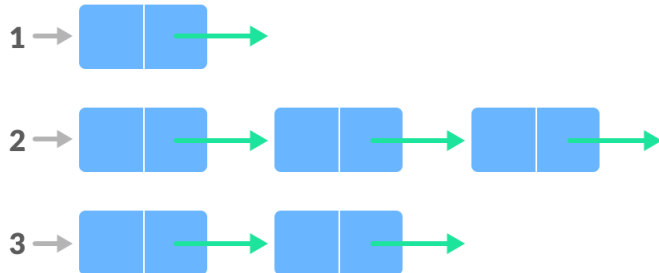
In a circular queue, the last element points to the first element making a circular link.



Priority Queue

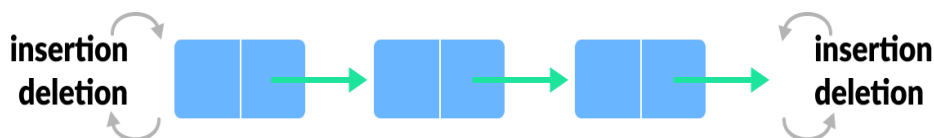
A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. If elements with the same priority occur, they are served according to their order in the queue.

Priority



Deque (Double Ended Queue)

In a double ended queue, insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow the FIFO (First In First Out) rule.



82. List the Applications of Queues?

Ans-

1. Implementation of a queue is also an important application of data structure. Nowadays computer handles multiuser, multiprogramming environment and time-sharing environment. In this environment a system (computer) handles several jobs at a time, to handle these jobs the concept of a queue is used.
2. To implement printer spooler so that jobs can be printed in the order of their arrival.

3. Round robin scheduling technique is implemented using queue
4. All types of customer service (like railway reservation) centres are designed using the concept of queues.

Real world applications: -

1. Cashier line in any store.
 2. Waiting on hold for tech support.
 3. People on an escalator.
 4. Checkout at any book store.
- Applications related to computer science:
1. When data is transferred asynchronously between two processes. e.g., IO Buffers.
 2. When a resource is shared among multiple consumers. Examples include CPU scheduling, Disk Scheduling.
 3. In recognizing palindrome.
 4. In shared resources management.
 5. Keyboard buffer.
 6. Round robin scheduling.
 7. Job scheduling.
 8. Simulation.

83. Define minimum number of Queues needed to implement the priority queue?

Ans - Priority Queue is an extension of queue with following properties.

- 1) Every item has a priority associated with it.
- 2) An element with high priority is de-queued before an element with low priority.
- 3) If two elements have the same priority, they are served according to their order in the queue.

The minimum number of queues needed in this case is two. One queue is intended for sorting priorities while the other queue is intended for actual storage of data.

84. When can you tell that a Memory Leak will occur?

Ans- 1. A memory leak may also happen when an object is stored in memory but cannot be accessed by the running code.

2. In other words, leaks mean that dynamically-allocated memory cannot be released back to the operating system because the program no longer contains pointers that can access it.

You've lost control of that piece of memory regardless of size and can no longer access it or free it.

3. A Memory Leak is a situation when there are objects present in the heap that are no longer used, but the garbage collector is unable to remove them from memory and, thus they are unnecessarily maintained.

4. A memory leak is bad because it blocks memory resources and degrades system performance over time. And if not dealt with, the application will eventually exhaust its resources, finally terminating with a fatal `java.lang.OutOfMemoryError`.

5. There are two different types of objects that reside in Heap memory — referenced and unreferenced. Referenced objects are those who have still active references within the application whereas unreferenced objects don't have any active references.

6. The garbage collector removes unreferenced objects periodically, but it never collects the objects that are still being referenced. This is where memory leaks can occur:

Types of Memory Leaks in Java -

1. Memory Leak Through static Fields
2. Through Unclosed Resources
3. Improper `equals ()` and `hash Code ()` Implementations
4. Inner Classes That Reference Outer Classes
5. Through `finalize ()` Methods

85. How do signed and unsigned numbers affect memory.

Ans-

In the case of signed numbers, the first bit is used to indicate whether positive or negative, which leaves you with one bit short. With unsigned numbers, you have all bits available for that number. The effect is best seen in the number range (unsigned 8 bit number has a range 0-255, while 8-bit signed number has a range -128 to +127).

86. State the different Types of Linked Lists?

Ans-

Types of Linked list

- ☐ Singly **Linked list**.
- ☐ Doubly **Linked list**.
- ☐ Circular **Linked list**.
- ☐ Doubly Circular **Linked list**.

Singly Linked List : It is the simplest type of linked list in which every node contains some data and a pointer to the next node of the same data type. The node contains a

pointer to the next node means that the node stores the address of the next node in the sequence. A single linked list allows traversal of data only in one way.

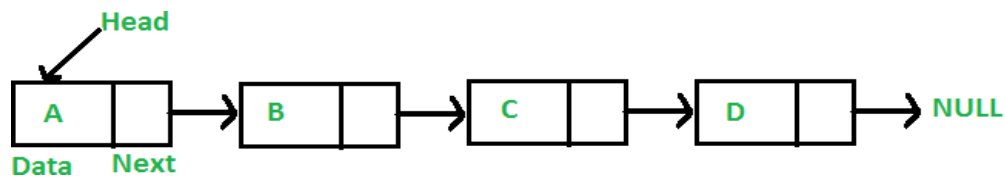


Diagram: Singly Linked List

Application:

- ☐ Implementation of stacks and queues
- ☐ Implementation of graphs: Adjacency list representation of graphs is most popular which uses linked list to store adjacent vertices.
- ☐ Dynamic memory allocation: We use linked list of free blocks.
- ☐ Maintaining directory of names

Real time example:

1. Image viewer- Previous and next images are linked, hence can be accessed by next and previous button.
2. Previous and next page in web browser- We can access previous and next url searched in web browser by pressing back and next button since, they are linked as linked list.
3. Music player - Songs in music player are linked to previous and next song. you can play songs either from starting or ending of the list.

Doubly Linked List: A doubly linked list or a two-way linked list is a more complex type of linked list which contains a pointer to the next as well as the previous node in sequence. Therefore, it contains three parts: data, a pointer to the next node, and a pointer to the previous node. This would enable us to traverse the list in the backward direction as well. Below is the image for the same:

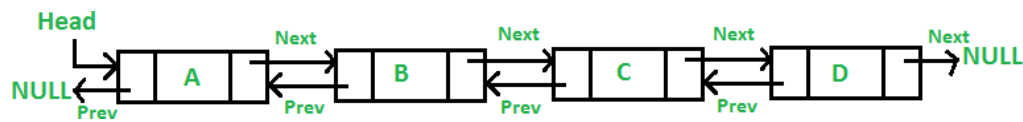


Diagram: Doubly Linked List

Application:

- Represent a deck of cards in game.
- The browser cache which allows you to hit the Back-Forward pages.
- Application that have a most Recently used list (a linked of file names)
- Undo-Redo functionality

Circular Linked List: A circular linked list is that in which the last node contains the pointer to the first node of the list.

While traversing a circular linked list, we can begin at any node and traverse the list in any direction forward and backward until we reach the same node we started.

Thus, a circular linked list has no beginning and no end.

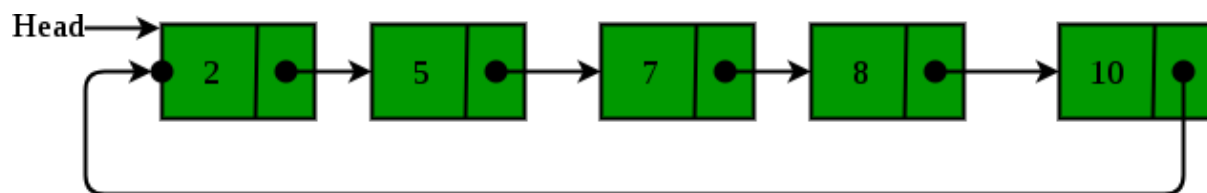


Diagram: Circular Linked List

Applications:

- Useful for implementation of queue. Unlike this implementation, we don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.

- Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.

Doubly Circular Linked List: Circular Doubly Linked List has properties of both doubly linked list and circular linked list in which two consecutive elements are linked or connected by previous and next pointer and the last node points to first node by next pointer and also the first node points to last node by previous pointer.

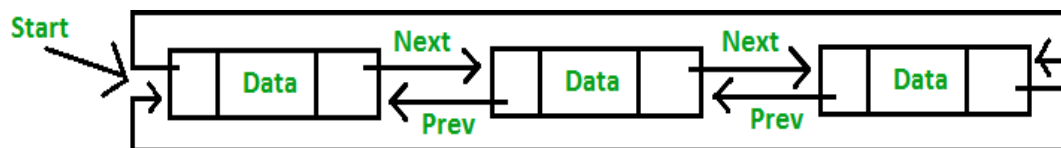


Diagram: Doubly circular Linked List

Application :

- Managing shopping cart in online shopping.

87) List the Basic Operations Carried Out in A Linked List?

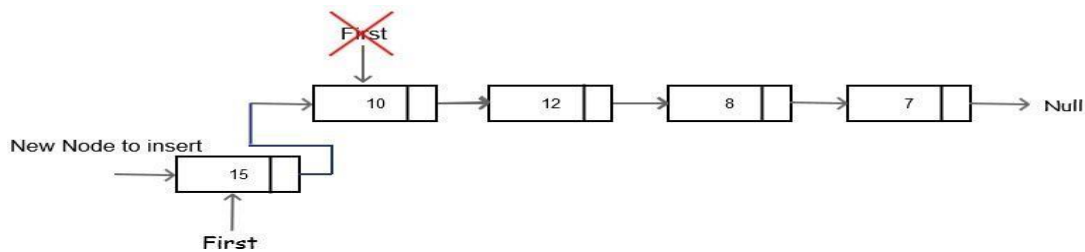
Ans- Linked List: A linked list is a data structure used for storing collections of data. Basic Operations on a List:

- Traversing the list: access each element of the linked list



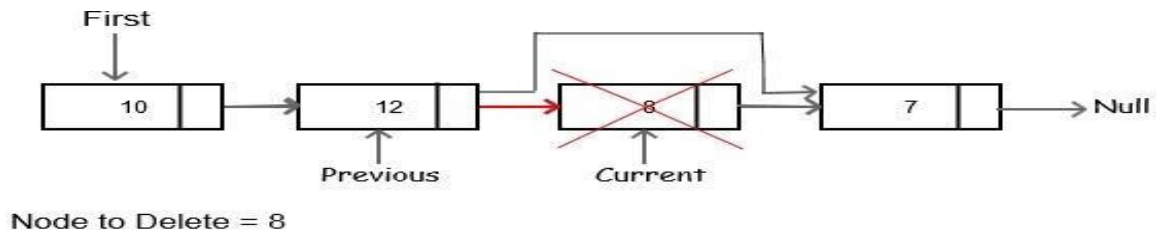
- Inserting an item in the list: adds a new element.

Insert at the beginning:



- Deleting an item from the list : removes the existing elements

Delete item from specific position:



88) write a syntax in c to create a node in the singly linked list? Ans- A

linked list is a data structure used for storing collections of data. Generally, “linked list” means a singly linked list.

Syntax:

```
struct Node
{
    int data;
    struct Node *next;
};
```

89. Can we apply Binary search algorithm to a sorted Linked list?

Ans- To perform a Binary search based on Divide and Conquer Algorithm, determination of the middle element is important. Binary Search is usually fast and efficient for arrays because accessing the middle index between two given indices is easy and fast (Time Complexity $O(1)$). But memory allocation for the singly linked list is dynamic and non-contiguous, which makes finding the middle element difficult.

One approach could be of using skip list one could be traversing the linked list using one pointer.

Prerequisite: Finding the middle element of a linked list.

Approach:

- Here, start node (set to Head of list), and the last node (set to NULL initially) are given.

- ❑ Middle is calculated using two pointers approach.
- ❑ If middle's data matches the required value of search, return it.
- ❑ Else if middle's data < value, move to upper half (setting start to middle's next).
- ❑ Else go to lower half (setting last to middle).
- ❑ The condition to come out is, either element found or entire list is traversed. When entire list is traversed, last points to start i.e. last -> next == start.
- ❑ In main function, function **InsertAtHead** inserts value at the beginning of linked list. Inserting such values (for sake of simplicity) so that the list created is sorted.

```
// Java code to implement binary search
// on Singly Linked List
// Node Class
class Node
{
    int data;
    Node next;

    // Constructor to create a new node
    Node(int d)
    {
        data = d;
        next = null;
    }
}

class BinarySearch
{
    // function to insert a node at the beginning
    // of the Singly Linked List
    static Node push(Node head, int data)
    {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
        return head;
    }

    // Function to find middle element
    // using Fast and Slow pointers
    static Node middleNode(Node start, Node last)
    {
        if (start == null)
            return null;

        Node slow = start;
        Node fast = start.next;
    }
}
```

```

while (fast != last)
{
    fast = fast.next;if
    (fast != last)
    {
        slow = slow.next;
        fast = fast.next;
    }
}
return slow;
}

// function to insert a node at the beginning
// of the Singly Linked List
static Node binarySearch(Node head, int value)
{
    Node start = head;Node
    last = null;

    do
    {
        // Find Middle
        Node mid = middleNode(start, last);

        // If middle is empty
        if (mid == null)
            return null;

        // If value is present at middleif
        (mid.data == value)
            return mid;

        // If value is less than mid
        else if (mid.data > value)
        {
            start = mid.next;
        }

        // If the value is more than mid.
        else
            last = mid;
    } while (last == null || last != start);

    // value not present
    return null;
}

```

```
}  
// Driver Code  
public static void main(String[] args)  
{  
    Node head = null;  
  
    // Using push() function to  
    // convert singly linked list  
    // 10 -> 9 -> 8 -> 7 -> 4 -> 1  
    head = push(head, 1);  
    head = push(head, 4);  
    head = push(head, 7);  
    head = push(head, 8);  
    head = push(head, 9);  
    head = push(head, 10);  
    int  
    value = 7;  
  
    if (binarySearch(head, value) == null)  
    {  
        System.out.println("Value not present");  
    }  
    else  
    {  
        System.out.println("Present");  
    }  
}  
}
```

90. How do you search for a target key in a linked list?

Ans- To find the target key in a linked list, you have to apply sequential search. Each node is traversed and compared with the target key, and if it is different, then it follows the link to the next node. This traversal continues until either the target key is found or if the last node is reached.

Steps:

- 1) Initialize a node pointer, current = head.
- 2) Do following while current is not NULL
 - a) current->key is equal to the key being searched return true.
 - b) current = current->next
- 3) Return false

Code:

```
// Iterative Java program to search an element  
// in linked list
```

```
//Node class  
class Node  
{  
    int data;  
    Node next;  
    Node(int d)  
    {  
        data = d;  
        next = null;  
    }  
}
```

```
//Linked list class  
class LinkedList  
{
```

```

Node head;    //Head of list

//Inserts a new node at the front of the list
public void push(int new_data)
{
    //Allocate new node and putting data Node
    new_node = new Node(new_data);

    //Make next of new node as head
    new_node.next = head;

    //Move the head to point to new Node
    head = new_node;
}

//Checks whether the value x is present in linked list
public boolean search(Node head, int x)
{
    Node current = head;    //Initialize current
    while (current != null)
    {
        if (current.data == x)
            return true;    //data found
        current = current.next;
    }
    return false;    //data not found
}

//Driver function to test the above functions
public static void main(String args[])
{
    //Start with the empty list LinkedList
    llist = new LinkedList();

    /*Use push() to construct below list 14-
    >21->11->30->10 */
    llist.push(10);
    llist.push(30);
    llist.push(11);
    llist.push(21);
    llist.push(14);

    if (llist.search(llist.head, 21))
        System.out.println("Yes");
    else

```

```
        System.out.println("No");  
    }  
}
```

Q91) How does selection sort work?

Ans- Selection sort is a simple sorting algorithm. This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end. Initially, the sorted part is empty and the unsorted part is the entire list.

The smallest element is selected from the unsorted array and swapped with the leftmost element, and that element becomes a part of the sorted array. This process continues moving unsorted array boundary by one element to the right.

This algorithm is not suitable for large data sets as its average and worst case complexities are of $O(n^2)$, where n is the number of items.

How selection sort work

Consider the following depicted array as an example.



For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.



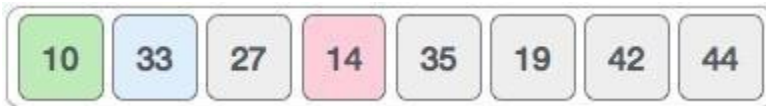
So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.



For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.



We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.



After two iterations, two least values are positioned at the beginning in a sorted manner.



The same process is applied to the rest of the items in the array. Code

Example:

```
class SelectionSort
{
    void sort(int arr[])
    {
        int n = arr.length;

        // One by one move boundary of unsorted subarray for
        (int i = 0; i < n-1; i++)
```

```
{  
    // Find the minimum element in unsorted array  
    int min_idx = i;  
    for (int j = i+1; j < n; j++)  
        if (arr[j] < arr[min_idx])  
            min_idx = j;  
  
    // Swap the found minimum element with the first  
    // element  
    int temp = arr[min_idx];  
    arr[min_idx] = arr[i];  
    arr[i] = temp;  
}  
}  
  
// Prints the array  
void printArray(int arr[])  
{  
    int n = arr.length;  
    for (int i=0; i<n; ++i)  
        System.out.print(arr[i]+" ");  
    System.out.println();  
}
```

```
// Driver code to test above

public static void main(String args[])
{
    SelectionSort ob = new SelectionSort();int
    arr[] = {64,25,12,22,11};

    ob.sort(arr);

    System.out.println("Sorted array");

    ob.printArray(arr);
}
}
```

Q92) Briefly write in brief the approaches to develop algorithm?

Ans- An Algorithm is a procedure to solve a particular problem in a finite number of steps for a finite-sized input.

The algorithms can be classified in various ways. They are:

1. Implementation Method
2. Design Method
3. Other Classifications

Classification by Implementation Method: There are primarily three main categories into which an algorithm can be named in this type of classification. They are:

1. Recursion or Iteration: A recursive algorithm is an algorithm which calls itself again and again until a base condition is achieved whereas iterative algorithms use loops and/or data structures like stacks, queues to solve any problem. Every recursive solution can be implemented as an iterative solution and vice versa.
Example: The Tower of Hanoi is implemented in a recursive fashion while Stock Span problem is implemented iteratively.
2. Exact or Approximate: Algorithms that are capable of finding an optimal solution for any problem are known as the exact algorithm. For all those problems, where it is not possible to find the most optimized solution, an approximation algorithm is used. Approximate algorithms are the type of algorithms that find the result as an average outcome of sub outcomes to a problem.

Example: For NP-Hard Problems, approximation algorithms are used. Sorting algorithms are the exact algorithms.

3. Serial or Parallel or Distributed Algorithms: In serial algorithms, one instruction is executed at a time while parallel algorithms are those in which we divide the problem into subproblems and execute them on different processors. If parallel algorithms are distributed on different machines, then they are known as distributed algorithms

Classification by Design Method: There are primarily three main categories into which an algorithm can be named in this type of classification. They are:

1. Greedy Method: In the greedy method, at each step, a decision is made to choose the local optimum, without thinking about the future consequences. Example: Fractional Knapsack, Activity Selection.
2. Divide and Conquer: The Divide and Conquer strategy involves dividing the problem into sub-problem, recursively solving them, and then recombining them for the final answer.
Example: Merge sort, Quicksort.
3. Dynamic Programming: The approach of Dynamic programming is similar to divide and conquer. The difference is that whenever we have recursive function calls with the same result, instead of calling them again we try to store the result in a data structure in the form of a table and retrieve the results from the table. Thus, the overall time complexity is reduced. "Dynamic" means we dynamically decide, whether to call a function or retrieve values from the table.

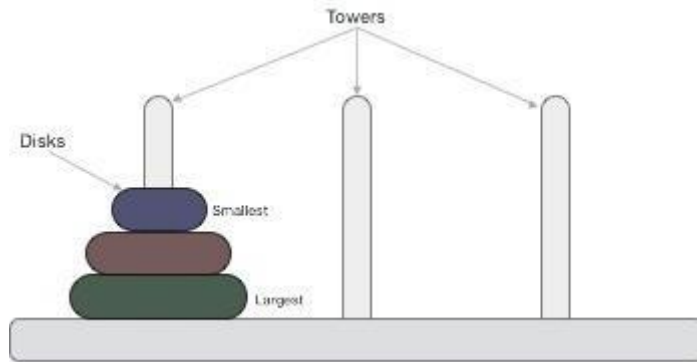
93. Which data structure is ideal to perform Recursion operation and why? Ans- Stack.

Because of its LIFO (Last In First Out) property it remembers its 'caller' so knows whom to return when the function has to return. Recursion makes use of system stack for storing the return addresses of the function calls.

Every recursive function has its equivalent iterative (non-recursive) function. Even when such equivalent iterative procedures are written, explicit stack is to be used.

94. Explain tower of Hanoi.

Ans- Tower of Hanoi, is a mathematical puzzle which consists of three towers (pegs) and more than one rings is as depicted –



These rings are of different sizes and stacked upon in an ascending order, i.e. the smaller one sits over the larger one. There are other variations of the puzzle where the number of disks increase, but the tower count remains the same.

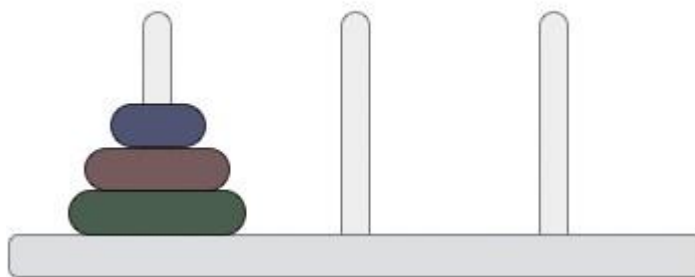
Rules

The mission is to move all the disks to some another tower without violating the sequence of arrangement. A few rules to be followed for Tower of Hanoi are –

- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.
- No large disk can sit over a small disk.

Following is an animated representation of solving a Tower of Hanoi puzzle with three disks.

Step: 0



Tower of Hanoi puzzle with n disks can be solved in minimum $2^n - 1$ steps. This presentation shows that a puzzle with 3 disks has taken $2^3 - 1 = 7$ steps.

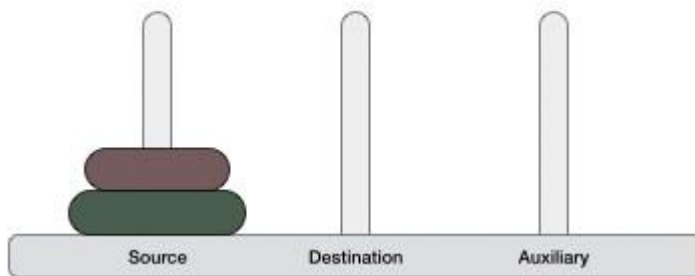
Algorithm

To write an algorithm for Tower of Hanoi, first we need to learn how to solve this problem with lesser amount of disks, say $\rightarrow 1$ or 2 . We mark three towers with name, source, destination and aux (only to help moving the disks). If we have only one disk, then it can easily be moved from source to destination peg.

If we have 2 disks –

- First, we move the smaller (top) disk to aux peg.
- Then, we move the larger (bottom) disk to destination peg.
- And finally, we move the smaller disk from aux to destination peg.

Step: 0



So now, we are in a position to design an algorithm for Tower of Hanoi with more than two disks. We divide the stack of disks in two parts. The largest disk (nth disk) is in one part and all other (n-1) disks are in the second part.

Our ultimate aim is to move disk n from source to destination and then put all other (n-1) disks onto it. We can imagine to apply the same in a recursive way for all given set of disks.

95. Explain interpolation search technique?

Ans- Interpolation search is an improved variant of binary search. This search algorithm works on the probing position of the required value. For this algorithm to work properly, the data collection should be in a sorted form and equally distributed.

Binary search has a huge advantage of time complexity over linear search. Linear search has worst-case complexity of $O(n)$ whereas binary search has $O(\log n)$.

There are cases where the location of target data may be known in advance. For example, in case of a telephone directory, if we want to search the telephone number of Morpheus. Here, linear search and even binary search will seem slow as we can directly jump to memory space where the names start from 'M' are stored.

Positioning in Binary Search

In binary search, if the desired data is not found then the rest of the list is divided in two parts, lower and higher. The search is carried out in either of them.



Even when the data is sorted, binary search does not take advantage to probe the position of the desired data.

Position Probing in Interpolation Search

Interpolation search finds a particular item by computing the probe position. Initially, the probe position is the position of the middle most item of the collection.



96. Explain hashing?

Ans- Hashing is a technique or process of mapping keys, values into the hash table by using a hash function. It is done for faster access to elements. The efficiency of mapping depends on the efficiency of the hash function used.

Let a hash function $H(x)$ maps the value x at the index $x\%10$ in an Array. For example if the list of values is [11,12,13,14,15] it will be stored at positions {1,2,3,4,5} in the array or Hash table respectively.

97. Explain merge sort and how it works?

Ans-

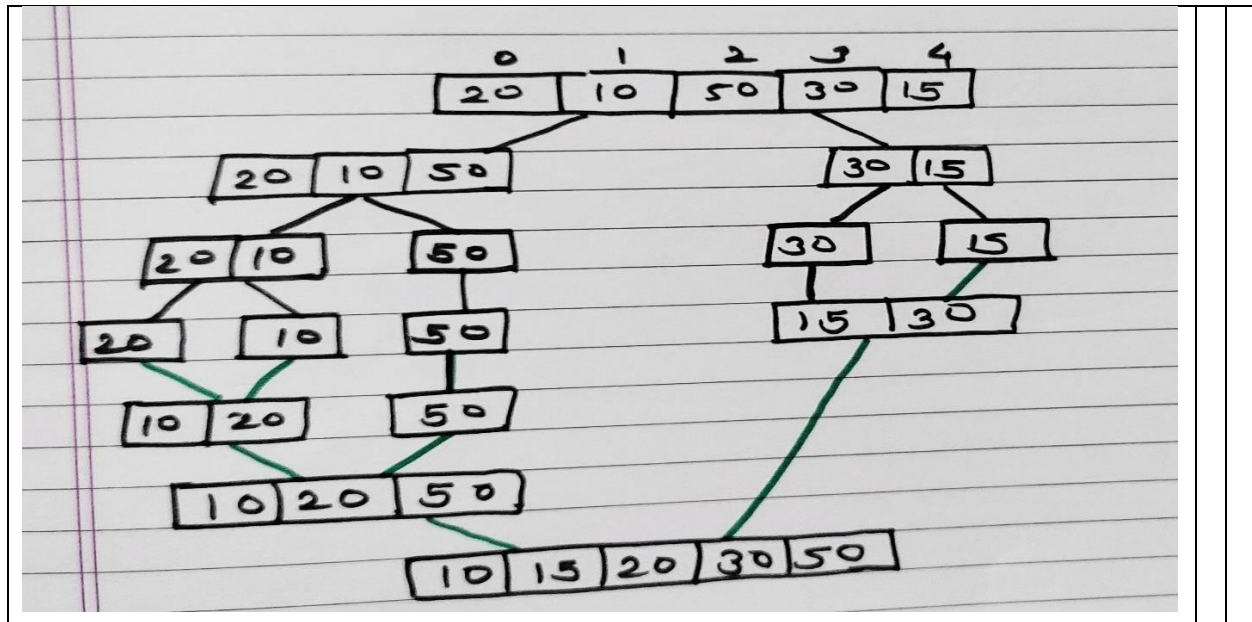


Fig. Merge Sort.

Merge sort is based on divide and conquer strategy.

DIVIDE & CONQUER STRATEGY: Large problem is gets divided into smaller sub problem. For each subproblem solution will be find out and then combine the solution of subproblem and finally you will get the solution for main problem.

Explanation:

- 1) First the Array is gets divided into 2 subparts and so on. The mid-point will be the pivot element and the process will be carried out recursively until the no possible division further.
- 2) Then sort and merge the divided array (from bottom to top) and will get the sorted array. As 10, 15, 20, 30, 50.

98. Explain shell sort?

Ans- The **shell sort**, works on the insertion sort by breaking the original list into a number of smaller sub lists, each of which is sorted using an insertion sort. The unique way that these sub lists are chosen is the key to the shell sort. Instead of breaking the list into sub lists of contiguous items, the shell sort uses gap for each pass, and select the elements towards the right of gap.

Explanation:

Firstly, the algorithm sorts the elements that are far away from each other. Then, the gap becomes smaller and closer elements are compared. This way, some elements that aren't in a correct position can be positioned faster than if we made the subsets out of the neighbouring elements.

- 1) is the example with the gaps of 3 and 1 and the unsorted list of 9 elements:
- 2) If we follow the above description, in the first iteration, we'll have three subsets with 3 elements

- 3) After sorting each of the subsets in the first iteration, the list would look like
- 4) Finally, we need to do one more sort with the increment of one and it's actually a basic insertion sort. The number of shifting operations that we need to perform to sort a list is now smaller than it would be the case if we didn't do the first iteration:



Final Sorted List (Shell Sort)

99. List some applications of Tree-data structure?

Ans- One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer:

file system

```
/ <-- root
/
...   home
      /
      ugrad   course
      /      / |
      ...    cs101 cs112 cs113
```

1. If we organize keys in form of a tree (with some ordering e.g., BST), we can search for a given key in moderate time (quicker than Linked List and slower than arrays).
2. We can insert/delete keys in moderate time (quicker than Arrays and slower than Unordered Linked Lists).

3. Store hierarchical data, like folder structure, organization structure, XML/HTML data.
4. Binary Search Tree is a tree that allows fast search, insert, delete on a sorted data. It also allows finding closest item.
5. Heap is a tree data structure which is implemented using arrays and used to implement priority queues.
6. Spanning Trees and shortest path trees are used in routers and bridges respectively in computer networks

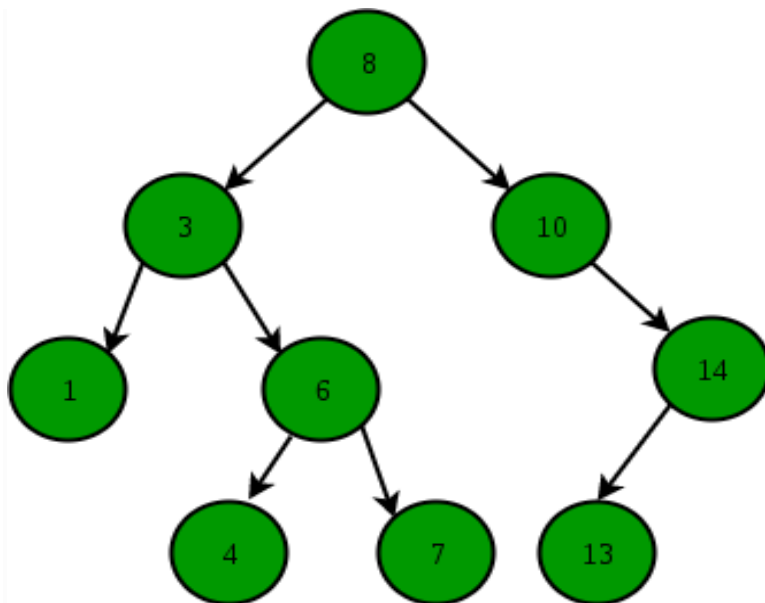
100. why it is said that searching a node in binary search tree is efficient than that of simple binary tree?

Ans-

Binary search tree:

Binary Search Tree is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.



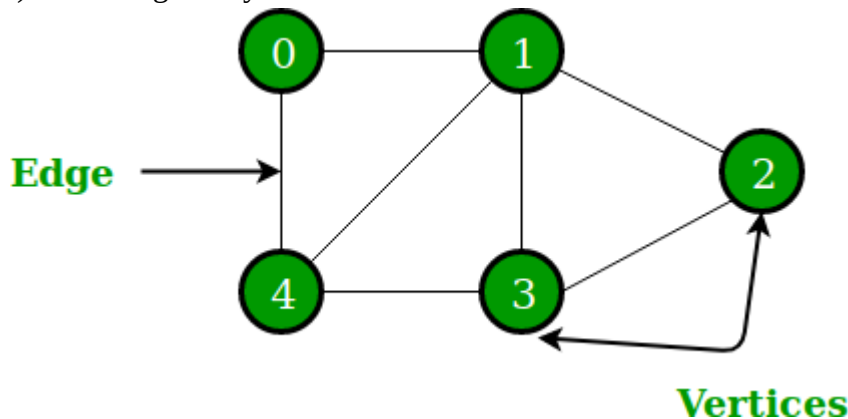
Binary Search Tree

In binary search tree, the nodes are arranged in such a way that the left node is having less data value than root node value and the right nodes are having larger value than that of root. Because of this while searching any node the value of the target node will be compared with the parent node and accordingly either left sub branch or right sub branch will be searched. So, one has to compare only particular branches. Thus, searching becomes efficient.

101. Define Applications of Graph Data Structure? Ans-

1. Graphs are used to represent networks of communication.
2. Graphs are used to represent data organization.
3. Graph theory is used to find shortest path in road or a network.
4. In Google Maps, various locations are represented as vertices or nodes and the roads are represented as edges and graph theory is used to find the shortest path between two nodes.
5. In Electrical Engineering, graph theory is used in designing of circuit connections. These circuit connections are named as topologies. Some topologies are series, bridge, star and parallel topologies.
6. Statistical physics also uses graphs. In this field graphs can represent local connections between interacting parts of a system, as well as the dynamics of a physical process on such systems.
7. In computer network, the relationships among interconnected computers within the network, follow the principles of graph theory.
8. Graphs are used to represent the routes between the cities.

A graph is a non-linear data structure, which consists of vertices (or nodes) connected by edges (or arcs) where edges may be directed or undirected.



- In Computer science graphs are used to represent the flow of computation.
- Google maps uses graphs for building transportation systems, where intersection of two (or more) roads are considered to be a vertex and the road connecting two vertices is considered to be an edge, thus their navigation system is based on the algorithm to calculate the shortest path between two vertices.
- In Facebook, users are considered to be the vertices and if they are friends then there is an edge running between them. Facebook's Friend suggestion algorithm uses graph theory. Facebook is an example of undirected graph.
- In World Wide Web, web pages are considered to be the vertices. There is an edge from a page u to another page v if there is a link of page v on page u . This is an example of Directed graph.
- In Operating System, we come across the Resource Allocation Graph where each process and resources are considered to be vertices. Edges are drawn from resources to the allocated process, or from requesting process to the requested resource. If this leads to any formation of a cycle then a deadlock will occur.
- Flight Networks.
For flight networks, efficient route optimizations perfectly fit graph data structures. Using graph models, airport procedures can be modelled and optimized efficiently. Computing best connections in flight networks is a key application of algorithm engineering. In flight network, graph data structures are used to compute shortest paths and fuel usage in route planning, often in a multi-modal context. The vertices in flight networks are places of departure and destination, airports, aircrafts, cargo weights.